



Unicity Yellowpaper

The Unicity Team

Specification

May 22, 2026

Version TestNet V1 (<not in>/<git repo>)

May 22, 2026

© Unicity Labs, 2026

This work is licensed under a Creative Commons “Attribution 4.0 International” license.



Contents

1	General Description	9
1.1	System Architecture	9
2	Execution Layer	10
2.1	Execution Model	10
2.1.1	Token Lifecycle	10
2.1.2	Token State	11
2.1.3	Token Structure	11
2.1.3.1	Mint Transaction	11
	Certifying a mint transaction	12
	Verifying a certified mint transaction	12
2.1.3.2	Transfer Transaction	12
	Certifying a transfer transaction	13
	Verifying a certified transfer transaction	13
2.1.4	Token Verification	13
2.2	Programmable Spending Conditions	14
2.2.1	Predicate Framework	14
2.2.1.1	Time Reference	14
	Per-round fixed reference time.	14
	BFT Core enforcement.	14
	Extraction from inclusion proofs.	14
2.2.2	Standard Predicates	15
2.2.2.1	Signature Predicate	15
2.2.2.2	P2PKH Predicate	15
2.2.2.3	P2SH Predicate	15
2.2.2.4	Multi-Signature Predicate	15
2.2.2.5	Threshold-Signature Predicate	16
2.2.2.6	Burn Predicate	16
2.2.2.7	Timelock Predicate	16
2.2.2.8	Hashed Timelock Predicate	17
2.3	Interaction with Unicity Service	17
2.3.1	Certification Request	17
2.3.2	Processing	17
2.4	Transaction Flow	18
2.5	Fungible Token Payload	19
2.5.1	Asset Structure	19
2.5.2	Mint Reason and Initial Payload	19
2.5.3	Token Splitting	19
2.5.3.1	Split Protocol	19
2.5.3.2	Split Verification	20
2.5.4	Burn Transaction	20
2.6	Private Transactions	20
2.6.1	Interactive Protocol	21

2.6.2	Non-Interactive Protocol	22
3	Unicity Certificate Framework	24
3.1	Sharding Schemes	24
3.2	Networks	24
3.3	Partitions and Shards	24
3.4	Partition Type and Partition Type Descriptor	25
3.4.1	Partition Type Descriptor	25
3.4.2	Standard Partition Types	25
3.4.3	Partition Description Record	25
3.5	Shard Split	26
	Partition description update.	26
	Aggregator state handover.	26
	BFT Core state handover.	26
	First post-split round.	26
3.6	Unicity Service Request	27
3.7	Unicity Service Request Validation	27
3.7.1	Validation and Processing Algorithm	27
3.8	Certificates	28
3.8.1	Sparse Merkle Tree	29
	3.8.1.1 SMT Leaf Structure	29
	3.8.1.2 SMT Certificates and Updates	29
3.8.2	Shard Tree Certificate	29
	3.8.2.1 Sharded SMT Structure	30
	3.8.2.2 Creation: CreateShardTreeCert	30
	3.8.2.3 Computation: CompShardTreeCert	30
3.8.3	Unicity Tree Certificate	30
	3.8.3.1 Creation: CreateUnicityTreeCert	31
	3.8.3.2 Computation: CompUnicityTreeCert	31
3.8.4	Unicity Seal	31
3.8.5	Unicity Certificate	32
	3.8.5.1 Verication: VerifyUnicityCert	32
3.9	Proofs	32
3.9.1	Inclusion Proof	32
	3.9.1.1 Verification: VerifyInclusionProof	33
3.9.2	Non-Inclusion Proof	33
	3.9.2.1 Verification: VerifyNonInclusionProof	33
3.9.3	Consistency Proof	34
	3.9.3.1 Security Model	34
	3.9.3.2 Verification Function	35
4	BFT Core	37
4.1	Data Structures of the BFT Core	37
4.1.1	Shard Input Record	37
4.1.2	Statistical Record	37
4.1.3	Technical Record	37
4.1.4	Shard Info	38
4.1.5	Shard Tree	38

4.1.5.1	Shard Tree Creation: CreateShardTree	38
4.1.6	Unicity Tree	39
4.1.6.1	Creation: CreateUnicityTree	39
4.2	State of the BFT Core	39
4.3	Messages of the BFT Core	39
4.3.1	Certification Request	39
4.3.2	Certification Response	40
4.4	Functional Description of the BFT Core	40
5	Consensus Layer: BFT	43
5.1	Background	43
5.1.1	Definitions	43
5.1.2	Scope	43
5.1.3	Repeating Notation	44
5.2	Aggregation Layer	44
5.2.1	Timing	44
5.2.2	Configuration and State	45
5.2.3	Subcomponents	48
5.2.3.1	Input Handling	48
5.2.3.2	Block Proposal	49
5.2.3.3	Validation and Execution	49
5.2.3.4	Processing an Unicity Certificate and Finalizing a Round	52
5.2.3.5	Processing a Block Proposal	53
5.2.3.6	Ledger Replication	55
5.2.4	Recovery Procedure	55
5.2.5	Protocols – Shard Validators	57
5.2.5.1	Protocol TransactionMsg – Unicity Service Request De- livery	57
5.2.5.2	Protocol CR – Round Certification Request	57
5.2.5.3	Protocol RoundCertificationResponse (CReS)	59
5.2.5.4	Protocol Subscription – subscribing to CReS messages	59
5.2.5.5	Protocol InputForwardMsg – Input Forwarding	59
5.2.5.6	Protocol BlockProposalMsg – Block Proposal	59
5.2.5.7	Protocol LedgerReplication – Ledger Replication	59
5.2.5.8	Protocol GetTrustBase – Unicity Trust Base Distribution	60
5.3	BFT Core State Machine	61
5.3.1	Summary	61
5.3.2	Timing	61
5.3.3	State	62
5.3.4	Analysis	63
5.3.4.1	Safety	63
5.3.4.2	Liveness	63
5.3.4.3	Data Availability	63
5.3.5	Monolithic Implementation	64
5.3.5.1	Certification Request Processing	65
5.3.5.2	Unicity Certificate Generation	65
5.3.6	Distributed Implementation	65
5.3.6.1	Summary of Execution	66

	Peer Node Selection	66
	CR Validation	67
	Shard Quorum Check	67
	IR Change Request Validation	67
	Proposal Generation	67
	Proposal Validation	70
	State Signing	70
	UC Generation	70
5.3.6.2	Proposal	70
	State Synchronization	70
5.3.6.3	Atomic Broadcast Primitive	71
	Round Pipeline	72
	Pacemaker	72
	Leader Election	72
5.4	Dynamic System	72
5.4.1	Configuration Changes	72
5.4.2	BFT Core Epoch Change	73
5.4.3	Shard Epoch Change	74
5.4.4	Controlling Shard Epochs	74
5.4.5	Validator's life cycle	75
5.4.5.1	Shard Validator, joining	75
5.4.5.2	Shard Node, leaving	76
5.4.5.3	Shard Node, ambiguous records	77
5.4.5.4	BFT Core Node, joining	77
5.4.5.5	BFT Core Node, leaving	77
5.4.6	Dynamic Data Structures	78
5.4.6.1	Versioning	78
5.4.6.2	Evolving	78
5.4.6.3	Monolithic, Static BFT Core	78
5.4.6.4	Monolithic, Dynamic BFT Core	79
5.4.6.5	Distributed, Static BFT Core	80
5.4.6.6	Distributed, Dynamic BFT Core	81
5.4.6.7	Signature Aggregation	81
6	PoW Chain	83
6.1	PoW Blockchain	83
6.1.1	Motivation	83
6.1.2	Longest Chain Rule	84
6.1.3	RandomX Proof-of-Work	84
6.1.4	Block Structure	84
6.1.4.1	Block Header	85
	Block hash.	85
6.1.4.2	Block Payload	85
6.1.5	Difficulty Adjustment	86
6.1.6	Mining	86
6.1.6.1	Block Mining	86
6.1.6.2	Block Validation	87
6.1.6.3	Block Reward Token Minting	87

6.1.7	Network Parameters	87
6.2	Snapshot Publishing	87
6.2.1	Data Flow	91
6.2.2	Snapshot Generation	91
6.3	Finality Gadget	91
6.3.1	Finality Gadget Partition	91
6.3.2	Operation	92
6.3.3	Obtaining Finalized PoW Blocks	93
6.4	Trust Model	94
6.4.1	Maximalist Token Verification	94
6.4.2	Practical Token Verification	94
6.4.3	Comparison	96
A	Bitstrings, Orderings, and Codes	97
A.1	Bitstrings and Orderings	97
A.2	Prefix-Free Codes	97
B	Encodings	98
B.1	CBOR	98
B.2	Tuples	98
B.3	Bit-strings	98
B.4	Hash Values	98
B.5	Unit Identifiers	99
B.6	Time	99
B.7	Identifiers	99
B.8	Cryptographic Algorithms	99
C	Hash Trees	100
C.1	Plain Hash Trees	100
C.1.1	Function PLAIN_TREE_ROOT	100
C.1.2	Function PLAIN_TREE_CHAIN	100
C.1.3	Function PLAIN_TREE_OUTPUT	101
C.1.4	Inclusion Proofs	101
C.2	Indexed Hash Trees	102
C.2.1	Function INDEX_TREE_ROOT	102
C.2.2	Function INDEX_TREE_CHAIN	103
C.2.3	Function INDEX_TREE_OUTPUT	103
C.2.4	Inclusion and Exclusion Proofs	104
C.3	Sparse Merkle Sum Trees	104
C.3.1	Node Structure	104
C.3.2	Function SMST_LEAF_HASH	105
C.3.3	Function SMST_BRANCH_HASH	105
C.3.4	Inclusion Certificate	106
C.3.5	Function SMST_VERIFY_INCLUSION	106
C.4	Radix Sparse Merkle Trees	107
C.4.1	Tree Structure	107
C.4.1.1	Node Format	107
C.4.2	Hash Computation	107
C.4.2.1	Function RSMT_LEAF_HASH	107

	C.4.2.2	Function RSMT_NODE_HASH	108
C.4.3		Batch Insertion	108
	C.4.3.1	Function RSMT_BATCH_INSERT_NP	108
C.4.4		Shard Split	110
	C.4.4.1	Function RSMT_SHARD_SPLIT	110
C.4.5		Inclusion Certificate	111
	C.4.5.1	Format	111
	C.4.5.2	Generation	111
	C.4.5.3	Function RSMT_VERIFY_INCLUSION	112
C.4.6		Non-Inclusion Certificate	112
	C.4.6.1	Format	112
	C.4.6.2	Generation	113
	C.4.6.3	Function RSMT_VERIFY_NON_INCLUSION	113
C.4.7		Consistency Proof	114
	C.4.7.1	Sort Order	114
	C.4.7.2	Proof Format	114
	C.4.7.3	Generation	114
	C.4.7.4	Function RSMT_VERIFY_CONSISTENCY	115
C.4.8		ZK-Compressed Consistency Proofs	116
	C.4.8.1	ZK Proof Structure	116
	C.4.8.2	Verification	116
D		Unicity Token Encoding	118
D.1		Constants	118
D.2		Predicate Encoding	118
	D.2.1	Built-in Predicate Type Codes	119
	D.2.2	Unlocking Argument Encoding	119
	D.2.3	Predicate Soundness	120
	D.2.4	Recipient Field	121
D.3		Transaction Encoding	121
	D.3.1	Mint Transaction	121
	D.3.2	Transfer Transaction	121
	D.3.3	Certified Transaction	122
	D.3.4	Token	122
D.4		Hash Derivations	122
	D.4.1	Token Identifier	122
	D.4.2	State Identifier	122
	D.4.3	State Hash	122
	D.4.4	Transaction Hash	123
D.5		Signature Format	123
D.6		Split Proof Encoding	123
	D.6.1	Split Reason	123
	D.6.2	Split Reason Proof	123
E		Data Structures (illustrative)	124
Index			128

1 General Description

1.1 System Architecture

The Unicity protocol employs a hierarchical three-layer architecture designed for scalability and decentralization. Each layer serves a distinct function in the system:

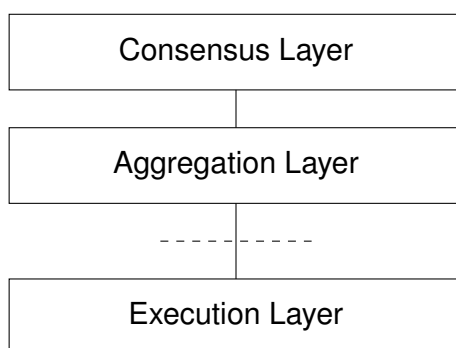


Figure 1. Hierarchical architecture of the Unicity Network.

Consensus Layer PoW chain and BFT Core. Provides token minting, decentralized agreement and finality through Byzantine Fault Tolerant consensus. This layer verifies the integrity of the Aggregation Layer's state transitions and serves as the root of trust for the entire system. The BFT Core certifies SMT root hashes submitted by the aggregation layer.

Aggregation Layer Maintains a global append-only registry of spent token states using sharded Sparse Merkle Trees (SMT). Each partition within this layer processes state certification requests, provides inclusion proofs, and uses consistency proofs to enable trustless verification. The layer is organized into partitions for service design flexibility, with each partition potentially divided into shards for horizontal scalability.

Execution Layer Handles off-chain transaction processing and business logic. Token transactions occur peer-to-peer between users, with only cryptographic commitments to state transitions submitted to the Aggregation Layer for certification. This design moves computational overhead off-chain while maintaining trustless double-spending prevention.

The BFT Core certifies state transitions by cryptographically signing the root hashes of the Aggregation Layer's SMTs. This hierarchical design enables linear scalability: the BFT Core's throughput requirements grow only with the number of shards, not with the number of transactions processed by the system.

2 Execution Layer

The execution layer handles off-chain transaction processing and business logic. Tokens are self-contained digital assets transferred peer-to-peer between users. Only cryptographic commitments to state transitions are submitted to the Aggregation Layer for double-spending prevention. Transaction execution and verification are performed by the interested parties (recipients), not by consensus participants. This design moves computational overhead off-chain while maintaining trustless guarantees against double-spending.

2.1 Execution Model

In the Unicity execution model, digital assets called *tokens* are bearer instruments: whoever holds the token data and can satisfy the spending condition is the owner. Transactions occur directly between parties using any communication channel, with the Unicity Service consulted only to prevent double-spending.

The model relies on three key principles:

1. **Off-chain execution:** Transaction creation, validation, and verification happen between sender and recipient without consensus participation.
2. **On-chain uniqueness:** The Unicity Service maintains an append-only registry of spent token states, ensuring each state can be spent at most once.
3. **Recipient verification:** The recipient (relying party) independently verifies the complete token history before accepting a transfer.

2.1.1 Token Lifecycle

A token progresses through the following stages:

Minting Creates a new token with an initial state. Minting uses a publicly known minting key pair (sk_{mint} , pk_{mint}) which MAY be network specific.

Transfer Changes ownership by spending the current state and creating a new state with a new owner predicate.

Compression Compacts the token by replacing a set of transactions with a cryptographic proof of validity of these transactions.

Split (Fungible tokens only.) Burns the original token and mints new tokens, with a cryptographic value conservation proof ensuring no value is created or destroyed. See Sec. 2.5.3.

Burn Permanently locks a token by transferring it to a burn predicate that cannot be unlocked. See Sec. 2.5.4.

2.1.2 Token State

A *token state* is a pair (ν, h_{st}) , where:

- ν – the spending predicate (defines who can spend the token); see Sec. 2.2
- $h_{\text{st}} \in \mathbb{H}$ – the state hash, linking the token’s transaction history

The *state identifier* is the key used in the Unicity Service registry:

$$\text{sid} = H(\nu, h_{\text{st}})$$

When a transaction with random value $x \leftarrow \{0, 1\}^\ell$ is executed, the next state hash is computed as:

$$h_{\text{st}}' = H(h_{\text{st}}, x)$$

The randomness of *state mask* x ensures that the next state identifier $H(\nu', h_{\text{st}}')$ is unpredictable, preventing the Unicity Service from linking consecutive states of the same token.

The state mask x MUST be sampled with at least 128 bits of min-entropy.

2.1.3 Token Structure

A *token* is a self-contained data structure carrying its complete provenance. Any party can independently verify the entire history without consulting external state. Formally, a token is a sequence:

$$\mathcal{L} = (T_0^{\text{cert}}, T_1^{\text{cert}}, \dots, T_n^{\text{cert}})$$

where T_0^{cert} is a certified mint transaction and T_i^{cert} for $i > 0$ are certified transfer transactions.

2.1.3.1 Mint Transaction

A *certified mint transaction* is a tuple (T_0, u_0, π_0) , where $T_0 = (h_{\text{st}}, D_{\text{mint}})$ and

$$D_{\text{mint}} = (\alpha, \nu', \text{salt}, \text{ty}, \text{reason}, \text{aux}').$$

- $h_{\text{st}} = H(\text{id}, \text{MINT_SUFFIX})$ – source state hash, for state identifier derivation
- D_{mint} – mint transaction data, containing in this order:
 - $\alpha \in \mathbb{A}$ – network identifier, binding the token to a specific Unicity network instance
 - ν' – spending predicate of the first owner
 - $\text{salt} \in \{0, 1\}^{256}$ – token identifier salt, chosen by the minter and required to be unique among the minter’s salt choices (e.g. by uniform random sampling or by a local counter); salt has no masking role and need not be high-entropy
 - ty – token type, classifying the token category, defining its mandatory validation rules
 - reason – mint reason; content depends on the token type. Justifies minting this token, e.g., by proving minting authority
 - aux' – auxiliary payload (application-specific; e.g. asset value for fungible tokens)

- u_0 – unlocking argument (signature with sk_{mint})
- π_0 – inclusion proof from the Unicity Service

The *token identifier* is derived as

$$\text{id} := H(\text{salt}, \alpha) \in \mathbb{I}.$$

This ensures that two tokens minted on different networks cannot induce colliding state identifiers.

MINT_SUFFIX is a fixed domain separator constant. The minting key pair $(sk_{\text{mint}}, pk_{\text{mint}})$ is a public constant; it serves only to provide a uniform request interface with the Unicity Service. Values: Appendix D.1.

Certifying a mint transaction $T_0 = (h_{\text{st}}, D_{\text{mint}})$:

1. $h_{\text{tx}} \leftarrow H(D_{\text{mint}})$
2. $h_T \leftarrow H(h_{\text{st}}, h_{\text{tx}})$
3. $\sigma \leftarrow \text{Sign}(sk_{\text{mint}}, h_T)$
4. $Q \leftarrow (pk_{\text{mint}}, h_{\text{st}}, h_{\text{tx}}, \sigma)$
5. $\pi_0 \leftarrow \text{US}(Q)$

Verifying a certified mint transaction (T_0, u_0, π_0) against trust base $\mathcal{T}$, using known, fixed mint keys:

1. require $T_0.D_{\text{mint}}.\alpha = \pi_0.UC.C^r.\alpha = \mathcal{T}.\alpha$
2. derive $\text{id} \leftarrow H(T_0.D_{\text{mint}}.\text{salt}, T_0.D_{\text{mint}}.\alpha)$; require $T_0.h_{\text{st}} = H(\text{id}, \text{MINT_SUFFIX})$
3. $h_{\text{tx}} \leftarrow H(D_{\text{mint}})$; require $h_{\text{tx}} = \pi_0.h_{\text{tx}}$
4. $\text{Verify}(pk_{\text{mint}}, H(h_{\text{st}}, h_{\text{tx}}), u_0) = 1$
5. $\mathcal{V}(H(pk_{\text{mint}}, h_{\text{st}}), h_{\text{tx}}, \pi_0, \mathcal{CD}_\beta, \mathcal{T}) = 1$
6. Token type-specific checks on reason and aux' (e.g. mint reason validity and auxiliary payload format and semantics check)

Correctness of the token id derivation is enforced implicitly by step 2: h_{st} is bound to id by the source-state-hash check, and id is recomputed from (salt, α) .

2.1.3.2 Transfer Transaction

A *certified transfer transaction* is a tuple (T, u, π) , where $T = (h_{\text{st}}, D)$:

- $D = (v', x, \text{aux}')$ – transaction data:
 - v' – spending predicate of the next owner
 - $x \leftarrow \{0, 1\}^\ell$ – random value for state hash derivation
 - aux' – auxiliary data for the next state
- u – unlocking argument satisfying the current predicate v
- π – inclusion proof from the Unicity Service

Certifying a transfer transaction $T = (h_{st}, D)$ in the current state (v, h) :

1. $h_{tx} \leftarrow H(D)$
2. $h_T \leftarrow H(h_{st}, h_{tx})$
3. Create u such that $v(\tau_{exp}, h_T, u) = 1$ for an expected system time τ_{exp}
4. $Q \leftarrow (v, h_{st}, h_{tx}, u)$
5. $\pi \leftarrow US(Q)$

Verifying a certified transfer transaction (T, u, π) in state (v, h) :

1. $T.h_{st} = h$ (state hash matches expected)
2. $\pi.UC.C^r.\alpha = \mathcal{T}.\alpha$ (inclusion proof is from the same network as the trust base)
3. $h_{tx} \leftarrow H(T.D)$; require $h_{tx} = \pi.h_{tx}$ (transaction data binds to the certified SMT leaf)
4. $v(\text{time}(\pi), H(h_{st}, h_{tx}), u) = 1$ (predicate satisfied)
5. $\mathcal{V}(H(v, T.h_{st}), h_{tx}, \pi, CD_\beta, \mathcal{T}) = 1$ (inclusion proof valid and certified by the correct shard)
6. Additional token type-specific checks

2.1.4 Token Verification

VerifyToken – token verification function of type $\mathbb{L} \times \mathbb{UB} \rightarrow \mathbb{B}$

Input:

1. $\mathcal{L} = (T_0^{\text{cert}}, T_1^{\text{cert}}, \dots, T_n^{\text{cert}})$ – token
2. $\mathcal{T}$ – trust base

Output: TRUE OR FALSE

Computation:

```

parse  $T_0^{\text{cert}}$  as  $(T_0, u_0, \pi_0)$ 
if  $T_0.D_{\text{mint}}.\alpha \neq \mathcal{T}.\alpha$  then return FALSE
end if
if  $\neg \text{VERIFYMINTTRANSACTION}(T_0, u_0, \pi_0, \mathcal{T})$  then return FALSE
end if
 $v^0 \leftarrow T_0.D_{\text{mint}}.v'$ 
 $h^0 \leftarrow H(H(T_0.D_{\text{mint}}.\text{salt}, T_0.D_{\text{mint}}.\alpha), \text{MINT\_SUFFIX})$ 
for  $i \leftarrow 1 \dots n$  do
  parse  $T_i^{\text{cert}}$  as  $(T_i, u_i, \pi_i)$ 
  if  $\neg \text{VERIFYTRANSFERTRANSACTION}(T_i, u_i, \pi_i, v^{i-1}, h^{i-1}, \mathcal{T})$  then return FALSE
  end if
   $v^i \leftarrow T_i.D.v'$ 
   $h^i \leftarrow H(h^{i-1}, T_i.D.x)$ 
end for
return TRUE

```

VERIFYMINTTRANSACTION and VERIFYTRANSFERTRANSACTION are defined by verification steps in chapters 2.1.3.1, 2.1.3.2.

2.2 Programmable Spending Conditions

Instead of restricting ownership to a single public key, the Unicity execution layer generalizes ownership to *predicates* – programmable boolean functions that define the conditions under which a token state can be spent. This enables smart-contract-like functionality executed off-chain by relying parties rather than by consensus participants.

2.2.1 Predicate Framework

A *predicate* $\nu_{\text{name}}(\text{par}; \cdot)$ is a boolean function with:

- **Name** *name* – identifies the predicate type
- **Parameters** *par* – type-dependent configuration (e.g. a public key)
- **Arguments** (τ, m, u) :
 - $\tau \in \mathbb{T}$ – system time, extracted from the Unicity Seal timestamp $C^r.t_r$
 - $m \in \mathbb{H}$ – message hash $H(h_{\text{st}}, h_{\text{tx}})$
 - u – unlocking argument, structure depends on predicate type

The predicate evaluates to $\nu_{\text{name}}(\text{par}; \tau, m, u) \in \{0, 1\}$.

2.2.1.1 Time Reference

Predicate evaluation must be deterministic and transparent: the Unicity Service (at certification time) and any later verifier (re-validating from an inclusion proof) must agree on the exact value of τ used. The system therefore fixes τ per Aggregator round, with the value coming through BFT Core consensus.

Per-round fixed reference time. The BFT Core’s Unicity Seal carries a wall-clock round-creation timestamp $C^r.t_r$ (Sec. 3.8.5), with one-second precision, agreed by BFT Core consensus. When Aggregator assembles a certification request for its round $n+1$, it copies the timestamp from the Unicity Seal of the *previous* UC it is extending into the Shard Input Record field $\mathcal{IR}.t$ (Sec. 4.1.1):

$$\text{CR}.\mathcal{IR}.t \leftarrow \text{UC}_-.C^r.t_r$$

where UC_- is the last valid Unicity Certificate of that shard (which the Aggregator is *extending*). This is the *reference time for request validation* of the round being validated. All predicates evaluated against Unicity Service requests in that round use this single value as τ .

BFT Core enforcement. The BFT Core verifies the Aggregator’s choice of reference time as part of request admission (Sec. 4.3.1, check $\text{CR}.\mathcal{IR}.t = \mathcal{SI}[\beta, \sigma].\text{UC}_-.C^r.t_r$).

Extraction from inclusion proofs. Given an inclusion proof $\pi^{\text{inc}} = (\text{sid}, h_{\text{tx}}, C^{\text{inc}}, \text{UC})$, the *system time* used for predicate evaluation is extracted as

$$\text{time}(\pi^{\text{inc}}) = \text{UC}.\mathcal{IR}.t$$

This is the same value the Aggregator used when admitting the request – the previous seal's t_r , copied into $\mathcal{IR}.t$ and BFT-verified. Re-validation by any later verifier is therefore deterministic and yields the identical predicate evaluation outcome.

Note that $UC.\mathcal{IR}.t \neq UC.C^r.t_r$ in general: $\mathcal{IR}.t$ is the reference time of the round *being* certified (taken from the prior seal), whereas $C^r.t_r$ is the wall-clock time at which the new seal itself was produced. Predicate evaluation uses $\mathcal{IR}.t$.

2.2.2 Standard Predicates

2.2.2.1 Signature Predicate

The *signature predicate* $\nu_{\text{sig}}(\text{pk}; \cdot)$ with a single public key pk as parameter is equivalent to digital signature verification. This is the basic “pay to public key” ownership model.

- **Parameters:** pk (public key)
- **Unlocking argument:** $u = \sigma$ (digital signature)
- **Definition:** $\nu_{\text{sig}}(\text{pk}; \tau, m, \sigma) = 1$ iff $\text{Verify}(\text{pk}, m, \sigma) = 1$

2.2.2.2 P2PKH Predicate

The *Pay-to-Public-Key-Hash* predicate $\nu_{\text{p2pkh}}(h_{\text{pk}}; \cdot)$ stores only the hash of the public key, requiring the spender to reveal the full public key alongside the signature.

- **Parameters:** $h_{\text{pk}} \in \mathbb{H}$ (public key hash)
- **Unlocking argument:** $u = (\text{pk}, \sigma)$
- **Definition:** $\nu_{\text{p2pkh}}(h_{\text{pk}}; \tau, m, (\text{pk}, \sigma)) = 1$ iff $H(\text{pk}) = h_{\text{pk}} \wedge \text{Verify}(\text{pk}, m, \sigma) = 1$

2.2.2.3 P2SH Predicate

The *Pay-to-Script-Hash* predicate $\nu_{\text{p2sh}}(h_v; \cdot)$ generalizes P2PKH by storing the hash of an arbitrary predicate rather than just a public key hash. The recipient is identified by $h_v = H(\text{CBOR}(v'))$, which provides a uniform-size recipient field regardless of predicate complexity and preserves privacy by hiding the predicate type until the token is spent.

- **Parameters:** $h_v \in \mathbb{H}$ (predicate hash)
- **Unlocking argument:** $u = (v', u')$ where v' is the full predicate and u' is its unlocking argument
- **Definition:** $\nu_{\text{p2sh}}(h_v; \tau, m, (v', u')) = 1$ iff $H(\text{CBOR}(v')) = h_v \wedge v'(\tau, m, u') = 1$

When spending, the full predicate v' is revealed in the lock script field of the transfer transaction, and verifiers check that $H(\text{CBOR}(v'))$ matches the recipient hash from the previous transaction.

2.2.2.4 Multi-Signature Predicate

The *multi-signature predicate* $\nu_{\text{msig}}(\text{pk}_1, \dots, \text{pk}_n; \cdot)$ requires n valid signatures from n specified public keys (all-of- n).

- **Parameters:** $(\text{pk}_1, \dots, \text{pk}_n)$

- **Unlocking argument:** $u = (\sigma_1, \dots, \sigma_n)$
- **Definition:** $\nu_{\text{msig}}(\text{pk}_1, \dots, \text{pk}_n; \tau, m, u) = 1$ iff $\text{Verify}(\text{pk}_i, m, \sigma_i) = 1$ for every $i \in \{1, \dots, n\}$

2.2.2.5 Threshold-Signature Predicate

The *threshold-signature predicate* $\nu_{\text{tsig}}(k, \text{pk}_1, \dots, \text{pk}_n; \cdot)$ requires k -of- n valid signatures from distinct public keys.

- **Parameters:** $(k, \text{pk}_1, \dots, \text{pk}_n)$ where $k \in \{1, \dots, n\}$
- **Unlocking argument:** $u = ((\sigma_1, j_1), \dots, (\sigma_k, j_k))$ where j_i is the index of the public key used for σ_i
- **Definition:** $\nu_{\text{tsig}}(k, \text{pk}_1, \dots, \text{pk}_n; \tau, m, u) = 1$ iff:
 1. $\text{Verify}(\text{pk}_{j_i}, m, \sigma_i) = 1$ for every $i \in \{1, \dots, k\}$
 2. All indices j_i are distinct: $j_i = j_{i'}$ implies $i = i'$

The distinctness requirement prevents satisfying the predicate by presenting k copies of the same signature.

2.2.2.6 Burn Predicate

The *burn predicate* $\nu_{\text{burn}}(h_{\text{reason}})$ cannot be unlocked. A token transferred to a burn predicate is permanently spent.

- **Parameters:** $h_{\text{reason}} \in \mathbb{H}$ (reason hash, linking the burn to its purpose)
- **Unlocking argument:** none
- **Definition:** $\nu_{\text{burn}}(h_{\text{reason}}; \tau, m, u) = 0$ for all arguments

The burn predicate is used in token splitting (Sec. 2.5.3), where the reason hash references the aggregation root of the split proof.

2.2.2.7 Timelock Predicate

The *timelock predicate* $\nu_{\text{tlock}}(\text{pk}, \tau_{\text{min}}; \cdot)$ prevents spending until the system time reaches a specified threshold. This is used to implement e.g. vesting schedules and lock-up periods for tokens.

- **Parameters:** $(\text{pk}, \tau_{\text{min}})$ where pk is a public key and $\tau_{\text{min}} \in \mathbb{T}$ is the unlock time
- **Unlocking argument:** $u = \sigma$ (digital signature)
- **Definition:** $\nu_{\text{tlock}}(\text{pk}, \tau_{\text{min}}; \tau, m, \sigma) = 1$ iff $\tau \geq \tau_{\text{min}} \wedge \text{Verify}(\text{pk}, m, \sigma) = 1$

The token remains locked until system time $\tau \geq \tau_{\text{min}}$, after which it can be spent by producing a valid signature under pk and forming a witness unlocking the predicate and appending a spending transaction to the token ledger.

2.2.2.8 Hashed Timelock Predicate

The *hashed timelock predicate* $\nu_{\text{htlc}}(\text{pk}, \text{pk}', y, \tau_{\text{max}}; \cdot)$ combines hash-locked and time-locked spending paths. The parameter pk is the refund key (original sender), pk' is the claim key (intended recipient), $y \in \mathbb{H}$ is a hash lock, and $\tau_{\text{max}} \in \mathbb{T}$ is a timeout.

- **Parameters:** $(\text{pk}, \text{pk}', y, \tau_{\text{max}})$
- **Unlocking argument:** $u = (x, \sigma)$ where x is a preimage candidate and σ is a digital signature
- **Definition:** $\nu_{\text{htlc}}(\text{pk}, \text{pk}', y, \tau_{\text{max}}; \tau, m, (x, \sigma)) = 1$ iff (a) or (b):
 - (a) *Claim path:* $\text{Verify}(\text{pk}', m, \sigma) = 1 \wedge y = H(x) \wedge \tau \leq \tau_{\text{max}}$
 - (b) *Refund path:* $\text{Verify}(\text{pk}, m, \sigma) = 1 \wedge \tau > \tau_{\text{max}}$

Clause (a) allows the recipient to claim the token before the timeout by revealing the H -preimage x of y and signing with pk' . Clause (b) allows the sender to reclaim the token after the timeout by signing with pk .

Example use-case is atomic swap, where two HTLC predicates on different ledgers/partitions share the same hash lock y ; claiming on one ledger reveals x , enabling the counterparty to claim on the other. Note that this assumes ledgers to be public, which is not always the case on p2p networks.

2.3 Interaction with Unicity Service

This section describes how the execution layer interacts with the Unicity Service (Sec. 3.7) for state transition certification.

2.3.1 Certification Request

A certification request from the execution layer is a tuple $Q = (\nu, h_{\text{st}}, h_{\text{tx}}, u)$, where:

- ν – spending predicate of the current owner
- h_{st} – current state hash
- h_{tx} – transaction data hash
- u – unlocking argument satisfying ν

2.3.2 Processing

The Unicity Service processes the request as follows:

1. Compute state identifier: $\text{sid} \leftarrow H(\nu, h_{\text{st}})$
2. Compute message hash: $m \leftarrow H(h_{\text{st}}, h_{\text{tx}})$
3. Evaluate predicate: verify $\nu(\tau, m, u) = 1$
4. Check uniqueness: verify $R[\text{sid}] = \perp$
5. If both checks pass: $R[\text{sid}] \leftarrow h_{\text{tx}}^1$

¹The current SDK implementation stores $H(\text{CBOR}(\nu, h_{\text{st}}, h_{\text{tx}}, u))$ as the leaf value instead of h_{tx} , because of backward compatibility with the legacy SDK.

If the request is accepted, the Unicity Service returns an inclusion proof π_{inc} . If the state has already been spent (uniqueness check fails) or the predicate evaluation fails, the request is rejected.

2.4 Transaction Flow

Figure 2 illustrates one complete token transfer, from the recipient preparing ownership information through certification to final verification.

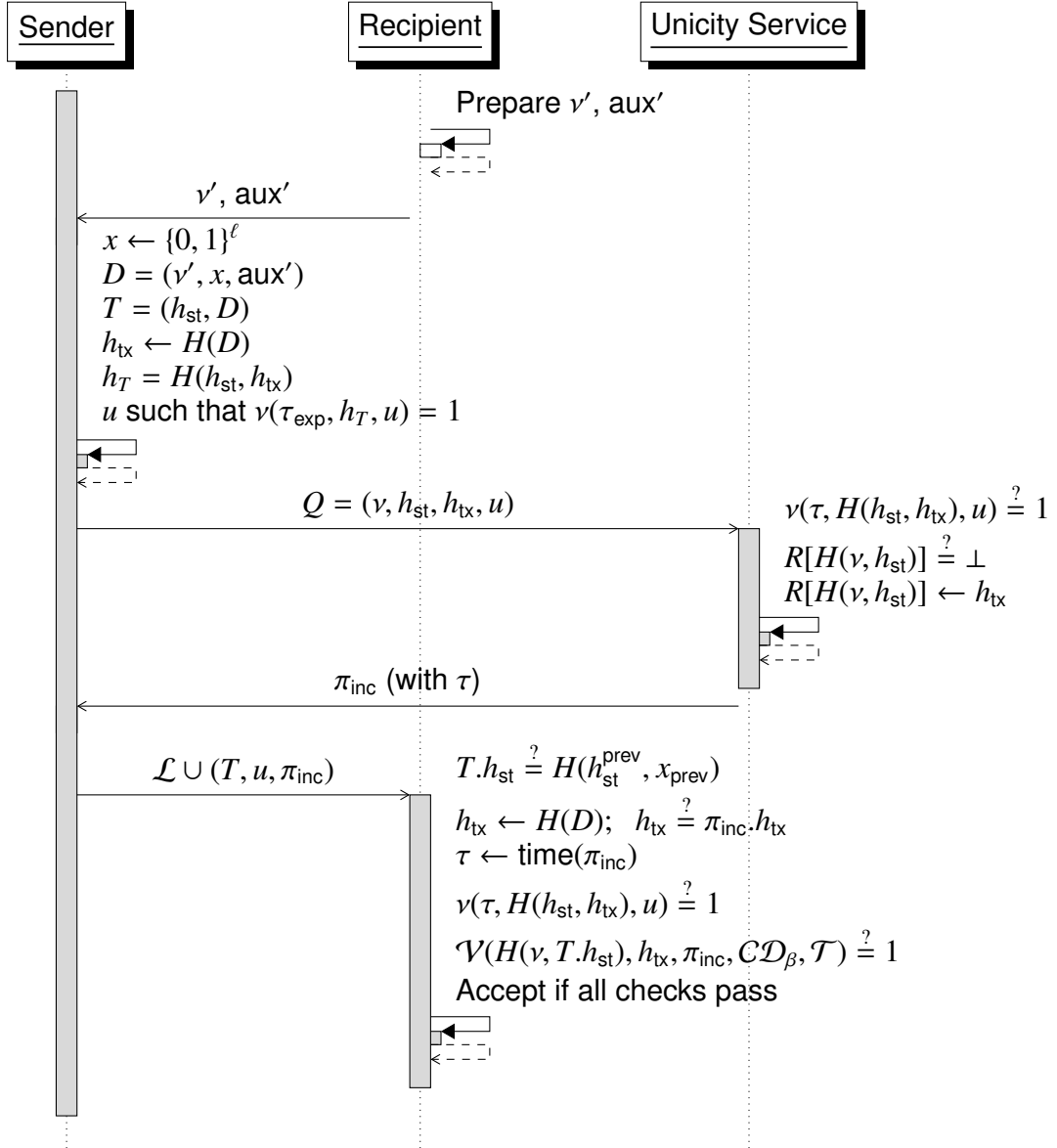


Figure 2. One Unicity transaction. The predicate v represents the current ownership condition, and the capability to unlock v' defines the new owner. System time τ is extracted from the inclusion proof and used in predicate evaluation by future verifiers.

2.5 Fungible Token Payload

For fungible tokens (e.g. digital currency), the execution layer defines additional structure within the token's auxiliary data to support money-like operations: value tracking, splitting, and value conservation proofs.

2.5.1 Asset Structure

An *asset* is a pair (aid, v) , where:

- aid – asset identifier, specifying the asset type (e.g. currency)
- $v \in \mathbb{N}$ – asset value (non-negative integer amount)

A token may carry multiple assets, represented as a collection:

$$\mathcal{A} = \{(aid_1, v_1), \dots, (aid_k, v_k)\}$$

where all asset identifiers aid_i are distinct.

2.5.2 Mint Reason and Initial Payload

The following fields provide additional meta-data necessary for token validation:

- *reason* – mint justification, such as backing proof, authorization data, or a split reason proof
- *aux'* – initial auxiliary payload, e.g., encapsulated asset metadata

The token type identifier ty in the mint transaction classifies the token (e.g. a specific currency or asset class) and defines the validation rules which both *reason* and *aux'* must satisfy and provide all sufficient data.

2.5.3 Token Splitting

Splitting allows a fungible token to be divided into multiple new tokens while provably preserving total value. The protocol uses *Sparse Merkle Sum Trees* to provide cryptographic value conservation proofs.

2.5.3.1 Split Protocol

Given a token $\mathcal{L}$ bound to network identifier α and carrying asset collection $\mathcal{A}$, the owner splits it into k new tokens on the same network α with respective asset allocations $\mathcal{A}_1, \dots, \mathcal{A}_k$ such that for every asset identifier aid :

$$\sum_{j=1}^k v_j(aid) = v(aid)$$

where $v(aid)$ is the original value and $v_j(aid)$ is the value allocated to token j .

The split proceeds as follows:

1. For each asset identifier $\text{aid} \in \mathcal{A}$, construct a *Sparse Merkle Sum Tree* $\mathcal{S}_{\text{aid}}$ with leaves keyed by new token identifiers $\text{id}_j = H(\text{salt}_j, \alpha)$ derived from distinct minter-chosen salts $\text{salt}_j \in \{0, 1\}^{256}$ (uniqueness among the minter's choices required; no masking role, no min-entropy requirement), and valued by the respective asset amounts $v_j(\text{aid})$.
2. Construct an *aggregation tree* $\mathcal{G}$ combining the root hashes of all per-asset sum trees.
3. **Burn the original token:** create a transfer transaction from $\mathcal{L}$ to the burn predicate $v_{\text{burn}}(h_{\text{agg}})$, where h_{agg} is the root hash of the aggregation tree $\mathcal{G}$. Certify this burn transaction via the Unicity Service.
4. **Mint each new token** $j \in \{1, \dots, k\}$: create a mint transaction for token id_j on network α with reason set to a *split reason proof* containing:
 - The burned token $\mathcal{L}$ (complete history including the burn transaction)
 - An aggregation tree path proving that aid is part of the split
 - A sum tree path proving the value allocation $v_j(\text{aid})$ and that the total value is conserved

2.5.3.2 Split Verification

Given a new token claiming to originate from a split, the verifier checks:

1. The burned source token is valid and its last recipient is $v_{\text{burn}}(h_{\text{agg}})$
2. The new token and the burned source token verify under the same trust base $\mathcal{T}$; equivalently, both mint transaction network identifiers equal $\mathcal{T}.\alpha$
3. The aggregation tree path correctly leads to h_{agg}
4. The sum tree path is valid: the leaf value matches the claimed asset amount, and the root sum equals the original token's value for this asset (value conservation)

2.5.4 Burn Transaction

A *burn transaction* is a transfer transaction where the recipient predicate is $v_{\text{burn}}(h_{\text{reason}})$ (Sec. 2.2.2.6). Since the burn predicate cannot be unlocked, the token is permanently spent and cannot be transferred further.

The reason hash h_{reason} links the burn to its purpose. In a split operation, it is the aggregation tree root hash, enabling verifiers of the resulting tokens to trace back to the burned source.

2.6 Private Transactions

In the transaction flow described above, Sec. 2.4, the Unicity Service learns only the public key and signature (predicate and unlock data in general) of the token spending party; but it is not possible to associate transactions with tokens. Looking at a token it is possible to associate transactions if made by the same party using the same key. In order to achieve perfect privacy it is necessary to generate a fresh predicate v' for every transaction. This requires generating and storing a fresh keypair per transaction, which may be impractical

when the recipient's secure storage is limited, and forces the transaction sender to obtain recipient's key for every transaction, making the transacting protocol interactive.

This section presents protocols that allow the recipient to derive unlinkable predicates from a single persistent keypair, requiring no persistent state beyond the keypair itself. Also, this persistent keypair can be used as a persistent address (recipient's identifier).

The protocols below are implemented at the "wallet" level and need no support by the execution layer and Unicity Service. User can choose selectively wheter to make fully private transactions or not.

The privacy protocols produce a transaction-specific public key pk_{tx} that the sender embeds in the predicate $v' = v_{sig}(pk_{tx})$. An ephemeral public key R is included in the auxiliary data aux' of the transaction, enabling the recipient to reconstruct the corresponding private key sk_{tx} when spending.

The protocols below assume an elliptic curve group of order n with generator G , and that the signature predicate v_{sig} uses a compatible scheme (e.g. ECDSA). The persistent keypair is (sk, pk) where $pk = sk \cdot G$.

2.6.1 Interactive Protocol

For each incoming transaction, the recipient generates an ephemeral blinding factor r and derives a transaction-specific public key $pk_{tx} = (sk + r) \cdot G$. The protocol is shown in Figure 3.

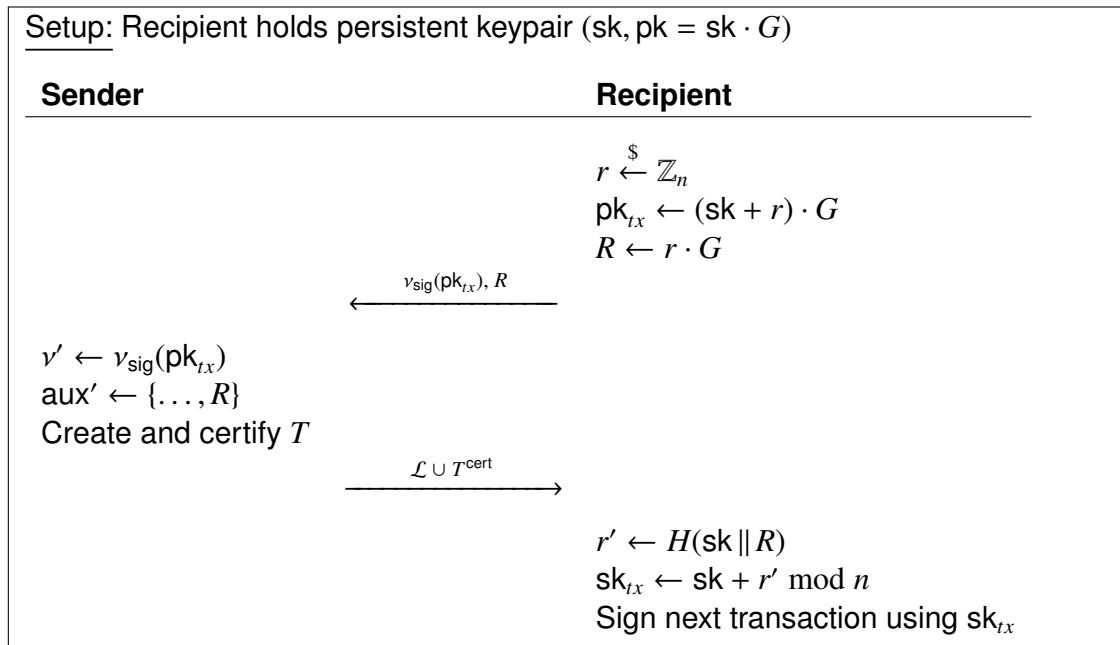


Figure 3. Interactive protocol for generating unlinkable predicates.

The recipient sends $v_{sig}(pk_{tx})$ and the ephemeral public key R to the sender, who includes R in the auxiliary data aux' of the transfer transaction. When spending the received token, the recipient reconstructs the blinding factor $r' = H(sk \parallel R)$ from the persistent private key and R (read from aux'), and computes the transaction-specific private key $sk_{tx} = sk + r' \bmod n$.

2.6.2 Non-Interactive Protocol

For applications where the recipient publishes a persistent public key pk as an address, the sender can derive unlinkable predicates without interaction using Diffie-Hellman key exchange inspired protocol, shown in Figure 4.

The key challenge is protecting against a malicious sender who might choose a predictable ephemeral key r or leak it to compromise unlinkability. The construction addresses this by binding the blinding factor to both the shared secret and public transaction data.

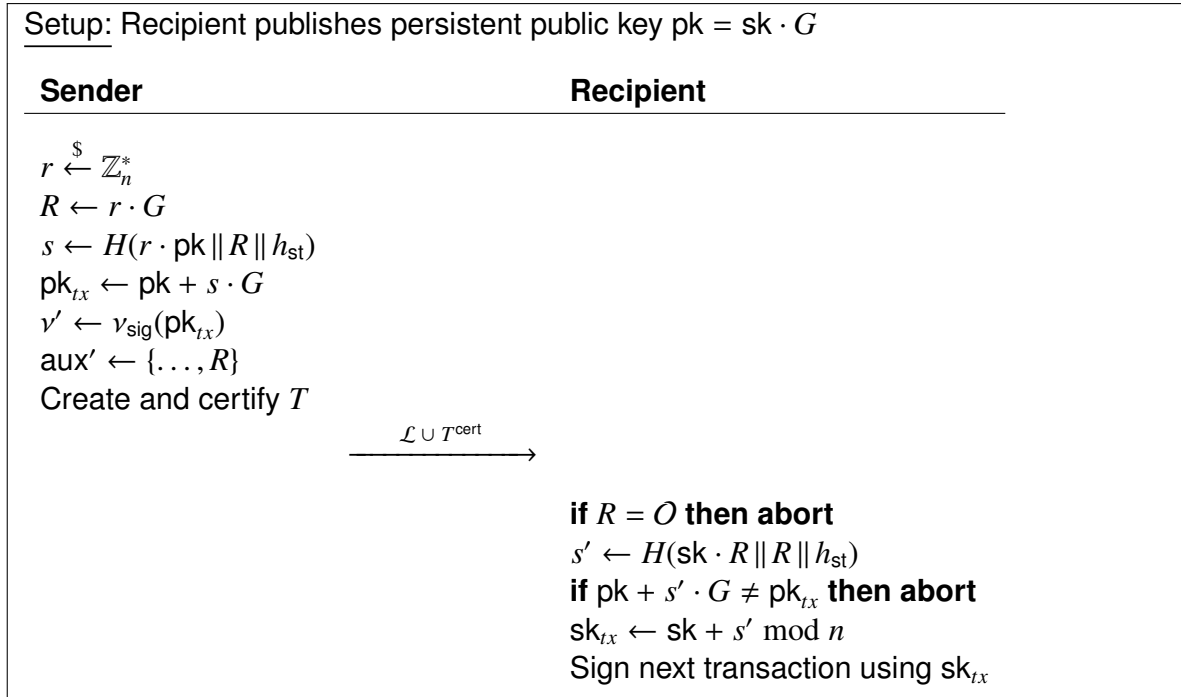


Figure 4. Non-interactive protocol for generating unlinkable predicates.

The sender generates an ephemeral keypair $(r, R = r \cdot G)$ and computes a Diffie-Hellman shared secret $r \cdot pk$. The blinding factor is derived by hashing the shared secret together with R and the current state hash h_{st} :

$$s = H(r \cdot pk \parallel R \parallel h_{st})$$

The transaction-specific public key is $pk_{tx} = pk + s \cdot G$, and R is included in aux' . The sender sets $v' = v_{sig}(pk_{tx})$ in the transaction data. Upon receiving the transaction, the recipient:

1. Verifies $R \neq O$ (not trivially insecure)
2. Computes the shared secret $sk \cdot R = r \cdot pk$ using the persistent private key
3. Derives $s' = H(sk \cdot R \parallel R \parallel h_{st})$ and verifies $pk + s' \cdot G = pk_{tx}$
4. Computes $sk_{tx} = sk + s' \bmod n$

Including both R and h_{st} in the hash input ensures that:

- Even if the sender chooses a predictable r , the blinding factor s remains unpredictable to external observers
- The recipient can verify that the sender correctly computed pk_{tx} without learning r

- Each transaction uses a unique blinding factor, since h_{st} is unique by construction (Sec. 2.1.2), preventing linkability even if the sender reuses the same r

3 Unicity Certificate Framework

3.1 Sharding Schemes

Sharding scheme $\mathcal{SH}$ of type $\mathbb{SH}$ is an irreducible prefix-free code (Appendix A).

If $\mathcal{SH} = \{\square\}$, then there is a single shard.

In the description of a sharding scheme, the shard identifiers are listed in the topological order $\sigma_1 < \sigma_2 < \dots < \sigma_n$ (Appendix A).

Every sharding scheme $\mathcal{SH}$ induces a sharding function $f_{\mathcal{SH}}: \mathbb{I} \rightarrow \mathcal{SH}$. The shard $f_{\mathcal{SH}}(\text{sid})$ responsible for recording a token state with identifier sid is the shard whose identifier σ_i is a prefix of sid . With $\mathcal{SH}$ an irreducible prefix-free code, there is exactly one such σ_i .

3.2 Networks

In practice, there may be several instances of Unicity networks, each consisting of its own Consensus Layer and its own set of partitions. Each such network will have an identifier α :

- $\alpha = 1$ is the public mainnet instance;
- $\alpha = 2$ is the public testnet instance;
- $\alpha = 3$ is a local development instance;
- the identifiers $4 \dots 8$ are reserved for future extensions;
- any additional instances of Unicity networks should use identifiers starting from 9.

To reduce the risk of confusion, it is recommended for each major deployment to use a unique identifier. However, there is no central registry to enforce this constraint.

It is strongly recommended to avoid using the identifier 0, so that an uninitialized identifier (which defaults to 0 in many programming languages) would not match any actual network.

3.3 Partitions and Shards

Unicity network's aggregation layer consists of multiple *partitions*. Each partition is identified by a partition identifier $\beta \in \mathbb{C}$. Each partition runs an unicity proving service instance (Unicity Service), or some other utility service instance².

A partition may be divided into one or more *shards* according to a sharding scheme (Sec. 3.1). Each shard is implemented by one or more *aggregation layer nodes* that main-

²Roughly, this is similar to Layer 1, Layer 2 division of Ethereum type blockchains

tain the SMT for their portion of the key space, with the invariant $\forall$ leaves $(k, \cdot)$ in shard σ 's SMT, $f_{SH}(k) = \sigma$.

3.4 Partition Type and Partition Type Descriptor

Every partition in the Unicity Framework with partition identifier β has a type that is either:

1. a *standard type* (SMT), or
2. a *defined type* that is described as a data structure of type $\mathcal{SD}$.

3.4.1 Partition Type Descriptor

A *partition type descriptor* is a tuple $\mathcal{SD} = (\text{hf}, \mathcal{CP}, V_{CP}, \gamma_{CP}) \in \mathcal{SD}$, where:

- hf – hash function identifier (e.g., SHA-256)
- $\mathcal{CP}$ – consistency proof type
- $V_{CP}: \mathbb{R} \rightarrow \mathcal{CP}$ – consistency proof computation function
- $\gamma_{CP}: \mathbb{R} \times \mathcal{CP} \rightarrow \mathbb{B}$ – consistency proof verification predicate

For the SMT-based aggregation, the partition maintains a sparse Merkle tree mapping state identifiers to transaction hashes. The hash function hf is used for all cryptographic operations, and the encoding scheme enc specifies how data structures are serialized before hashing (see Sec. 3.8.1 for SMT structure).

The *consistency proof* is an opaque data structure that allows BFT Core nodes to verify that aggregation layer shard nodes correctly performed SMT manipulations during a round. The actual mechanism and content of consistency proofs will be defined separately for each partition type.

3.4.2 Standard Partition Types

Unicity SMT ($\text{st} = 1$) – Sparse Merkle Tree based unicity prover

PoW FP ($\text{st} = 2$) – Finality Gadget partition for the PoW chain (Sec. 6.3.1)

EVM ($\text{st} = 8$) – Enshrined EVM runtime

3.4.3 Partition Description Record

A *partition description record* is a tuple $\mathcal{CD} = (\alpha, \beta, \text{st}, \mathcal{SD}, \mathcal{SH}) \in \mathcal{CD}$, where:

- $\alpha \in \mathbb{A}$ is the network identifier;
- $\beta \in \mathbb{C}$ is the partition identifier;
- $\text{st} \in \mathbb{ST}$ is the system type identifier (for standard types, $\text{st} > 0$);
- $\mathcal{SD} \in \mathcal{SD} \cup \{\perp\}$ is the system type descriptor (exists if $\text{st} = 0$);
- $\mathcal{SH} \in \mathbb{SH}$ is the sharding scheme.

Instead of $\mathcal{CD}.\mathcal{SD}.x$ (where x is any field of $\mathcal{SD}$), we use the shorthand notation $\mathcal{CD}.x$. For standard type partitions, the field $\mathcal{CD}.\mathcal{SD}$ does not exist. In this case, the notation $\mathcal{CD}.x$ will mean the constant value of x of the standard type st .

3.5 Shard Split

A shard σ of partition β may be split into two sub-shards $\sigma||0$ and $\sigma||1$ for scaling reasons. The split affects (i) the sharding scheme recorded in the partition description, (ii) the per-shard SMTs maintained by aggregation layer nodes, and (iii) the per-shard state tracked by the BFT Core in SI . The underlying tree operation is `RSMT_SHARD_SPLIT` (Appendix C.4.4.1).

Partition description update. Replacing σ by $\sigma||0, \sigma||1$ changes the sharding scheme SH and thus $H(CD[\beta])$, which appears as dhash in the Unicity Tree Certificate (Sec. 3.8.3). A split is configuration change executed at epoch change: a new partition description $CD[\beta]'$ with the updated SH' is activated at an epoch boundary, and the split takes effect in the first round of the new epoch. Old unicity certificates remain verifiable against the $CD[\beta]$ that was in force at the time of their issuance; verifiers must select the partition description record corresponding to $UC.C^r.e_r$ (or, alternatively, authenticate it by its dhash).

Aggregator state handover. At the epoch boundary of the split, the aggregation layer nodes serving the new shard $\sigma||b$ initialize their SMT by applying

$$T_{\sigma||b} \leftarrow \text{RSMT_SHARD_SPLIT}(T_\sigma, |\sigma|, b)$$

to the last pre-split tree T_σ (whose root was $SI[\beta, \sigma].h_-$). Let $\rho_{\sigma||b}$ denote the root hash of $T_{\sigma||b}$; if the retained subtree is empty, $\rho_{\sigma||b} = \perp$. No rehashing is required for the retained subtree (Appendix C.4.4).

BFT Core state handover. At the same epoch boundary, the BFT Core replaces $SI[\beta, \sigma]$ with two fresh entries $SI[\beta, \sigma||0]$ and $SI[\beta, \sigma||1]$. For each $b \in \{0, 1\}$:

- $SI[\beta, \sigma||b].n \leftarrow SI[\beta, \sigma].n$ and $SI[\beta, \sigma||b].e \leftarrow SI[\beta, \sigma].e + 1$;
- $SI[\beta, \sigma||b].h_- \leftarrow \rho_{\sigma||b}$, i.e. the root hash of the retained sub-shard SMT derived deterministically from the parent's last-certified root;
- $SI[\beta, \sigma||b].UC_- \leftarrow SI[\beta, \sigma].UC_-$ (the parent's last certificate remains the authenticated pre-state reference);
- the statistical records are reset as at any epoch change, and the validator set and leader for each sub-shard are assigned by the orchestration mechanism.

First post-split round. The first certification request issued by $\sigma||b$ after the split must satisfy the usual round chaining invariants of Sec. 3.7 with respect to the seeded values above. Specifically:

$$CR.IR.h' = \rho_{\sigma||b} = SI[\beta, \sigma||b].h_-,$$

and if no new leaves are inserted during that round then $CR.IR.h = \rho_{\sigma||b}$ and the batch B is empty. The consistency proof π_{CP} for this bridging round is therefore the empty-batch proof (accepted by `RSMT_VERIFY_CONSISTENCY` when $B = \langle \rangle$ and $h' = h$, Appendix C.4.7.4). Subsequent rounds proceed normally. The correctness of the $\rho_{\sigma||b}$ seeding is not proved on-chain by the child shard; it follows deterministically from the parent's last-certified root by Appendix C.4.4.1, which any validator can recompute.

3.6 Unicity Service Request

A *Unicity Service Request* is a request to the Unicity Service for state transition certification. It is a tuple $Q = (\nu, h_{\text{st}}, h_{\text{tx}}, u)$, where:

- ν – current owner’s spending predicate (Sec. 2.2)
- $h_{\text{st}} \in \mathbb{H}$ – current state hash
- $h_{\text{tx}} \in \mathbb{H}$ – transaction data hash
- u – unlocking argument satisfying ν

The Unicity Service processes the request by:

1. Evaluating the predicate: $\nu(\tau, H(h_{\text{st}}, h_{\text{tx}}), u) = 1$
2. Computing the state identifier: $\text{sid} = H(\nu, h_{\text{st}})$
3. Checking uniqueness: the state has not been spent before (i.e., sid does not already exist in the SMT)
4. If all checks pass: setting the SMT leaf $(\text{sid}, h_{\text{tx}})$ in the tree

The request is accepted without returning a synchronous response. The detailed validation algorithm will be specified in Sec. 3.7.

3.7 Unicity Service Request Validation

The aggregation layer nodes process Unicity Service Requests by validating them and, if valid, updating their SMT. Each shard maintains an SMT for its portion of the key space.

3.7.1 Validation and Processing Algorithm

Given a Unicity Service Request $Q = (\nu, h_{\text{st}}, h_{\text{tx}}, u)$, the aggregation layer node performs the following checks:

1. **Right shard check:** Compute the state identifier

$$\text{sid} = H(\nu, h_{\text{st}})$$

and verify that the current shard σ satisfies

$$f_{\mathcal{CD}[\beta].\mathcal{SH}}(\text{sid}) = \sigma.$$

If this check fails, the request was sent to the wrong shard and must be rejected (or forwarded to the correct shard).

2. **Predicate evaluation:** Verify that the unlocking argument satisfies the predicate:

$$\nu(\tau, H(h_{\text{st}}, h_{\text{tx}}), u) = 1$$

3. **Uniqueness check:** Verify that the SMT does not contain a leaf with key sid (i.e., the state has not been spent before).

If all checks are satisfied, the request is accepted and the aggregation layer node sets the SMT leaf:

Set SMT leaf: $(\text{sid}, h_{\text{tx}})$

This updates the SMT by inserting the new leaf.

If either check fails, the request is rejected:

- If predicate evaluation fails: the request is invalid and must be rejected
- If uniqueness check fails (leaf with key sid already exists): the state has already been spent (double-spend attempt) and the request must be rejected

After processing a batch of requests, the aggregation layer nodes compute the new SMT root hash h and send a certification request to the BFT Core. The BFT Core verifies the consistency proof π_{CP} and that last state is extended correctly and certifies the new root hash h via the Unicity Certificate.

3.8 Certificates

Certificates are compact proofs of inclusion (or sometimes uniqueness, or exclusion) of some data item in an authenticated data structure. Certificates may be chained. Combining certificates, it is possible to put together *proofs* (Sec. 3.9) proving e.g. inclusion, non-inclusion or uniqueness of state, consumed by associated transaction.

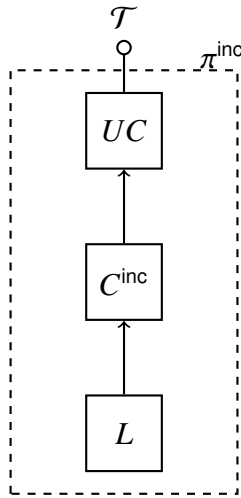


Figure 5. Inclusion Proof: SMT leaf L with certificate chain through UC to trust base $\mathcal{T}$

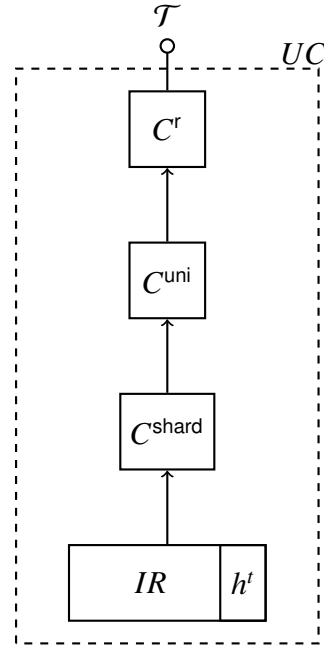


Figure 6. Unicity Certificate is a chain of certificates and a certified input data record IR

See Fig. 5 for the Inclusion Proof structure and Fig. 6 for the Unicity Certificate. The Inclusion Proof contains an SMT leaf $L = (\text{sid}, h_{\text{tx}})$ where $\text{sid} = H(v, h_{\text{st}})$ is a state identifier and h_{tx} is a transaction hash. The Inclusion Certificate C^{inc} connects the leaf to the SMT root hash h in the Input Record. The Unicity Certificate then chains through the shard tree, unicity tree, and unicity seal to the global root of trust: Unicity Trust Base $\mathcal{T}$.

3.8.1 Sparse Merkle Tree

The aggregation layer maintains a (*Radix*) *Sparse Merkle Tree* (SMT), a path-compressed authenticated map from 256-bit state identifiers to transaction hashes. Leaves are anchored by full keys, and internal nodes record the branching structure needed to route lookups and authenticate the root. The SMT supports batch insertion, inclusion certificates, non-inclusion certificates and consistency proofs. Exact formats and algorithms are given in Appendix C.4.

3.8.1.1 SMT Leaf Structure

An SMT leaf contains a key-value pair:

- **Key:** State identifier $\text{sid} = H(v, h_{\text{st}})$ where:
 - v – spending predicate of the token owner (Sec. 2.2)
 - h_{st} – state hash representing the current state of a token
- **Value:** Transaction data hash h_{tx} .

The SMT effectively maintains a mapping $\text{sid} \mapsto h_{\text{tx}}$ that proves a state transition from state identifier sid was authorized via transaction with hash h_{tx} . This prevents double-spending: once a state identifier is registered in the SMT with a transaction hash, that state cannot be spent again.

3.8.1.2 SMT Certificates and Updates

The SMT exposes the following functionalities and authenticated objects:

- **Batch insertion:** inserts a batch of new leaves into the tree and updates the root hash.
- **Inclusion Certificate** C^{inc} : authenticates that a leaf $(\text{sid}, h_{\text{tx}})$ is present under a given SMT root.
- **Non-Inclusion Certificate** C^{exc} : authenticates that a state identifier sid is absent from the tree.
- **Consistency proof** π_{CP} : authenticates that a new SMT root extends a previous root by a batch insertion.

Appendix C.4 defines the corresponding wire formats and verification procedures.

3.8.2 Shard Tree Certificate

Shard Tree Certificate connects a shard's certified input hash to the partition's shard-tree root, from which the Unicity Tree certificate continues to the global root of trust. It is a tuple $C^{\text{shard}} = (\sigma; h_1^s, \dots, h_{|\sigma|}^s)$, where:

1. $\sigma = \sigma_1 \sigma_2 \dots \sigma_{|\sigma|}$ is a shard identifier of type $\{0, 1\}^*$
2. $h_1^s, \dots, h_{|\sigma|}^s$ – sibling hashes of type $\mathbb{H}$

For the single shard case, the sharding scheme is $\{\llbracket \rrbracket\}$ and the certificate is $(\llbracket, \perp)$.

3.8.2.1 Sharded SMT Structure

In a sharded setting, each shard maintains the SMT for its portion of the key space and produces a certified shard input record (IR, h_t) . The Consensus Layer then deterministically builds the shard tree from the authenticated shard inputs of all shards in the partition (see Sec. 4.1.5 and Algorithm 16).

3.8.2.2 Creation: CreateShardTreeCert

Input:

1. σ – shard identifier
2. χ – shard tree of type $\chi: \overline{\mathcal{SH}} \rightarrow \mathbb{H}$

Output: shard tree certificate $C^{\text{shard}} = (\sigma; h_1^s, \dots, h_{|\sigma|}^s)$

Computation:

```

 $C \leftarrow ()$ 
for  $i \leftarrow |\sigma|$  downto 1 do
     $C \leftarrow C \parallel \chi(\sigma_1 \sigma_2 \dots \sigma_{i-1} \overline{\sigma_i})$ 
end for
return  $(\sigma; C)$ 

```

where $\sigma_1 \sigma_2 \dots \sigma_m$ is the binary representation of σ , and $\overline{\sigma_i}$ is the binary complement of σ_i .

For the single shard case, the shard certificate is $(\perp, \perp)$.

3.8.2.3 Computation: CompShardTreeCert

Input:

1. $(\sigma; h_1^s, \dots, h_{|\sigma|}^s)$ – shard tree certificate, where $\sigma = \sigma_1 \sigma_2 \dots \sigma_{|\sigma|}$ is a shard identifier
2. IR – Shard Input Record
3. h_t – hash value of type $\mathbb{H}$

Output: Root hash r of type $\mathbb{H}$

Computation:

```

 $r \leftarrow H(IR \parallel h_t)$ 
for  $i \leftarrow |\sigma|$  downto 1 do
    if  $\sigma_i = 0$  then  $r \leftarrow H(r, h_i^s)$ 
    if  $\sigma_i = 1$  then  $r \leftarrow H(h_i^s, r)$ 
end for
return  $r$ 

```

For the single shard case, $\text{CompShardTreeCert}((\perp, \perp), IR, h_t)$ returns $H(IR \parallel h_t)$.

3.8.3 Unicity Tree Certificate

Unicity Tree Certificate (Fig. 6) is a tuple $C^{\text{uni}} = (\beta, \text{dhash}; (\beta_2, h_2), \dots, (\beta_\ell, h_\ell))$, where:

1. β – partition identifier of type $\mathbb{C}$

2. dhash – partition description hash of type $\mathbb{H}$
3. $(\beta_2, h_2), \dots, (\beta_\ell, h_\ell)$ – a sequence of partition identifier and sibling hash pairs

3.8.3.1 Creation: CreateUnicityTreeCert

Input:

1. β – partition identifier of type $\mathbb{C}$
2. C – set of partition identifiers of type $\mathbb{C}$
3. CD – partition description of type $\mathbb{CD}[C]$
4. $I\mathcal{H}$ – input hashes of type $\mathbb{H}[C]$

Output: unicity tree certificate $(\beta, \text{dhash}; (\beta_2, h_2), \dots, (\beta_\ell, h_\ell))$

Computation:

```

 $n \leftarrow |C|$                                 ▶ Number of partitions
 $C \leftarrow \text{sorted}(C)$                       ▶ Sorted list
for  $i \leftarrow 1 \dots n$  do
     $x_i \leftarrow H(I\mathcal{H}[C_i] \parallel H(CD[C_i]))$ 
end for
 $\langle (\beta_1, h_1), (\beta_2, h_2), \dots, (\beta_\ell, h_\ell) \rangle \leftarrow \text{INDEX\_TREE\_CHAIN}((C_1, x_1), (C_2, x_2), \dots, (C_n, x_n), \beta)$ 
return  $(\beta, H(CD[\beta]); (\beta_2, h_2), \dots, (\beta_\ell, h_\ell))$  ▶ Drop redundant first hash step

```

3.8.3.2 Computation: CompUnicityTreeCert

Input:

1. $(\beta, \text{dhash}; (\beta_2, h_2), \dots, (\beta_\ell, h_\ell))$ – unicity tree certificate
2. x – input hash (output of the CompShardTreeCert function)

Output: Root hash r of type $\mathbb{H}$

Computation:

```

 $L \leftarrow \langle (\beta, H(x, \text{dhash})), (\beta_2, h_2), \dots, (\beta_\ell, h_\ell) \rangle$  ▶ Restore the first hash step
return  $\text{INDEX\_TREE\_OUTPUT}(L; \beta)$  ▶ Sec. C.2.3

```

3.8.4 Unicity Seal

Unicity Seal is a tuple $C^r = (\alpha, n_r, e_r, t_r, r_-, r; s)$, where:

1. α – Network identifier
2. n_r – BFT Core's Round number
3. e_r – BFT Core's Epoch number
4. t_r – Round creation time (wall clock value specified and verified by the BFT Core), with one-second precision. See Appendix B for encoding
5. r_- – Root hash of previous round's Unicity Tree
6. r – Root hash of the Unicity Tree (denoted as r^{root} if necessary for clarity)

7. s – Signature, computed by the BFT Core over preceding fields ($s = \text{Sign}_{\text{sk}_r}(n_r, e_r, t_r, r_-, r)$). The formulation of signature field depends on underlying consensus mechanism and its parameters. Here we assume an opaque data structure which can be verified based on the Unicity Trust Base, i.e., there is an implementation of an abstract function $\text{Verify}_{\mathcal{T}}((n_r, e_r, t_r, r_-, r), s)$, encapsulated into the implementation of VerifyUnicitySeal .

VerifyUnicitySeal – unicity seal verification function of type $\mathbb{H} \times \mathbb{US} \times \mathbb{UB} \rightarrow \mathbb{B}$. This function also verifies, if Unicity Seal's network identifier matches with the Unicity Trust Base.

3.8.5 Unicity Certificate

Unicity Certificate is a tuple $UC = (IR, h_t, C^{\text{shard}}, C^{\text{uni}}, C^r)$, where:

1. IR is a shard input record of type $\mathbb{IR}$ (Sec. 4.1.1),
2. h_t is the hash (of technical record) of type $\mathbb{H}$,
3. C^{shard} is a shard tree certificate,
4. C^{uni} is a unicity tree certificate, and
5. C^r is a unicity seal

Elements of the tuple form an authenticated chain, see function $\text{Verification: VerifyUnicityCert}$.

3.8.5.1 Verification: VerifyUnicityCert

Verifies if unicity certificate is valid, based on unicity trust base as the root of trust. This function authenticates the certificate chain itself. It does not, by itself, prove that a particular state identifier belongs to the shard named in C^{shard} ; that binding is checked by the proof verification functions below.

Input:

1. $UC = (IR, h_t, C^{\text{shard}}, C^{\text{uni}}, C^r)$ – Unicity Certificate
2. $\mathcal{T}$ – Unicity Trust Base

Output: TRUE OR FALSE

Computation:

```

ensure( $UC.C^r.\alpha = \mathcal{T}.\alpha$ )
 $r \leftarrow \text{CompShardTreeCert}(UC.C^{\text{shard}}, UC.IR, UC.h_t)$ 
 $r \leftarrow \text{CompUnicityTreeCert}(UC.C^{\text{uni}}, r)$ 
return  $\text{VerifyUnicitySeal}(r, UC.C^r, \mathcal{T}) = 1$ 

```

3.9 Proofs

3.9.1 Inclusion Proof

Inclusion Proof proves that a state identifier has been recorded in the SMT as a proof of the spend being unique, and that the resulting shard state was certified by the BFT Core. It is a tuple $\pi^{\text{inc}} = (\text{sid}, h_{\text{tx}}, C^{\text{inc}}, UC)$, where:

1. $\text{sid} \in \text{OCT}^{32}$ – state identifier (SMT leaf key)
2. $h_{\text{tx}} \in \mathbb{H}$ – transaction data hash (SMT leaf value)
3. C^{inc} – Inclusion Certificate (Sec. 3.8.1.2)
4. UC – unicity certificate (Sec. 3.8.5)

3.9.1.1 Verification: VerifyInclusionProof

VerifyInclusionProof – inclusion proof verification function of type $\mathbb{IP} \times \mathbb{CD} \times \mathbb{UB} \rightarrow \mathbb{B}$

Input:

1. $\pi^{\text{inc}} = (\text{sid}, h_{\text{tx}}, C^{\text{inc}}, UC)$ – inclusion proof
2. $\mathcal{CD}_\beta$ – partition description of the partition identified by $UC.C^{\text{uni}}.\beta$
3. $\mathcal{T}$ – trust base

Output: TRUE OR FALSE

Computation:

```

ensure( $UC.C^r.\alpha = \mathcal{T}.\alpha$ )      ▶ Certificate belongs to the same network as the trust base
ensure( $\mathcal{CD}_\beta.\alpha = \mathcal{T}.\alpha \wedge H(\mathcal{CD}_\beta) = UC.C^{\text{uni}}.\text{dhash}$ )  ▶ Authenticate partition description
 $\sigma \leftarrow UC.C^{\text{shard}}.\sigma$ 
ensure( $\sigma \in \mathcal{CD}_\beta.SH$ )
ensure( $f_{\mathcal{CD}_\beta.SH}(\text{sid}) = \sigma$ )      ▶ Proof comes from the right shard
 $h \leftarrow UC.IR.h$                   ▶ Extract SMT root hash from IR
if  $\neg \text{RSMT\_VERIFY\_INCLUSION}(C^{\text{inc}}, h, \text{sid}, h_{\text{tx}})$  then return FALSE
end if
if  $\neg \text{VerifyUnicityCert}(UC, \mathcal{T})$  then return FALSE
end if
return TRUE

```

3.9.2 Non-Inclusion Proof

Non-Inclusion Proof proves that a state identifier does NOT exist in the SMT. It is a tuple $\pi^{\text{exc}} = (C^{\text{exc}}, UC)$, where:

1. C^{exc} – Non-Inclusion Certificate (Appendix C.4.6)
2. UC – unicity certificate (Sec. 3.8.5)

3.9.2.1 Verification: VerifyNonInclusionProof

VerifyNonInclusionProof – non-inclusion proof verification function of type $\mathbb{EP} \times \mathbb{CD} \times \mathbb{UB} \rightarrow \mathbb{B}$

Input:

1. $\pi^{\text{exc}} = (C^{\text{exc}}, UC)$ – non-inclusion proof
2. $\mathcal{CD}_\beta$ – authenticated partition description of the partition identified by $UC.C^{\text{uni}}.\beta$
3. $\mathcal{T}$ – trust base
4. $\text{sid} \in \text{OCT}^{32}$ – queried state identifier

Output: TRUE OR FALSE

Computation:

```

ensure( $UC.C^r.\alpha = \mathcal{T}.\alpha$ )      ▶ Certificate belongs to the same network as the trust base
ensure( $CD_\beta.\alpha = \mathcal{T}.\alpha \wedge H(CD_\beta) = UC.C^{uni}.dhash$ )  ▶ Authenticate partition description
 $\sigma \leftarrow UC.C^{shard}.\sigma$ 
ensure( $\sigma \in CD_\beta.SH$ )
ensure( $f_{CD_\beta.SH}(sid) = \sigma$ )      ▶ Proof comes from the right shard
 $h \leftarrow UC.IR.h$                 ▶ Extract SMT root hash from IR
if  $\neg RSMT\_VERIFY\_NON\_INCLUSION(C^{exc}, h, sid)$  then return FALSE
end if
if  $\neg VerifyUnicityCert(UC, \mathcal{T})$  then return FALSE
end if
return TRUE

```

3.9.3 Consistency Proof

A *Consistency Proof* (also called *non-deletion proof*) is a cryptographic construction that validates one round of operation of the append-only SMT maintained by aggregation layer nodes. It proves that the state transition from previous SMT root hash h' to current SMT root hash h was performed correctly: only new leaves were added, and no pre-existing leaves were removed or modified. For a round with inserted batch

$$B = \langle (sid_1, h_{tx,1}), \dots, (sid_j, h_{tx,j}) \rangle,$$

the security objective is to prove that the map committed by h is obtained from the map committed by h' by extending it with exactly the leaves in B .

The consistency proof enables the BFT Core to verify the integrity of the Aggregation Layer's SMT manipulation with respect to the authenticated prior root and the declared round batch.

For a batch of state transitions processed in a single round, the consistency proof π_{CP} is the post-order opcode stream defined in Appendix C.4.7.

3.9.3.1 Security Model

The aggregation layer node that computes π_{CP} may be Byzantine: it may maintain an arbitrary internal tree representation, claim an arbitrary post-state root h , and send an arbitrary proof stream. The verifier trusts only the following inputs:

1. the previously certified root hash h' ;
2. the batch B determined by the validated requests of the round;
3. the claimed post-state root hash h in the shard input record; and
4. the collision resistance of the hash function H .

The consistency proof authenticates only the append-only SMT update from h' to h . It does not, by itself, prove that an element of B was semantically admissible for insertion; this is established separately by request validation (Sec. 3.7) and public audit of append only logs of Aggregation nodes.

3.9.3.2 Verification Function

VerifyConsistencyProof – consistency proof verification function of type $\mathbb{CP} \times (\text{OCT}^{32} \times \mathbb{H})^* \times \mathbb{H} \cup \{\perp\} \times \mathbb{H} \cup \{\perp\} \rightarrow \mathbb{B}$

Input:

1. π_{CP} – consistency proof (opcode stream)
2. $B = \langle (\text{sid}_1, h_{\text{tx},1}), \dots, (\text{sid}_j, h_{\text{tx},j}) \rangle$ – batch of inserted leaves
3. h' – previous SMT root hash (before the round)
4. h – current SMT root hash (after the round)

Output: TRUE OR FALSE

The verification algorithm (`RSMT_VERIFY_CONSISTENCY`) is defined in Appendix C.4.7.4.

The intended proof system properties are as follows. Specified algorithms enforce Items 1–4 below. Item 5 is a stronger structural property that is intentionally skipped.

1. **Completeness:** If an honest aggregation layer node starts from the tree committed by h' and inserts exactly the batch B , then it can produce π_{CP} such that $\text{VerifyConsistencyProof}(\pi_{CP}, B, h', h) = \text{TRUE}$.
2. **Soundness of prior-state preservation:** Let $M_0, M_1: \text{OCT}^{32} \rightarrow \mathbb{H} \cup \{\perp\}$ denote the partial maps committed by h' and h , respectively. If $\text{VerifyConsistencyProof}(\pi_{CP}, B, h', h) = \text{TRUE}$, then

$$\forall \text{sid} \in \text{OCT}^{32}: M_0(\text{sid}) \neq \perp \Rightarrow M_1(\text{sid}) = M_0(\text{sid}).$$

That is, no previously committed SMT leaf can be deleted or modified.

3. **Batch incorporation:** If $\text{VerifyConsistencyProof}(\pi_{CP}, B, h', h) = \text{TRUE}$, then

$$\forall i \in \{1, \dots, j\}: M_1(\text{sid}_i) = h_{\text{tx},i}.$$

Thus every declared inserted leaf appears in the post-state under the key determined by its state identifier.

4. **No phantom insertions:** If $\text{VerifyConsistencyProof}(\pi_{CP}, B, h', h) = \text{TRUE}$, then

$$\begin{aligned} \forall \text{sid} \in \text{OCT}^{32}: M_1(\text{sid}) \neq \perp \wedge M_0(\text{sid}) = \perp \\ \Rightarrow \exists i \in \{1, \dots, j\}: (\text{sid}, M_1(\text{sid})) = (\text{sid}_i, h_{\text{tx},i}). \end{aligned}$$

Therefore every new post-state leaf is justified by the declared batch.

5. **Canonical inserts** (stronger structural property): Let $\text{root}_{\text{canon}}(M)$ denote the root hash of the unique well-formed radix sparse Merkle tree induced by a finite map M . The strongest desirable statement is

$$h = \text{root}_{\text{canon}}(M_0 \cup B).$$

Equivalently, the post-state topology is uniquely determined by the committed keys and every leaf in the domain of $M_0 \cup B$ admits a valid inclusion certificate against h .

Equivalently, if B is interpreted as a finite partial map with pairwise distinct keys disjoint from the domain of M_0 , then acceptance of π_{CP} implies

$$M_1 = M_0 \cup B.$$

The specification deliberately omits Item 5 for efficiency reasons. In the worst case, a Byzantine aggregation layer node may obtain certification for a non-canonical post-state root that still preserves all previously committed leaves and incorporates the declared batch, yet leaves some committed leaves without valid inclusion certificates. Thus the loss is in structural canonicity and proof availability, not in prior-state preservation or batch incorporation.

4 BFT Core

4.1 Data Structures of the BFT Core

4.1.1 Shard Input Record

Shard input record (IR) of a shard of a partition (of type $\mathbb{IR}$) is a tuple (n, e, h', h, t) , where:

1. n – shard's round number of type $\mathbb{N}_{64}$
2. e – shard's epoch number of type $\mathbb{N}_{64}$
3. h' – previous round's SMT root hash of type $\mathbb{H}$
4. h – current round's SMT root hash of type $\mathbb{H}$
5. t – reference time for request validation of type $\mathbb{N}_{64}$

4.1.2 Statistical Record

Statistical record of type $\mathbb{SR}$ is a tuple $\text{SR} = (n_e, \bar{\ell}_B, \bar{\ell}_S, \hat{\ell}_B, \hat{\ell}_S)$, where:

1. n_e – number of non-empty rounds (adding at least one new leaf), of type $\mathbb{N}_{64}$
2. $\bar{\ell}_S$ – memory usage of the SMT, of type $\mathbb{N}_{64}$
3. $\hat{\ell}_B$ – maximum number of SMT additions processed during a round, of type $\mathbb{N}_{64}$
4. ... – other necessary data: fees, etc.

There is one Statistical Record of current epoch where the values are being updated, reflecting the current state since the beginning of the epoch; and one invariant Statistical Record of the preceding epoch of every shard of every public partition.

4.1.3 Technical Record

Technical record of type $\mathbb{TE}$ is a tuple $\text{TE} = (n_r, e_r, v_\ell, h_{\text{sr}})$, where:

1. n_r – suggested next round number of type $\mathbb{N}_{64}$
2. e_r – suggested next epoch number of type $\mathbb{N}_{64}$
3. v_ℓ – suggested leader identifier of type $\{0, 1\}^*$
4. h_{sr} – hash of statistical records; type $\mathbb{H}$

Technical record is delivered with an Unicity Certificate. When a shard is extending the Unicity Certificate with a next round certification attempt, then it must use the suggested values provided in Technical Record.

There is one Technical Record for every shard of every partition, providing synchronization for the next round.

4.1.4 Shard Info

Shard info of type $\mathbb{SI}$ is a tuple $(n, e, h_-, \text{SR}_-, \text{SR}, \mathcal{V}, v_\ell, UC_-)$, where:

1. n – shard's round number of type $\mathbb{N}_{64}$
2. e – shard's epoch number of type $\mathbb{N}_{64}$
3. h_- – last-certified root hash
4. SR_- – statistical record of the previous epoch
5. SR – statistical record of the current epoch, initially (at each epoch) $(0, 0, 0, 0, 0)$
6. $\mathcal{V}$ – validators of the shard, a set of identifiers, each of type $\{0, 1\}^*$
7. v_ℓ – leader identifier of type $\{0, 1\}^*$ (it is assumed that $v_\ell \in \mathcal{V}$)
8. UC_- – last created unicity certificate

4.1.5 Shard Tree

For a partition β , let $\mathcal{SH}_\beta = \text{CD}[\beta].\mathcal{SH}$, and $\text{IR}\mathcal{T}_\beta$ be of type $(\mathbb{IR}, \mathbb{H})[\mathcal{SH}_\beta]$, i.e. for every $\beta \in C$ and $\sigma \in \mathcal{SH}_\beta$, $\text{IR}\mathcal{T}_\beta[\sigma] = (\text{IR}, h_t)$ (for some IR and h_t).

If there is no input from a shard to certify then $\text{IR}\mathcal{T}_\beta[\sigma] = \text{IR}\mathcal{T}_\beta[\sigma_-]$, that is, the value from the previously built Shard Tree is used. If there is no previous value then $\text{IR}\mathcal{T}_\beta[\sigma] = \perp$.

Shard tree for a partition β is a function $\chi_\beta: \overline{\mathcal{SH}_\beta} \rightarrow \mathbb{H}$ such that:

1. If $\sigma \in \mathcal{SH}_\beta$, then $\chi_\beta(\sigma) = H(\text{IR}\mathcal{T}_\beta[\sigma]) = H(\text{IR}\mathcal{T}_\beta[\sigma].\text{IR} \parallel \text{IR}\mathcal{T}_\beta[\sigma].h_t)$.
2. If $\sigma \in \overline{\mathcal{SH}_\beta} \setminus \mathcal{SH}_\beta$, then $\chi_\beta(\sigma) = H(\chi_\beta(\sigma \parallel 0) \parallel \chi_\beta(\sigma \parallel 1))$.

The value $\chi_\beta(\perp)$ is called the root hash of the shard tree.

Shard tree certificate for a shard $\sigma \in \mathcal{SH}_\beta$ is a sequence $h_1^s, \dots, h_m^s$ of sibling hash values of type $\mathbb{H}$, where $m = |\sigma|$ (the number of bits in σ); see Sec. 3.8.2.

4.1.5.1 Shard Tree Creation: CreateShardTree

Input:

1. $\mathcal{SH}$ – sharding scheme of type $\mathbb{SH}$
2. $\text{IR}\mathcal{T}$ – shard-specific data of type $(\mathbb{IR}, \mathbb{H})[\mathbb{SH}]$

Output: χ – shard tree of type $\overline{\mathcal{SH}} \rightarrow \mathbb{H}$

Computation: $\chi \leftarrow \perp$, $\text{genST}(\perp)$

where genST is the following recursive function of type $\{0, 1\}^* \rightarrow \mathbb{H}$ with side effects:

$\text{genST}(\sigma)$:

1. **if** $\sigma \in \mathcal{SH}$, **then store** $\chi(\sigma) \leftarrow H(\text{IR}\mathcal{T}[\sigma])$ **and return** $\chi(\sigma)$.
2. **if** $\sigma \in \overline{\mathcal{SH}} \setminus \mathcal{SH}$, **then store** $\chi(\sigma) \leftarrow H(\text{genST}(\sigma \parallel 0) \parallel \text{genST}(\sigma \parallel 1))$ **and return** $\chi(\sigma)$

For the single shard case, the shard tree is $\{(\perp, H(\text{IR}\mathcal{T}[\sigma]))\}$, i.e. it only has a single record for the root hash $\chi(\perp) = H(\text{IR}\mathcal{T}[\sigma])$.

4.1.6 Unicity Tree

Unicity Tree is an indexed Merkle tree that aggregates the state of all partitions in the network.

4.1.6.1 Creation: CreateUnicityTree

Input:

1. C – set of partition identifiers of type $\mathbb{C}$
2. CD – partition description of type $\mathbb{CD}[C]$
3. $I\mathcal{H}$ – input hashes of type $\mathbb{H}[C]$

Output: r – unicity tree root hash of type $\mathbb{H}$

Computation:

```

 $n \leftarrow |C|$                                 ▶ Number of partitions
 $C \leftarrow \text{sorted}(C)$                     ▶ Sorted list
for  $i \leftarrow 1 \dots n$  do
     $x_i \leftarrow H(I\mathcal{H}[C_i] \parallel H(CD[C_i]))$ 
end for
return  $\text{INDEX\_TREE\_ROOT}((C_1, x_1), (C_2, x_2), \dots, (C_n, x_n))$  ▶ Sec. C.2.1

```

4.2 State of the BFT Core

State of the BFT Core is a tuple $(\alpha, n, e, r_-, \mathcal{T}, C, CD, SI)$, where:

1. α – network identifier of type $\mathbb{A}$
2. n – BFT Core's round number of type $\mathbb{N}_{64}$
3. e – BFT Core's epoch number of type $\mathbb{N}_{64}$
4. r_- – previous root hash of the unicity tree of type $\mathbb{H}$
5. $\mathcal{T}$ – unicity trust base of type $\mathbb{UB}$
6. C – set of partition identifiers, with elements of type $\mathbb{C}$
7. CD – partition descriptions of type $\mathbb{CD}[C]$
8. SI – shard info of type $\mathbb{SI}[\mathbb{C}, \{0, 1\}^*]$

Epoch number can be interpreted as the version number of some shard's or BFT Core's configuration. It is used by supporting layers like orchestration and consensus. On static configuration, the epoch is 0.

4.3 Messages of the BFT Core

4.3.1 Certification Request

Certification Request (CR) of a shard is a message $\langle \alpha, \beta, \sigma, \nu; I\mathcal{R}, \ell_B, \ell_S, [\pi_{CP}]; s; \rangle$, where:

1. α – network identifier of type $\mathbb{A}$
2. β – partition identifier of type $\mathbb{C}$

3. σ – shard identifier of type $\mathbb{SH}$
4. ν – validator identifier (aggregation layer node identifier) of type $\{0, 1\}^*$
5. $\mathcal{IR}$ – shard input record of type $\mathbb{IR}$
6. ℓ_B – number of added new SMT leaves during the round, type $\mathbb{N}_{64}$
7. ℓ_S – SMT memory usage in bytes of type $\mathbb{N}_{64}$
8. $\pi_{CP} \vee \perp$ – consistency proof of type $\mathbb{CP}$ (Sec. 3.9.3)
9. s – signature authenticating the message

Certification requests are sent to the BFT Core by the aggregation layer nodes (validators) implementing the shards.

4.3.2 Certification Response

Certification Response (CReS) of a shard is a message $\langle \alpha, \beta, \sigma, \text{TE}; UC \rangle$, where:

1. α – network identifier of type $\mathbb{A}$
2. β – partition identifier of type $\mathbb{C}$
3. σ – shard identifier of type $\mathbb{SH}$
4. TE – technical record of type $\mathbb{TE}$
5. UC – certified unicity certificate

Certification response is sent by the BFT Core to the nodes of an aggregation layer nodes as a response to a certification request message. This is an asynchronous message: response is not immediate, and there can be multiple subsequent responses.

4.4 Functional Description of the BFT Core

During every round, the BFT Core receives certification requests from the shards of partitions $\beta \in \mathbb{C}$.

The BFT Core certifies the SMT root hashes provided by the aggregation layer nodes. Each shard maintains an SMT for its portion of the key space. When aggregation layer nodes process Unicity Service Requests (as specified in Sec. 3.7), they update their SMTs by setting leaves. The BFT Core certifies these updates by:

1. Receiving certification requests containing the new SMT root hash h and previous root hash h'
2. Verifying the consistency proof π_{CP} that proves correct SMT manipulation
3. Certifying the new root hash via the Unicity Certificate

As every shard σ of a partition β is implemented by a redundant distributed system with certain number $m = |\mathcal{SI}[\beta, \sigma].\mathcal{V}|$ of validator machines, it is possible that several certification requests $\text{CR}_{\beta, \sigma}$ with the same $\text{CR}_{\beta, \sigma}.\mathcal{IR}.h'$ but different $\text{CR}_{\beta, \sigma}.\mathcal{IR}.h$ are received. This is solved by a majority voting mechanism requiring that a required majority, at least $\lfloor m/2 \rfloor + 1$ of the dedicated validators, send $\text{CR}_{\beta, \sigma}$ with an identical value of $\text{CR}_{\beta, \sigma}.\mathcal{IR}.h$. It is possible that not every shard sends its shard input record to the BFT Core during the round, and the majority voting may fail (see the Consensus specification for details).

For every incoming certificate request $CR = \langle \alpha, \beta, \sigma, \nu; t, IR, \ell_B, \ell_S \rangle$ the following checks are made (if any of them fails, CR is discarded):

1. $CR.\alpha = \alpha$ – request came from the same network instance
2. $CR.\beta \in C$ and $\sigma \in CD[CR.\beta].SH$ – request came from a legitimate shard of a registered partition
3. $CR.\nu \in SI[CR.\beta, CR.\sigma].V$ – request came from an authorized aggregation layer node
4. $CR.IR.n = SI[\beta, \sigma].n + 1$ – round number is correctly incremented
5. $CR.IR.e = SI[\beta, \sigma].e$ – epoch number matches
6. $CR.IR.h' = SI[\beta, \sigma].h_-$ – previous round's SMT root hash in IR matches the recorded one
7. $CD[\beta].\gamma_{CP}(CR.IR, CR.\pi_{CP})$ – consistency proof verification (Sec. 3.9.3) succeeds: verify $VerifyConsistencyProof(CR.\pi_{CP}, B, CR.IR.h', CR.IR.h) = \text{TRUE}$ and $|B| = CR.\ell_B$ (batch size matches).
8. $CR.IR.t = SI[\beta, \sigma].UC-.IR.C^r.t$ – time reference is equal to the time field of the previous unicity seal

Only one (majority-voted) certificate request (denoted by $CR_{\beta, \sigma}$) is accepted from every shard.

When the BFT Core's round n is completed, then:

1. For every $\beta \in C$, and $\sigma \in SH_\beta$ with accepted certificate request ³:
 - 1.1 $SI[\beta, \sigma].n \leftarrow CR.IR.n$
 - 1.2 $SI[\beta, \sigma].h_- \leftarrow CR.IR.h$
 - 1.3 $SI[\beta, \sigma].SR.n_e \leftarrow SI[\beta, \sigma].SR.n_e + 1$
 - 1.4 $SI[\beta, \sigma].SR.\bar{\ell}_B \leftarrow SI[\beta, \sigma].SR.\bar{\ell}_B + CR.IR.\ell_B$
 - 1.5 $SI[\beta, \sigma].SR.\bar{\ell}_S \leftarrow SI[\beta, \sigma].SR.\bar{\ell}_S + CR.IR.\ell_S$
 - 1.6 $SI[\beta, \sigma].SR.\hat{\ell}_B \leftarrow \max\{SI[\beta, \sigma].SR.\hat{\ell}_B, CR.IR.\ell_B\}$
 - 1.7 $SI[\beta, \sigma].SR.\hat{\ell}_S \leftarrow \max\{SI[\beta, \sigma].SR.\hat{\ell}_S, CR.IR.\ell_S\}$
 - 1.8 The leader for next round:
 $SI[\beta, \sigma].\nu_\ell \leftarrow \text{LEADERFUNC}(UC-, SI[\beta, \sigma].V)$
 - 1.9 Technical record:
 $\mathcal{TE}_{\beta, \sigma} \leftarrow (SI[\beta, \sigma].n + 1, SI[\beta, \sigma].e, SI[\beta, \sigma].\nu_\ell, h_{sr})$,
 where $h_{sr} = H(SI[\beta, \sigma].SR-, SI[\beta, \sigma].SR)$
2. For every registered partition $\beta \in C$, the collected certification requests $CR_{\beta, \sigma}$ are converted to a temporary data structure IRT_β of type $(\mathbb{IR}, \mathbb{H})[SH_\beta]$, where $SH_\beta = CD[\beta].SH$, and $IRT_\beta[\sigma] = (CR_{\beta, \sigma}.IR, H(\mathcal{TE}_{\beta, \sigma}))$; if there is no request then respective leaf repeats its previous value. If there is no previous value, the leaf is initialized to $\perp$.
3. For every $\beta \in C$, the shard tree χ_β (Sec. 4.1.5) is created by the function call $\chi_\beta \leftarrow \text{CreateShardTree}(SH_\beta, IRT_\beta)$ (Sec. 4.1.5.1).

³Please refer to the Consensus chapter for details; notably a “request” may be induced for technical reasons

4. A temporary data structure $\mathcal{IH}$ of type $\mathbb{H}[C]$ is created such that $\mathcal{IH}[\beta] \leftarrow \chi_\beta(\perp)$ for every $\beta \in C$, i.e. $\mathcal{IH}$ contains the root hashes of the shard trees.
5. The root of unicity tree is computed as $r \leftarrow \text{CreateUnicityTree}(C, \mathcal{CD}, \mathcal{IH})$ (Sec. 4.1.6.1)
6. Unicity seal $C^r = (\alpha, n, e, t, r_-, r; s)$ is created, where $t \leftarrow \text{time}()$ is the current time and s is a "signature" on all other fields. The form of s depends on the used consensus mechanism.
7. For every $\beta \in C$, the unicity tree certificate C_β^{uni} is created by the function call $C_\beta^{\text{uni}} \leftarrow \text{CreateUnicityTreeCert}(\beta, C, \mathcal{CD}, \mathcal{IH})$ (Sec. 3.8.3.1)
8. For every $\beta \in C$, and $\sigma \in \mathcal{SH}_\beta$:
 - 8.1 The shard tree certificate $C_{\beta,\sigma}^{\text{shard}}$ (Sec. 3.8.2) is created by the function call $C_{\beta,\sigma}^{\text{shard}} \leftarrow \text{CreateShardTreeCert}(\sigma, \chi_\beta)$ (Sec. 3.8.2.2)
 - 8.2 The unicity certificate $UC_{\beta,\sigma} = (\mathcal{IRT}_\beta[\sigma].\mathcal{IR}, \mathcal{IRT}_\beta[\sigma].h_t, C_{\beta,\sigma}^{\text{shard}}, C_\beta^{\text{uni}}, C^r)$ (Sec. 3.8.5) and the certification response $\text{CReS}_{\beta,\sigma} = \langle \alpha, \beta, \sigma; UC_{\beta,\sigma}, \mathcal{TE}_{\beta,\sigma} \rangle$ are composed (if the shard input have changed)
 - 8.3 The last unicity certificate field is updated by $\mathcal{SI}[\beta, \sigma]UC_- \leftarrow UC$ (if the shard input have changed).
9. The round number and the previous root hash of the unicity tree are updated by $n \leftarrow n + 1$ and $r_- \leftarrow r$.

When a shard's (identified by β, σ) epoch with at least one non-empty round ends, the following assignments are executed:

1. $\mathcal{SI}[\beta, \sigma].\text{SR}_- \leftarrow \mathcal{SI}[\beta, \sigma].\text{SR}$
2. $\mathcal{SI}[\beta, \sigma].\text{SR} \leftarrow (0, 0, 0, 0, 0)$
3. $\mathcal{SI}[\beta, \sigma].e \leftarrow \mathcal{SI}[\beta, \sigma].e + 1$
4. The new validator set $\mathcal{SI}[\beta, \sigma].\mathcal{V}$ is chosen

5 Consensus Layer: BFT

5.1 Background

5.1.1 Definitions

Block is a set of requests, grouped together for mostly efficiency reasons, to be processed during one round. Unicity BFT Core does not produce an explicit blockchain – its certificates are persisted as proofs within the tokens.

UC is *Unicity Certificate*.

We call UC a **repeat UC** if it has incremented round number for a particular shard, but the certified hash has not changed compared to the UC of the previous round.

All aggregation layer nodes and BFT Core validators operate in **rounds**.

A round **extends** another finalized round by including its cryptographic hash as the hash of previous round.

The validators of a shard are synchronized based on input from the BFT Core. There are some fixed time-outs.

System has one BFT Core and an arbitrary number of partitions, which may be split into arbitrary number of shards.

Within a shard, there are k validators with identifier v , of which f might be faulty. For the BFT Core $k > 3f$. For a shard σ , $k_{\beta,\sigma} > 2f$. We assume that all faulty validators may be controlled by a coordinated, non-adaptive adversary. We assume trusted setup (Genesis) and authenticated data links (signed messages). We assume partially synchronous communication model where after unknown time GST message delivery time is upper-bounded by known Δ . We assume that in every shard, at least one non-faulty validator is able to persist its state.

A signature is denoted as s . Signed message with message name $name$ is denoted as $\langle name \mid a, b, c; s \rangle$. Array of message fields is denoted as $\{f\}$.

Clients send **Unicity Service Requests** (requests).

5.1.2 Scope

Implementation details of BFT Core's atomic broadcast primitive (implementing the protocol Ordering) are not given. This is a modular component. Only safety-critical validation rules are provided.

5.1.3 Repeating Notation

n_r – round number of the BFT Core

$n_{\beta,\sigma}$ – round number of shard σ of partition β

$k_{\beta,\sigma}$ – number of aggregation layer nodes in shard σ of partition β , $k_{\beta,\sigma} = |\mathcal{V}_{\beta,\sigma}|$

v – aggregation layer node identifier, unique within the Unicity System instance; set of a shard's nodes is $\mathcal{V} = \{v_i\}_{i \in \{1, \dots, k_\sigma\}}$

$v_l \leftarrow \text{LEADERFUNC}(\cdot)$ – leader identifier for this round

h – SMT root hash

h' – previous SMT root hash

ensure(...) – function modeled after the Solidity language – if its argument evaluates to true then nothing happens; if it evaluates to false then execution stops and function returns 0. Unlike Solidity, should be complemented with returning and logging informative errors. Mostly used in message handlers for input validation.

function($a, b \leftarrow c$) – default value of function arguments, like in Python language. If 2nd argument is not specified by caller then parameter b obtains the value of expression c .

5.2 Aggregation Layer

5.2.1 Timing

An aggregation shard is synchronized using Input Records in returned UCs. For a shard, a UC can have the following options:

1. IR has not changed. Our shard can ignore this UC.
2. UC certifies an input from our shard, this input has never been certified before, and round number is incremented. This UC finalizes a round and starts a new round.
3. Round number is incremented, but state root hash remains the same (repeat UC). This UC starts another consensus attempt extending the same state as previous (likely failed) one.
4. UC is newer, certifying a future state. Indicates to a validator that it is behind the others and must roll back the pending proposal and initiate recovery.

If the latest UC certifies a state of this shard, then the certification response delivering UC determines the leader and starts a new round. While accepting incoming requests, the leader starts assembling his next block proposal, extending the latest finalized round (round with a valid UC).

When timer t_1 runs out the leader stops accepting new requests, finishes state updates and broadcasts a block proposal to followers and then sends Certification Request to the BFT Core. See Figure 7.

BFT Core has a timer t_2 for every shard within every partition; it is reset when a valid UC for this shard is issued. If this timer has run out, then a *repeat UC* is issued with incremented round number. This initiates a new consensus attempt for the shard. New round is executed with a different leader. Nodes can determine which UC is the latest based on round number. A block proposal, generated by the leader, includes a UC and this

UC must point to this leader. Round is finalized when its UC is embedded into the block. The retry mechanism is illustrated by Fig. 8.

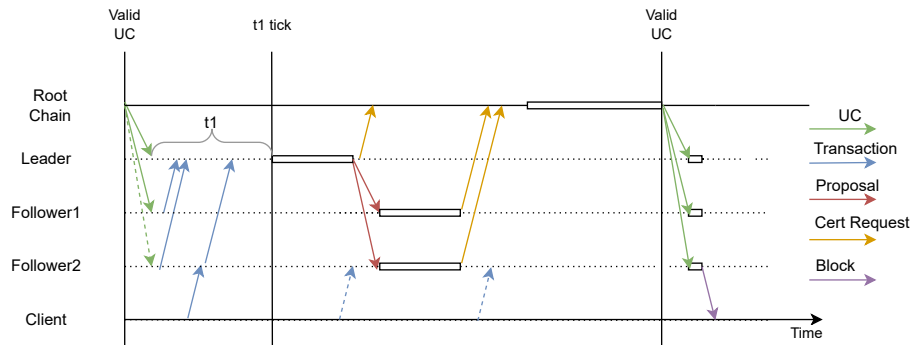


Figure 7. Successful Shard Round

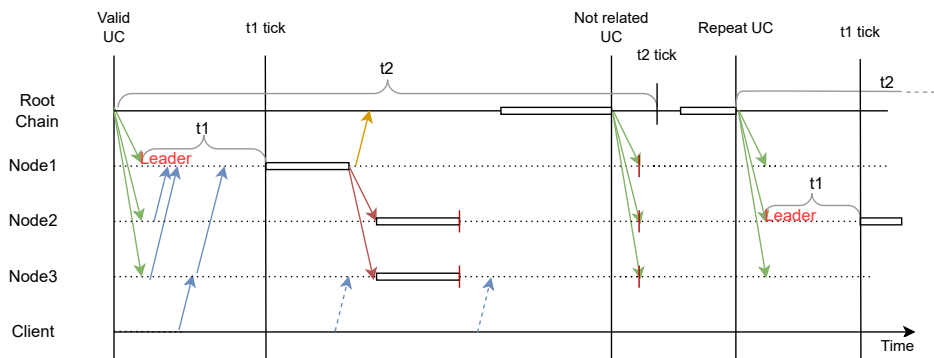


Figure 8. Shard Round attempt which did not produce a valid UC for this shard

5.2.2 Configuration and State

Configuration (managed by the Coordination Process) of every validator includes:

- Network Identifier (α).
- Partition Identifier (β).
- Shard Identifier (σ); present for multi-shard partitions.
- Aggregation Layer Node Identifier (ν). There are $k_{\beta,\sigma}$ nodes in shard σ of partition β .
- Timeout value $\tau 1$: after a node sees a UC which appoints a leader, the leader waits for $\tau 1$ time units before stopping accepting new Unicity Service Requests and creating a block proposal.

Communication layer:

- secret key used to sign messages
- related public key; known to other validators and the BFT Core
- public keys of other validators within the shard
- communication addresses of other validators within the shard

- communication addresses of the BFT Core validators

Data layer:

- Unicity *Trust Base* ($\mathcal{T}$)
- other *partition* defining parameters; refer to Unicity Platform Specification, State of a Shard.

Variables:

- v_l : Current round leader's identifier; NULL if not known
- buf : buffer with pending Unicity Service Requests
- N : State Tree
- cp : State Tree checkpoint, helps the State Tree to roll back to previously certified state if a state extending attempt fails. Checkpoints can be released when a following round gets finalized.
- luc : latest UC. Importantly, this structure encapsulates the *state hash* of the last certified state, to be extended by the round finalization attempt.
- lte : latest Technical Record, certified by luc . Carries the required *round number*, leader identifier, epoch of the round finalization attempt.
- log : log of verified and executed (but not final) requests; respective changes in State Tree can be rolled back by reverting it to checkpoint cp .
- pr : Pending Certification Request waiting for UC; includes state tree hash and applied requests and round number as the time reference used for request validation. We are avoiding situation where there can be multiple pending requests and speculative validation; fresh UC invalidates all pending requests. There may be multiple parallel pending requests in future extensions.
- $\mathcal{B}$: the shared append-only log.

The variables are handled via state transitions like this:

1. *Initial state*. State tree is certified with ' luc '; log and pr are empty; cp points to the current state.
2. *After applying any request(s)*: There are changes in the state tree; executed requests are recorded in *proposal* (that is, cp is the starting point and N and log are updated in sync). Processing of requests continues.
3. *Waiting for UC*. This state is reached on t_1 click, after sending a BlockProposalMsg message (if being the leader) and sending a Certification Request. There are changes in the state tree on top of snapshot cp ; executed requests were recorded in log ; Now, root hash of the state tree and respective log are saved in pr , which extends luc . pr must be preserved as long as it is possible that it gets certified by a UC. No new requests are processed in this state.
4. *After receiving a CReS message with new UC*:
 - UC certifies pr : block is finalized and added to $\mathcal{B}$. OK to clear.
 - UC is '*repeat UC*': state is rolled back to cp ; we assume simplified case that consensus for prev. request is not possible any more and clean pr .

Algorithm 1 State and Initialization

Constants:

α : Network Identifier

β : Partition Identifier

σ : Shard Identifier

ν : Node Identifier

$k_{\beta,\sigma}$: number of nodes in the shard σ of partition β

$\mathcal{T}$: Unicity Trust Base, may evolve

Variables:

$\nu_l \leftarrow \text{NULL}$: leader node identifier of the current round; NULL if not known

$N \leftarrow \{\}$: SMT (Sparse Merkle Tree)

$\text{cp} \leftarrow \perp$: SMT Checkpoint

$\text{luc} \leftarrow \text{NULL}$: latest valid UC

$\text{lte} \leftarrow \text{NULL}$: latest technical record sent with latest UC

▷ $\langle \text{round number to be certified} \rangle = \text{lte}.n$

▷ $\langle \text{last certified hash} \rangle = \text{luc}.IR.h$

$\text{buf} \leftarrow \{\}$: input requests buffer

$\text{log} \leftarrow \{\}$: executed requests log for proposal creation

$\text{pr} \leftarrow \text{NULL}$: pending proposal corresponding to a CR request waiting for UC

sr : statistical record data. See Statistical Record

ℓ_B : number of new SMT leaves added during round

ℓ_S : SMT memory usage in bytes

π_{CP} : optional consistency proof (hash-based or ZK-compressed)

$\mathcal{B}$: shared log

function $\text{START_NEW_ROUND}(uc, te)$

 ensure($\text{VERIFYUNICITYCERT}(uc, \mathcal{T})$)

 ensure($uc.h_t = h(te)$)

 ensure($te.n > \text{luc}.IR.n$)

 ensure($uc.IR.h' = \text{luc}.IR.h$)

▷ Double-checking

$\text{cp} \leftarrow \text{CHECKPOINT}(N)$

$\text{RINIT}()$

$\text{log} \leftarrow \{\}$

$\text{pr} \leftarrow \{\}$

$\nu_l \leftarrow te.\nu_l$

$\text{luc} \leftarrow uc, \text{lte} \leftarrow te$

$\text{RESET_TIMER}(\tau_1)$

if $\nu_l = \nu$ **then**

▷ Leader

$\text{PROCESS}(\text{buf})$

▷ Process for no longer than until τ_1 tick

$\text{buf} \leftarrow \{\}$

else

▷ Follower

if $\text{SEND_INPUTFORWARDMSG}(l, \text{buf})$ **then**

$\text{buf} \leftarrow \{\}$

▷ Clean buffer on successful connection

end if

end if

end function

function $\text{LEADERFUNC}(uc)$

return $\{\nu_i \mid i \leftarrow \text{integer}(H(uc)) \bmod k_{\beta,\sigma} + 1\}$

▷ Simplest example

end function

expired Unicity Service Requests. There can be a light-weight partial validation, referred as *sanity check*, before continuing with the processing. If the leader is not known, or rejects messages, then keep Unicity Service Requests in a buffer and try again when the next leader is known and accepts Unicity Service Requests. If the validator is the current leader, then he processes available Unicity Service Requests immediately. At the moment when a leader can not include transactions into a proposal anymore, or have collected enough transactions to fill a block, it starts rejecting incoming Unicity Service Requests from other validators.

A validator should retain a Unicity Service Request if accepted from a client or other validator; until it is either expired or included into a finalized block. A validator may forget a Unicity Service Request if accepted by another validator. A validator should not forward a Unicity Service Request to a distinct validator more than once.

Validators may limit the number of times a Unicity Service Request is forwarded.

Please refer to Algorithm 2 for an example without optional functionality.

5.2.3.2 Block Proposal

Summary: On clock tick, stop immediate validation and execution of incoming Unicity Service Requests. Validate and execute Unicity Service Requests from the Request Buffer, updating the State Tree (N) and *log* for proposal creation. Executed requests from *log* go into Block Proposal, in the exactly same order they were validated and executed. Broadcast Block Proposal to Follower Nodes. Create and send Uniqueness Certificate Request, retaining necessary state in a Pending Block Proposal (*pr*) data structure.

A round must extend a previously certified round. If a party approves a block proposal, then it also approves the entire history. This ensures safety of the protocol.

Pending Block Proposal (*pr*) must be stored in durable way before Certification Request can be sent, by e.g. writing it to persistent storage. Losing all copies of pending block proposals, while obtaining a UC for this round, would be an unrecoverable error.

Please refer to Algorithm 3 for details.

Note that a block can be without any requests; however, this does not necessarily imply $h' = h$, as system-initiated “housekeeping” actions may have changed the state.

5.2.3.3 Validation and Execution

Sanity checking of Unicity Service Requests is quick and lightweight validation, with the main goal of protecting system resources by early detection of obvious garbage. All Unicity Service Requests will be fully verified later before actual execution. Thoroughness of sanity checking is a tuning parameter.

Validating Unicity Service Requests is performing their full verification, according to Unicity Platform Specification, section Unicity Service Request Validation, and performing partition specific additional checks. The requests must appear in the *proposal* in the same order. Requests without interdependencies (i.e., affecting distinct state) can be executed in parallel. Invalid requests are not executed and not included into produced proposal.

Algorithm 2 Input Handling

```
upon message <TransactionMsg |  $Q$ > do
  if SANITY_CHECK( $Q$ ) then
    if  $v_l = v$  then                                ▶ This process is the leader
      PROCESS( $\{Q\}$ )                                ▶ Beware of parallel execution
    else if  $v_l \neq \text{NULL}$  then                    ▶ We know someone else is the leader
      if  $\neg \text{SEND\_INPUTFORWARDMSG}(v_l, Q)$  then    ▶ Forward to  $v_l$ 
         $buf \leftarrow buf \cup Q$                     ▶ Store on failure
      end if
    else                                              ▶ Buffer requests until leader is known
       $buf \leftarrow buf \cup Q$ 
    end if
  end if
end upon

upon message <InputForwardMsg |  $reqs$ > do
  if  $v_l = v$  then                                ▶ This process is the leader
    defer PROCESS( $reqs$ )
    return "accepted"
  else
    return "reject"
  end if
end upon

on event next_leader_elected do
  PRUNE_EXPIRED( $buf$ )
  if  $v_l = v$  then                                ▶ This process is the leader
    PROCESS( $buf$ )                                ▶ Beware of parallel execution
  else
    if  $\text{SEND\_INPUTFORWARDMSG}(v_l, buf)$  then    ▶ Forward to  $v_l$ 
       $buf \leftarrow \{\}$                             ▶ Forget on successful send
      ▶ ... or keep in "forwarded" buffer and use when becoming the leader
    end if
  end if
end on
```

Algorithm 3 Producing a Block Proposal

```
on event  $t1$  do
  if  $v_l = v$  then                                ▶ This validator is the leader
     $v_l \leftarrow \text{NULL}$ 
    RCOMPL()                                ▶ Request processing must have stopped by now
     $sr \leftarrow \text{PRODUCESTATISTICS}()$ 
     $\text{SEND\_BLOCKPROPOSALMSG}(\alpha, \beta, \sigma, v, luc, lte, log, sr)$     ▶ Sign and Broadcast
     $\text{DO\_CERT\_REQ}(log, v, sr)$ 
  end if
   $v_l \leftarrow \text{NULL}$                                 ▶ Leader stops accepting new txs
end on
```

Algorithm 4 Producing a Certification Request

```

function DO_CERT_REQ( $txs, v_l, sr$ )
   $h' \leftarrow luc.IR.h, n \leftarrow lte.n, e \leftarrow lte.e, t \leftarrow luc.C^r.t$ 
   $h \leftarrow \text{STATE\_ROOT}(N)$ 
   $\ell_B \leftarrow \text{count of new SMT leaves added}, \ell_S \leftarrow \text{SMT memory usage}$ 
   $pr \leftarrow (n, e, h', h, t, v_l, log, sr)$  ▷ Pending Block Proposal
  if STORE_IN_DURABLE_WAY( $pr$ ) then
     $IR \leftarrow (n, e, h', h, t)$ 
     $\pi_{CP} \leftarrow \text{GENERATE\_CONSISTENCY\_PROOF}(h', h, \text{batch})$  ▷ Generate consistency proof
    SEND_CR( $\alpha, \beta, \sigma, v, IR, \ell_B, \ell_S, \pi_{CP}$ ) ▷ Sign and send
  else
    ▷ Do nothing as the current node can not guarantee data availability
    ▷ The round may get finalized though thanks to other partition nodes
  end if
end function

function DO_CERT_REQ_AGAINST( $sr$ ) ▷ Voting against the proposal
   $h' \leftarrow luc.IR.h, n \leftarrow lte.n, e \leftarrow lte.e, t \leftarrow luc.IR.C^r.t$ 
   $h \leftarrow \perp$ 
   $IR \leftarrow (n, e, h', h, t)$ 
   $\ell_B \leftarrow 0, \ell_S \leftarrow 0$  ▷ No additions when voting against
  SEND_CR( $\alpha, \beta, \sigma, v, IR, \ell_B, \ell_S, \perp$ ) ▷ No consistency proof needed
end function

```

Algorithm 5 Validate and Execute Unicity Service Requests

```

function SANITY_CHECK( $Q; t \leftarrow lte.n$ ) ▷ Requests will be fully validated later
  return  $Q.pk \neq \text{NULL}$  ▷ Basic validation of request structure
end function

function VALIDATE( $Q; n \leftarrow lte.n, t \leftarrow luc.C^r.t$ )
  ▷ Omitted, see Platform Specification, Unicity Service Request Validation
end function

function PROCESS( $reqs$ ) ▷ Should be implemented as processing queue
  for all  $Q \in reqs$  do
    if VALIDATE( $Q$ ) then
      EXECUTE( $N, Q$ ) ▷ Can be executed in parallel
       $log \leftarrow log \cup Q$ 
    end if
  end for
end function

```

The expiration of Unicity Service Requests is checked relative to shard round number. Validation context includes also the time value t obtained from previous UC and to be recorded as current round's $UC.IR.t$.

Refer to Algorithm 5 for details.

5.2.3.4 Processing an Unicity Certificate and Finalizing a Round

Summary: On receiving a UC, round is finalized, and a new round is started.

More specifically,

1. UC is verified cryptographically according to the Framework Specification. Partition and shard identifiers are checked.
2. The time-stamp in UC is checked for sanity: it must not “jump around” and it must reasonably match the local time if it can be reliably determined. UC with a suspicious time-stamp must be logged, and processing continues because rejecting a UC may end with a deadlock of the shard.
3. UC consistency is checked:
4. UC is checked for equivocation, that is, for arbitrary uc and uc' , the following must hold:

$$\begin{aligned}
 uc.IR.n = uc'.IR.n &\Rightarrow uc.IR = uc'.IR \\
 uc.IR.h' = uc'.IR.h' &\Rightarrow uc.IR.h = uc'.IR.h \\
 &\quad \vee uc.IR.h' = uc.IR.h \vee uc'.IR.h' = uc'.IR.h \\
 uc.IR.h = uc'.IR.h &\Rightarrow uc.IR.h' = uc'.IR.h' \\
 &\quad \vee uc.IR.h' = uc.IR.h \vee uc'.IR.h' = uc'.IR.h \\
 uc.IR.n = uc'.IR.n + 1 &\Rightarrow uc.IR.h' = uc'.IR.h \\
 uc.IR.n < uc'.IR.n &\Rightarrow uc.C^r.n < uc'.C^r.n
 \end{aligned}$$

On failing any of these checks, an equivocation proof must be logged with all necessary evidence.

5. UC round number and epoch number can not decrement.
6. On unexpected case where there is no pending round certification request, recovery is initiated, unless the state is already up-to-date with the UC.
7. Alternatively, if UC certifies the pending round certification request then round is finalized.
8. Alternatively, if UC certifies the round whose pending proposal tried to extend (‘repeat UC’) then state is rolled back to the previous state.
9. Alternatively, recovery is initiated, after rollback. Note that recovery may end up with newer last known UC than the one being processed.
10. Finally, on valid UC (validation reached the 3 alternatives above), a new round is started.

Please refer to Algorithm 6 for details. Round Finalization is presented in Algorithm 8.

On arbitrary timeout / lost connection: re-establish connection to the BFT Core.

If a block can not be saved and made available during the finalization, then the round must be not closed. This ensures that a) the block can be restored based on saved proposal and UC during the recovery process, and b) the node can not extend the round with non-persisted block of inputs.

Algorithm 6 Processing a received Unicity Certificate

```

upon message <CReS |  $\alpha, \beta, \sigma, te; UC$ > do
  ensure(VERIFYUNICITYCERT( $uc, \mathcal{T}$ ))
  ensure( $H(te) = UC.h_t$ )
  if GOT_NEW_UC( $UC$ ) then
    START_NEW_ROUND( $UC, te$ )
  end if
end upon

function GOT_NEW_UC( $uc$ )
  ensure( $uc.IR \neq luc.IR$ )                                ▷ Ignore UC certifying the same
  ensure( $uc.C^r.\alpha = \alpha$ )
  ensure( $uc.C^{uni}.\beta = \beta$ )
  if  $\neg$ CHECKSANITY( $uc.C^r.t, uc.C^r.n, GetUTCDateTime()$ ) then
    Log( $uc$ )                                              ▷ Rejecting a UC with strange time would break the shard
  end if
  ensure(NON_EQUIVOCATING_UCS( $uc, luc$ ))
  ensure( $uc.IR.n > luc.IR.n$ )                                ▷ Check late to catch equivocation
  if  $pr = \text{NULL}$  then                                    ▷ No pending Certification Request
    if  $uc.IR.h \neq \text{STATEROOT}(N)$  then
      RECOVERY( $uc$ )
    end if
  else
    if  $uc.IR.h = pr.h \wedge uc.IR.h' = pr.h'$  then
      FINALIZE_ROUND( $pr, uc$ )
    else if  $uc.IR.h = pr.h'$  then
      REVERT( $N, cp$ )
    else
      REVERT( $N, cp$ )
      RECOVERY( $uc$ )
    end if
  end if
  return 1
end function

```

5.2.3.5 Processing a Block Proposal

Summary: Upon receiving a BlockProposalMsg message, create a rollback checkpoint and then validate the signature and header fields, execute Unicity Service Requests from the proposal and updating the State Tree (N). Executed requests go into Block Proposal. Create and send Uniqueness Certificate Request message, and retain a Pending Block Proposal data structure.

This procedure is performed by the non-leader validators. There are following steps (See Fig. 10):

1. Block proposal is checked: valid signature, correct partition and shard identifier, valid UC, the UC must be not older than the latest known by the validator. Sender must be the leader for the round started by included UC and match the leader identifier field.

Algorithm 7 Checking two UC-s for equivocation

```
function NON_EQUIVOCATING_UCS( $uc, uc'$ )
```

ensure($uc. IR.n \geq uc'. IR.n$) ▶ To simplify, assume that uc is not older than uc'

$$\text{ensure}(uc.C^r.n_r > luc.C^r.n_r)$$

if $uc.IR.n = uc'.IR.n$ **then**

ensure($uc.IR = uc'.IR$) ▶ On all failures log uc and uc' as proof

end if

if $uc.IR.h' = uc'.IR.h' \wedge uc'.IR.h' \neq uc'.IR.h$ **then**

ensure($uc.IR.h = uc'.IR.h$) ▷ A hash can be extended only with one hash

end if

if $uc. IR.h = uc'. IR.h$ **then** ▷ ... and vice versa

$$\text{ensure}(uc.IR.h' = uc'.IR.h')$$

```
end if
```

if $uc.IR.n = uc'.IR.n + 1$ **then**

$$\text{ensure}(uc. IR.h' = uc'. IR.h)$$

```

end if

```

```

return 1

```

end function



Figure 10. Data Flow of a non-leader aggregation shard node

Algorithm 8 Finalizing a Round

```

function FINALIZE_ROUND( $pr, uc$ )
     $\mathcal{B} \leftarrow \mathcal{B} \cup (pr.tx; uc)$  ▷ Recording a new batch to shard's log
end function

```

2. If included UC is newer than latest UC then the new UC is processed; this rolls back possible pending change in state tree. If new UC is 'repeat UC' then update is reasonably fast; if recovery is necessary then likely it takes some time and there is no reason to finish the processing of current proposal.
3. If the state tree root is not equal to one extended by the processed proposal then processing is aborted and negative vote is delivered.
4. All Unicity Service Requests in proposal are validated; on encountering an invalid Unicity Service Request the processing is aborted and negative vote is delivered.
5. Transaction orders are executed by applying them to the state.
6. Pending certificate request data structure is created and persisted.
7. Certificate Request query (CR) is assembled and sent to the BFT Core—that is, positive vote for the proposal.

As an optimization, it is possible to vote against a proposal by sending a negative vote, where proposed state's hash is $\perp$. This helps the BFT Core to determine that convergence to consensus is not possible before waiting until time-out.

Please refer to Algorithm 9 for details.

5.2.3.6 Ledger Replication

Relatively independent subsystem for serving and replicating ledger data. Pseudocode of the service is provided in Algorithm 10.

On receiving log, the blocks are verified using embedded UC-s and cryptographic links. See Platform Specification, function `VerifyBlock()`.

5.2.4 Recovery Procedure

If a validator is behind then it must use recovery procedure to sync its state with other validators, and obtain the latest UC for this shard, whose authoritative source is the BFT Core.

Summary: Missing log entries are fetched from other validators, validated, and applied to the state tree. A pending block proposal, if certified but not finalized, is applied and finalized.

It is assumed that the state tree is already rolled back by calling `REVERT( $N, cp$ )` if it had requests of a not finalized block applied.

In more details:

1. Input UC is validated,
2. Missing log entries are fetched from other (random) validator(s),

Algorithm 9 Processing a received Block Proposal

```
upon message <BlockProposalMsg |  $m = (\alpha, \beta, \sigma, v, uc, te, txs, sr; s)$ > do
  ensure(VALID( $m$ ))                                ▷ Consistent and authorized message
  ensure(VERIFYUNICITYCERT( $uc, T$ ))
  ensure( $uc.h_t = h(te)$ )
  ensure( $m.v = te.v_l$ )                                ▷ Signed by authorized leader
  ensure( $m.\alpha = \alpha \wedge m.\beta = \beta \wedge m.\sigma = \sigma$ )
  ensure( $m.te.n \geq lte.n$ )
  if  $\neg$ CHECKREQUESTSTATISTICS( $sr$ ) then
    DO_CERT_REQ_AGAINST(NULL)                        ▷ Invalid data, vote against
    return
  end if
  if  $m.te.n > lte.n$  then
    ensure(GOT_NEW_UC( $m.uc, m.te$ ))                ▷ Newer UC must be validated and processed
    if processing of new UC took too much time (recovering?) then
      return START_NEW_ROUND( $m.uc, m.te$ )
    else
       $luc \leftarrow m.uc, lte \leftarrow m.te$ 
    end if
  end if
   $h' \leftarrow m.uc.IR.h$ 
  if STATEROOT( $N$ ) =  $h' \wedge \{ \forall Q \in m.reqs \mid \text{VALIDATE}(Q) \}$  then
     $cp \leftarrow \text{CHECKPOINT}(N)$ 
    RINIT()
    for all  $Q \in m.reqs$  do
      EXECUTE( $N, Q$ )
    end for
    RCOMPL()
    DO_CERT_REQ( $m.reqs, m.v, sr$ )                    ▷ Vote for
  else
    DO_CERT_REQ_AGAINST( $sr$ )                        ▷ Vote against
  end if
end upon
```

Algorithm 10 Ledger Replication

```
upon message <LedgerReplication |  $\alpha, \beta, \sigma, n_1, n_2 \leftarrow luc.IR.n; s$ > do
  ▷ First authorization and sanity check,
  return  $\{\mathcal{B}_{\alpha, \beta, \sigma, n} \mid n \in [n_1 \dots n_2]\}$ 
end upon
```

3. Each request block is verified: cryptographically using embedded UC, and for correct partition and shard ID;
4. Each request within the block is validated,
5. Requests are applied to the state tree,
6. Last known UC is updated if a block has newer one.

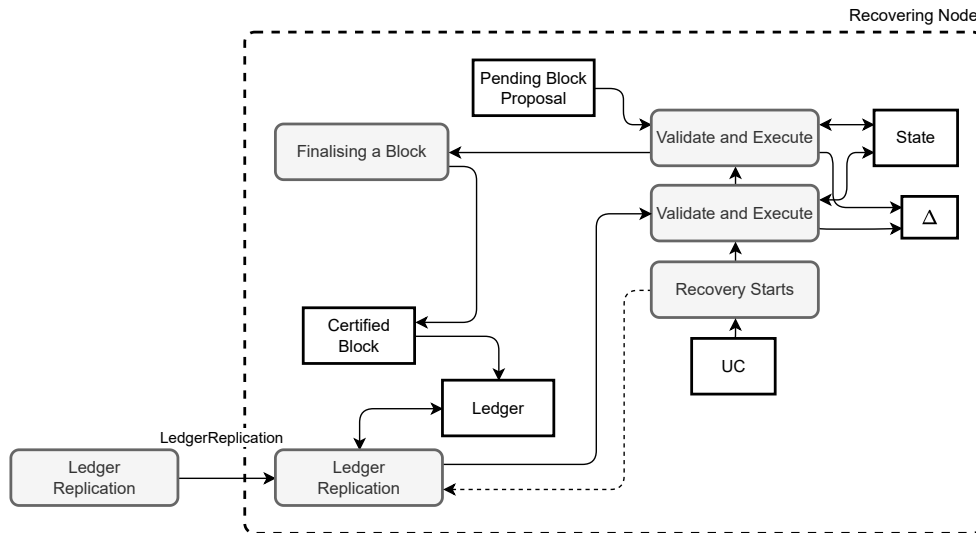


Figure 11. Data Flow of an out-of-sync (recovering) Shard Node

7. Then, if there is a pending block proposal which can be finalized using freshly obtained UC then it will be applied to current state and round is finalized.

Please refer to Algorithm 11 for full details. Recursive recovery is used to mark locations where last-resort failover/retry happens. More intelligent failover and back-off mechanism could be used, with gracious shut-down on unrecoverable situations.

5.2.5 Protocols – Shard Validators

5.2.5.1 Protocol TransactionMsg – Unicity Service Request Delivery

Users deliver their Unicity Service Requests to one or more validators (to account for byzantine validators censoring or re-ordering requests).

Message: $\langle \text{TransactionMsg} \mid Q \rangle$

5.2.5.2 Protocol CR – Round Certification Request

This section extends Sec. 4.3.1, Certification Request).

If h' is already 'extended' with UC then the latest UC is returned immediately via CReS message; otherwise validation and UC generation continues, UC is returned via CReS when available.

If h' is unknown to BFT Core then the latest UC is returned immediately via CReS.

Returned message has a technical data record, which may trigger a view change: next consensus attempt with incremented round number and different leader. An aggregation layer node can have only one pending CR per round number; subsequent messages are ignored. A message with higher round number is always preferred.

This message “subscribes” the validator to receive UC messages for a certain period, either a fixed number (e.g. 2 rounds), or until the shard have successfully proposed a following

Algorithm 11 Shard Node Recovery

```
function RECOVERY( $uc$ )                                ▶ Assuming that Revert() is done by caller
  ensure(VERIFYUNICITYCERT( $uc, \mathcal{T}$ ))
  for all  $b \in \text{SEND\_LEDGERREPLICATIONREQUEST}(luc.IR.n + 1)$  do ▶ To a random live validator
    if VERIFYBLOCK( $b, \mathcal{T}$ ) then                                ▶ Assuming request blocks are ordered
      ensure( $b.\alpha = \alpha \wedge b.\beta = \beta \wedge b.\sigma = \sigma$ )
      ensure( $b.UC.IR.n > luc.IR.n$ )
      ensure( $\text{STATEROOT}(N) = luc.IR.h = b.IR.h'$ )
      ensure( $\{ \forall T \in b.txs \mid \text{VALIDATE}(T, luc.IR.n + 1) \}$ )
       $cp \leftarrow \text{CHECKPOINT}(N)$ 
      RINIT()
      for all  $T \in b.txs$  do
        EXECUTE( $N, T$ )
      end for
      RCOMPL()
      if  $\text{STATEROOT}(N) \neq b.IR.h$  then
        REVERT( $N, cp$ )
        return RECOVERY( $uc$ )                                ▶ Failover
      end if
       $\mathcal{B} \leftarrow \mathcal{B} \cup b$ 
       $luc \leftarrow b.UC$                                 ▶ Respective  $lte$  arrives with proposal
    else
      return RECOVERY( $uc$ )                                ▶ Failover
    end if
  end for
  if  $uc.IR.h' = \text{STATEROOT}(N)$  then                                ▶ Apply pending request if possible
     $pr \leftarrow \text{FETCH\_PR\_FROM\_PERSISTENT\_STORAGE}(pr)$ 
    if  $pr \neq \text{NULL} \wedge pr.h' = uc.IR.h' \wedge pr.h = uc.IR.h$  then
      ensure( $uc.IR.n = pr.n$ )
      ensure( $\{ \forall T \in pr.txs \mid \text{VALIDATE}(T, pr.n) \}$ )
       $cp \leftarrow \text{CHECKPOINT}(N)$ 
      RINIT()
      for all  $T \in pr.txs$  do
        EXECUTE( $N, T$ )
      end for
      RCOMPL()
      if  $\text{STATEROOT}(N) = uc.IR.h$  then
        FINALIZE_BLOCK( $pr, uc$ )
         $luc \leftarrow uc$ 
      else
        REVERT( $N, cp$ )
      end if
    end if
  end if
end function
```

round, that is, there is another set of BFT Core validators which have received a quorum of CR messages and therefore “taken over” the subscription.

Message: $\langle CR \mid \alpha, \beta, \sigma, \nu, IR, sr; s \rangle$

If an aggregation layer node has reasons to suspect that BFT Core have generated a new UC, then he must try to fetch it by trying again. *sr* stands for Statistical Record; a technical data structure sent from aggregation layer nodes to the BFT Core.

5.2.5.3 Protocol RoundCertificationResponse (CReS)

This section extends Sec. 4.3.2, Certification Response).

CReS is asynchronous response (in the sense of data flow) to Certification Request (CR); there may be many CReS responses to one client request. TE stands for Technical Record, sent from the BFT Core to aggregation layer nodes. Valid if $UC.h_t = h(TE)$.

Optional field RootTrustBaseEntry indicates, that UTB $\mathcal{T}$ have changed, e.g., have grown by the provided entry.

Message: $\langle CReS \mid \alpha, \beta, \sigma, UC, TE, [RootTrustBaseEntry] \rangle$

5.2.5.4 Protocol Subscription – subscribing to CReS messages

This message “subscribes” the validator to future CReS messages, without presenting a Certification Request in the form of CR message. Synchronous response is the latest UC for requestor.

Subscription ends when the shard have successfully proposed a following round, that is, there is another set of BFT Core validators which have received a quorum of CR messages and therefore serving a subscription.

Query: $\langle SubscriptionMsg \mid \alpha, \beta, \sigma, \nu; s \rangle$

Response: $\langle CReS \rangle$

5.2.5.5 Protocol InputForwardMsg – Input Forwarding

Forward a set of Unicity Service Requests.

Message: $\langle InputForwardMsg \mid \{Q\} \rangle$

5.2.5.6 Protocol BlockProposalMsg – Block Proposal

Leader broadcasts its input block proposal to other partition nodes.

Message: $\langle BlockProposalMsg \mid \alpha, \beta, \sigma, \nu_l, uc, te, txs, sr; s \rangle$
where $txs = \{T\}$

5.2.5.7 Protocol LedgerReplication – Ledger Replication

Let’s assume that we have a separate layer of components implementing the ledger storage. Entire ledger can be verified based on latest finalized and available round and every block of inputs can be verified based on the Unicity Trust Base.

This protocol is provided by every functional aggregation layer node and dedicated *archive nodes*; arbitrary parties can join the latter.

Query: $\langle \text{LedgerReplication} \mid (\alpha, \beta, \sigma n_1, [n_2]) \rangle$

Response: $(\{B\})$

If 2nd number is missing then return everything till head. It is possible, that a reply misses some newer rounds, either because the queried node is behind or it prefers to return input blocks by smaller chunks.

5.2.5.8 Protocol GetTrustBase – Unicity Trust Base Distribution

The Unicity Trust Base is a chain of records, one per epoch, containing the BFT Core validator set configuration and forming a verifiable root of trust. This protocol allows nodes to obtain Trust Base entries for a specified epoch range.

Each Trust Base entry corresponds to one epoch and contains validator identities, stakes, quorum requirements, and cryptographic linkage to the previous epoch's entry through hashing and signatures. The chain structure enables verification: given an authentic base entry, subsequent entries can be verified by checking signatures and hash chains.

Nodes request Trust Base entries by specifying an epoch range. The response includes all entries from the starting epoch up to (and including) the ending epoch, or up to the latest available entry if no ending epoch is specified.

Query: $\langle \text{GetTrustBase} \mid \alpha, e_1, [e_2] \rangle$

where:

- α – network identifier of type $\mathbb{A}$
- e_1 – starting epoch number of type $\mathbb{N}_{64}$
- $[e_2]$ – optional ending epoch number of type $\mathbb{N}_{64}$

Response: $\langle \text{TrustBaseResponse} \mid \mathcal{T} \rangle$

where $\mathcal{T} = \langle T_{e_1}, T_{e_1+1}, \dots, T_{e_2} \rangle$ is a sequence of Trust Base entries.

Each Trust Base entry T_e (for epoch e) is a tuple:

$$T_e = (\alpha, e, n_e, \{v, b_v\}_e, k_e, r, h_{\text{cr}}, h_{e-1}, s_{e-1})$$

as defined in Table 3.

If the ending epoch e_2 is omitted, the response includes all entries from e_1 up to the latest committed epoch. If the requesting node specifies an epoch range beyond what is currently available, the response includes entries up to the latest committed epoch only.

Verification: Upon receiving Trust Base entries, the requester must verify the chain:

1. Verify that the first entry T_{e_1} connects to an already-authenticated Trust Base entry (either the genesis entry or a previously verified entry)
2. For each subsequent entry $T_{e_{i+1}}$, verify:
 - 2.1 $h_{e_i} = H(T_{e_i})$ (hash chain linkage)

2.2 Signature s_{e_i} is valid over $T_{e_{i+1}}$ using validator set from epoch e_i

2.3 Epoch numbers increment correctly: $T_{e_{i+1}}.e = T_{e_i}.e + 1$

This protocol is provided by all active BFT Core validators. Validators maintain the complete Trust Base chain from genesis and serve requests for any epoch range. The protocol supports both initial synchronization (requesting from genesis) and incremental updates (requesting only recent epochs).

5.3 BFT Core State Machine

5.3.1 Summary

Leader-based BFT consensus based SMR. Roughly, BFT Core validators:

1. Validate incoming Certification Requests: signature correctness, they must extend shard's previous UC, and partition specific checks must pass.
2. Forward shard's requests to the BFT Core leader
3. BFT Core leader verifies shard's requests (incl. majority), produces UC tree, signs.
4. BFT Core leader sends requests (all Certification Requests including signatures) and trees and signature to BFT Core validators.
5. Followers verify shard's requests (incl. majority), create trees, sign.
6. Followers distribute their signatures to other BFT Core validators
7. On reaching $k - f$ (there are k validators in the BFT Core) unique signatures all BFT Core validators send ack to others
8. On reaching $k - f$ ack-s all BFT Core validators commit and return responses to the aggregation layer nodes.

5.3.2 Timing

BFT Core serves many partitions and shards, each implemented as a logical group of nodes. Operation cycles work like this:

1. Shards operate in parallel:
 - Shard validators send RoundCertificationRequest requests.
 - Once there is a quorum of requests for a shard, the BFT Core updates an entry in the array of input records.
2. *Eventually*, BFT Core starts Unicity Certificate generation. Fills data structure. Computes root. Signs. In distributed case this all takes some time.
3. Individual tree certificates and UCs are generated for participating partitions. Responses are returned to individual validators.

'Eventually' above is a compromise: 1) there is no sense to start a new round too fast, it is necessary to collect some input requests to certify; 2) input requests with quorum should be served as fast as possible to improve latency; 3) no need for further wait if all inputs are present; 4) some inputs might struggle with quorum; no need to wait for the long tail; 5) in order to generate 'repeat UCs' a round must be restarted after τ_2 time units even when there are no requests.

Configuration parameters are: target round rate t_b and shard wait τ_1 .

System parameters are: average root Certificate generation time t_r ; average shard round processing time from receiving UC to sending Certification Request, including τ_1 wait: t_p ; with standard deviation σ_p . There are k shards.

Let m – how many BFT Core rounds fit into one shard round; $m \approx (t_r + t_p)/t_b$.

Let's denote cumulative (normal) distribution of RoundCertificationRequest query messages with $\Phi_\sigma(x)$. Assume $m = 1$, let's find a minimum of $(2 - \Phi_\sigma(t_b - (t_r + t_p)))(t_b)$ by adjusting t_b and τ_1 .

Practical rule for optimizing the average latency (Fig. 12):

Start BFT Core round when completed quorum ratio is $t_b/\Delta \approx t_b/m(t_b - (t_r + t_p))$. Note that t_b measures time from the moment when UC generation starts, thus it is circular and a rolling average must be used Median can be found by ignoring overflow from previous round and then waiting for completion of half of the quorums. Time from round start to next median is roughly $t_r + t_p$. Adjust τ_1 if t_b needs to be changed.

Even more practically:

1. Maintain a rolling average of t_b and Δ
2. Do not count the pending overflow from previous round
3. After counting half of expected quorums ($k/2$) start timer for measuring Δ and re-start timer for measuring t_b
4. After counting kt_b/Δ unique quorums stop timer Δ and start UC generation.

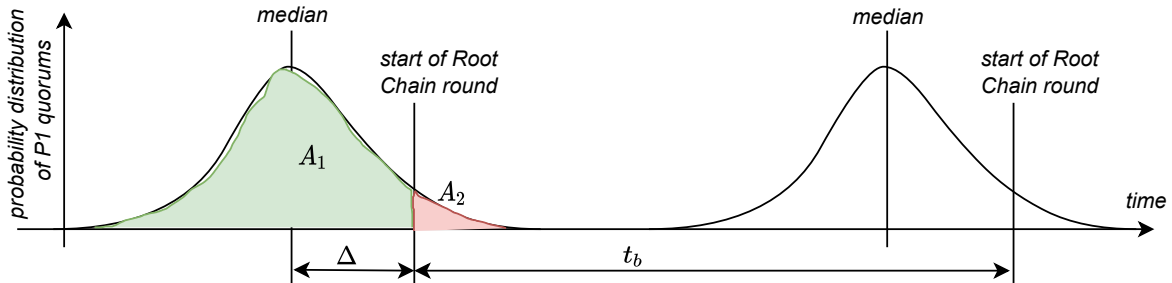


Figure 12. Optimal average latency is achieved when $\frac{A_1}{A_2} \approx \frac{t_b}{\Delta}$, where A represents the respective area (number of messages). $t_b - \Delta \approx t_r + t_p$.

If the distribution gets too large it makes sense to increase m , that is, have more than one core round per one average shard's round.

5.3.3 State

Please refer to Platform Configuration, State of the BFT Core, and Algorithm 12.

Configuration is provided by the Governance and other management processes. State must be persisted and synchronized (recoverable from other BFT Core validators).

Time-outs and timers:

- τ_2 – BFT Core waits for τ_2 time units before re-issuing a UC for a shard. This triggers a new round retrying certification of a block of inputs, with another leader. There is one timer instance per shard, referred as $\tau_{2,\sigma}$ for the shard σ of the partition β .
- τ_3 – Triggers Unicity Tree re-calculation and UC response generation (monolithic BFT Core only); Target Round Rate (t_b) – Distributed BFT Core specific parameter

Communication layer:

- Validator's secret key used to sign its messages
- related public key; known to other BFT Core validators via the Governance Records, and to other parties via the $\mathcal{T}$
- public keys of other BFT Core validators (used by the underlying communication layer)
- public keys of Shard Validators (usage captured by the opaque function `VALID()`)
- communication addresses of other BFT Core validators
- `conn[][]` – Connection contexts to return responses to validators which sent a RoundCertificationRequest request

5.3.4 Analysis

5.3.4.1 Safety

BFT Core enforces that each round is 'extended' by one round only. This excludes conflicting rounds (forks). BFT Core must not generate "equivocating" UCs (see Section 5.2.3.4 and Algorithm 7).

5.3.4.2 Liveness

Usual properties of a partially synchronous communication model apply.

5.3.4.3 Data Availability

If a validator issues a RoundCertificationRequest call then it must not lose the block proposal (request data) until the block is finalized and committed to persistent storage.

The mechanism of 'repeat UCs' presents a challenge: one block may receive multiple waves of extending attempts, initiated by subsequent 'repeat UCs'. There are following options:

1. Latest UC use is enforced strictly. If the BFT Core have issued a repeat UC, then all RoundCertificationRequest requests must be based on round number suggested by this response. Arriving and pending RoundCertificationRequest requests, referring to different round numbers, are dropped.
2. All RoundCertificationRequest requests extending the current state are considered; if there is a pending request from a validator then it is replaced by later request from this validator, with larger round number.
3. All RoundCertificationRequest requests extending the current state are considered, no matter the proposed round number, and included into pending request buffer; if

one proposed new state achieves majority then it wins; this state does not have to be the latest one.

On second and third option it is possible, that an aggregation layer node have issued multiple RoundCertificationRequest requests and eventually an older one gets certified. Thus, validators must retain all pending block proposals until one is committed.

These options provide somewhat better liveness of the protocol; in reality, if it is the case we can safely assume that time-out t_2 has too low value relative to system latencies. The rest of this specification assumes option 1.

Active aggregation layer nodes must retain their state until successfully—demonstrated by the ability to produce subsequent blocks—replaced by another set of validators after some epoch change. The long-term availability is provided by interested parties like RPC nodes, archive nodes, client middleware; these components must be able to replicate blocks with reasonable effort from the active validators.

5.3.5 Monolithic Implementation

This section (Algorithms 12, 13, 16) defines a monolithic, non-distributed BFT Core implementation. It serves distributed shards.

For a deployable implementation, please refer to Distributed BFT Core (Section 5.3.6); it provides the same functionality to shards without being a single point of failure, and can provide necessary level of decentralization.

Algorithm 12 Global Parameters and Variables

Configuration:

α : Network Instance identifier

$\mathcal{T}$: Unicity Trust Base

$C[]$: Partition Identifiers

$CD[]$: Partition Description Records

State:

$n \leftarrow 0$: BFT Core's Round number

$e \leftarrow 0$: BFT Core's Epoch number

$r_- \leftarrow \text{NULL}$: Previous round's Unicity Tree root hash

$SI[][]$ - shard info of every $\sigma \in \mathcal{SH}_\beta$ of every $\beta \in C$

Variables:

$\forall \beta \in C, \sigma \in \mathcal{SH}_\beta: IRT_\beta[\sigma] \leftarrow (\perp)$

χ_β : the shard tree for every $\beta \in C$ (Sec. 4.1.5)

v : unicity tree (Sec. 3.8.3)

$changes[][] \leftarrow \perp$ Changes to IR, applied at the end of round, indexed by partition ID and shard number

$req[][] \leftarrow \perp$: RoundCertificationRequest requests, indexed by partition ID, shard number and validator ID

$conn[][] \leftarrow \perp$: Shard validator connections

Timers:

RESET_TIMER(t_3)

$\forall \beta \in C, \forall \sigma \in \mathcal{SH}_\beta: \text{RESET_TIMER}(t_{2,\beta,\sigma})$

5.3.5.1 Certification Request Processing

Certification Request (CR) processing starts with sanity checking of the message. Then, checking if a newer UC is available; if yes then it is returned immediately. This would initiate shard recovery if necessary, and start a new shard round.

If a request tries to extend an unknown state, then the latest UC is returned immediately.

Next, the request is retained in request buffer if it is the first valid message; if the message is a repeating message then processing stops.

Equal requests (comparing entire IR to make sure that other fields also match) from the same shard are counted. If a request achieves simple majority then respective *IR* gets added to the changes array waiting for certification, at position indexed by shard ID. If it is clear, that a shard can not converge to a majority agreement, then the slot in changes array is filled with *IR* from the certified IR array, with incremented round number and previous certified hash set the same as the certified hash. This produces a 'repeat UC' which, once delivered downstream, initiates a new shard consensus attempt.

See Algorithm 13.

5.3.5.2 Unicity Certificate Generation

Periodically, do the following:

1. In case a shard has not shown progress for a period τ_2 since the last UC was delivered then respective slot is populated with the field content of previous certified IR array, with incremented round number. This produces a 'repeat UC'.
2. The pending changes in *changes* array are applied to *IR*. If there are no new changes then the previous value is used.
3. Based on the array of Shard Input Records, build the Shard Trees.
4. Based on the roots of Shard Trees, build the Unicity Tree. Extract root hash value. Obtain wall clock time. Create the Unicity Certificate.
5. For every record in IR changes array, respond to the shard based on request context from the request buffer. Then, clean up *req* buffer, and reset respective τ_2 timer.
6. Finally, reset *changes*, update the previous root hash used for linking⁴, increment BFT Core round number and reset the timer τ_3 which triggers the Unicity Certificate Generation (Algorithm 16).

5.3.6 Distributed Implementation

Distributed BFT Core is bisimilar to monolithic BFT Core in the sense that they provide the same service implementing the same business logic. Distributed BFT Core is Byzantine fault tolerant. It uses the SMR (State Machine Replication) concept.

The messaging between a shard and the BFT Core is illustrated by Figure 13, where a shard with two validators v_1 and v_2 requests a UC. BFT Core is also depicted with two validators, the next leader after reaching a quorum of shard requests is v'_1 .

⁴Note that linear hash-linking using r_- illustrates the idea that some sort of cryptographic linking is present; actual mechanism depends on the BFT Core implementation and used linking scheme.

Algorithm 13 CR Message Handling.

```
upon message <CR |  $m = (\alpha, \beta, \sigma, v, IR, \ell_B, \ell_S; s; [\pi_{CP}]); context$ > do
  ensure(VALID( $m$ ))
  ensure( $m.\alpha = \alpha$ )                                ▷ Right network id
  ensure( $\beta \in C$ )                                    ▷ Valid part. id
  ensure( $\sigma \in CD[\beta].SH$ )                            ▷ Valid shard id
  ensure( $v \in SI[\beta, \sigma].V$ )                        ▷ Authorized validator
   $conn[\beta][\sigma][v] \leftarrow context$                 ▷ Network connection for returning messages
  if  $IR.n \neq SI[\beta, \sigma].n + 1$                     ▷ Round is behind/ahead
     $\vee IR.e \neq SI[\beta, \sigma].e$                     ▷ Epoch can be incremented by RP only
     $\vee IR.h' \neq SI[\beta, \sigma].h$                     ▷ Extending of unknown state
     $\vee \neg CD[\beta].\gamma_{CP}(IR.h', IR.h, \ell_B, \pi_{CP})$     ▷ Consistency proof verification
     $\vee IR.t \neq SI[\beta, \sigma].UC_{-}.C'.t$  then        ▷ Time not set as expected
      SEND_CRES( $context; \beta, \sigma, SI[\beta, \sigma].UC_{-}, GET\_TE(SI[\beta, \sigma])$ )
      return
    end if
  if  $req[\beta, \sigma][v]$  then                            ▷ Reject duplicate request
    return
  end if
  ENSURE(TX-SYSTEM-SPECIFIC-CHECKS( $CD[\beta], IR$ )))
   $req[\beta, \sigma][v] \leftarrow IR$                         ▷ Add the new message
   $c \leftarrow \max_{r \in req[\beta][\sigma]} \sum_{p \in req[\beta][\sigma]} [r = p \wedge r.h \neq \perp]$     ▷ Number of the max. matching votes,
   $k \leftarrow |SI[\beta, \sigma].V|$                         ▷ Ignoring  $\perp$ , the “negative vote”
   $k \leftarrow |SI[\beta, \sigma].V|$                         ▷ Number of validators
  if  $c > k/2 \wedge \neg changes[\beta, \sigma]$  then          ▷ Consensus
     $changes[\beta, \sigma] \leftarrow CERTIFY(IR, SI[\beta, \sigma])$ 
  else if  $|req[\beta, \sigma]| - c \geq k/2$  then              ▷ Consensus not possible
     $changes[\beta, \sigma] \leftarrow REPEAT\_CERT(SI[\beta, \sigma])$     ▷ If max.match + |yet missing votes| < threshold
     $changes[\beta, \sigma] \leftarrow REPEAT\_CERT(SI[\beta, \sigma])$     ▷ Produce ‘repeat UC’
  end if
end upon
```

5.3.6.1 Summary of Execution

The summary follows the processing flow of a Certification Request by a BFT Core validator. The flow is illustrated in Fig. 14; loosely moving counter-clockwise.

Peer Node Selection In order to use the distributed BFT Core, aggregation layer nodes must choose an appropriate subset of BFT Core validators to communicate with. The set must be shared by all aggregation shard nodes during one shard round; on receiving a “repeat UC”, the validators must communicate with a different subset. The number of validators in this set is a tuning parameter: balances between availability and overhead. 2-3 validators is a good starting point. This number (in the sense of delivering messages in parallel) does not have security implications, as produced quorum sets retain aggregation layer nodes’ signatures which can be verified independently. UC responses must be checked for equivocation.

If an aggregation layer node haven’t received a UC-s from chosen BFT Core validators

Algorithm 14 Certification helper functions

```

function CERTIFY( $ir : \mathbb{IR}, si : \mathbb{SI}$ )
   $si \leftarrow \text{UPDATE\_SHARD\_INFO}(ir, si)$             $\triangleright$  Update caller's Shard Information record  $si$ 
   $\triangleright$  According to Platform Specification, functional description of BFT Core
   $te : \mathbb{TE} \leftarrow \text{GET\_TE}(si)$ 
  return ( $ir, te$ )
end function

function REPEAT_CERT( $si : \mathbb{SI}$ )
   $si.n \leftarrow si.n + 1$             $\triangleright$  Changing the parent's data structure as well
   $si.v_l \leftarrow \text{LEADERFUNC}(si.UC\_., si.V)$ 
   $rir : \mathbb{IR} \leftarrow (si.UC\_IR)$             $\triangleright$  Repeat the previous UC's IR
   $te : \mathbb{TE} \leftarrow \text{GET\_TE}(si)$ 
  return ( $rir, te$ )
end function

function GET_TE( $si : \mathbb{SI}$ )
  return ( $si.n + 1, si.e, si.v_l, h(si.SR\_., si.SR), h(si.VF\_., si.VF)$ )
end function

```

Algorithm 15 "SubscriptionMsg" subscribes to UC feed

```

upon message <SubscriptionMsg | ( $\alpha, \beta, \sigma, v; s$ ); context> do
  ensure(VALID( $(\alpha, \beta, \sigma, v; s)$ ))
   $conn[\beta, \sigma][v] \leftarrow context$     $\triangleright$  Network connection for returning subsequent messages
  return GET_UC( $\beta, \sigma$ )
end upon

```

within $2 \times \tau_2$ then it sends SubscriptionMsg requests to random other BFT Core validators. This ensures eventual synchronization if the firstly chosen validators happen to be faulty.

CR Validation No difference from Monolithic Implementation, please refer to Algorithm 13 for details. If available, or on invalid request, UC is returned immediately by the same BFT Core validator.

Shard Quorum Check Shard Quorum Check is the same as on Monolithic case (Alg. 13). If a quorum is achieved or considered impossible, then a message is assembled (IRChangeReq), which includes all CR messages, and forwarded to the next BFT Core leader, using the Atomic Broadcast submodule.

IR Change Request Validation The request must be validated analogously to the Algorithm 9.

Proposal Generation Leader collects all unique IR change requests and assembles the block proposal, which includes changed IR-s and a justification for each IR change request. Next, if a particular IR has not been changed during τ_2 timeout for this shard then the leader initiates "repeat UC" generation by including a specific record into the block proposal. There is no explicit justification, followers can validate timeouts based on their own timers.

In a proposal there can be up to one change request per shard.

Algorithm 16 Unicity Certificate Generation

on event t_3 **do** ▷ Simplified, see section “Timing”
 for all $\beta \in C$ **do** ▷ Process partition timeouts
 for all $\sigma \in SH_\beta$ **do**
 if $\neg changes[\beta, \sigma] \wedge \text{EXPIRED_TIMER}(t_{2_{\beta, \sigma}})$ **then**
 $changes[\beta, \sigma] \leftarrow \text{REPEAT_CERT}(SI[\beta, \sigma])$ ▷ Produce ‘repeat UC’
 end if
 end for
 end for
 for all $(\beta, \sigma, ir, te) \in changes$ **do** ▷ Apply changes
 $IRT_\beta[\sigma] \leftarrow (ir, h(te))$
 end for
 for all $\beta \in C$ **do**
 $\chi_\beta \leftarrow \text{CreateShardTree}(SH_\beta, IRT_\beta)$ ▷ Sec. 4.1.5.1
 $I\mathcal{H}[\beta] \leftarrow \chi_\beta(\perp)$
 end for
 $r \leftarrow \text{CreateUnicityTree}(C, CD, I\mathcal{H})$ ▷ Sec. 4.1.6.1
 $t \leftarrow \text{GETUTCDATE TIME}()$
 $C^r \leftarrow \text{CREATEUNICITYSEAL}(\alpha, n, e, t, r, r; sk_r)$
 for all $(\beta, \sigma, ir, te) \in changes$ **do**
 $req[\beta][\sigma] \leftarrow []$
 $C^{\text{shard}} \leftarrow \text{CREATESHARDTREECERT}(\sigma, \chi_\beta)$
 $C^{\text{uni}} \leftarrow \text{CREATEUNICITYTREECERT}(\beta, C, CD, I\mathcal{H})$
 $uc \leftarrow (IRT_\beta[\sigma].IR, IRT_\beta[\sigma].h_t, C^{\text{shard}}, C^{\text{uni}}, C^r)$
 $SI[\beta, \sigma].UC_- \leftarrow uc$
 for all $connection \in conn[\beta][\sigma]$ **do** ▷ To all validators of the shard, in parallel
 $\text{SEND_CRES}(connection; \alpha, \beta, \sigma, uc, te)$ ▷ Subscriptions expire – see protocol
 desc.
 end for
 $\text{RESET_TIMER}(t_{2_{\beta, \sigma}})$
 end for
 $changes \leftarrow \{\}$
 $r_- \leftarrow r$
 $n \leftarrow n + 1$
 $\text{RESET_TIMER}(t_3)$
 ▷ Note that IRT retains its current value for the next round
end on

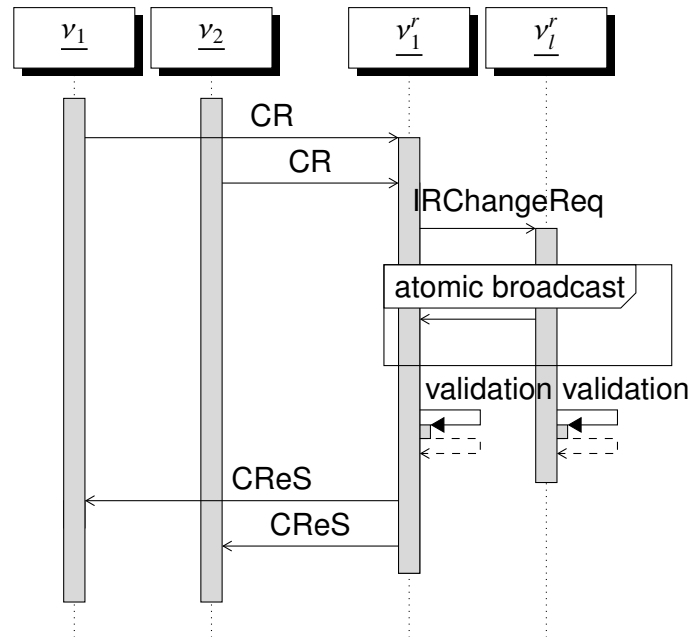


Figure 13. Message flow (simplified)

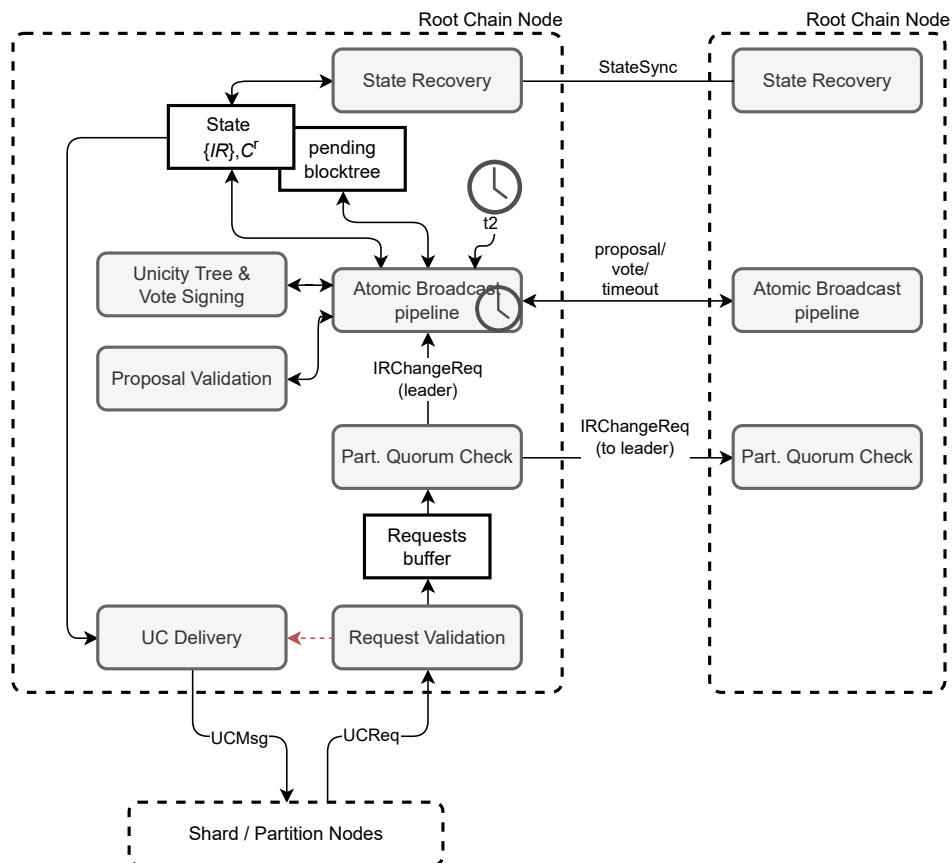


Figure 14. Data-flow of a Distributed BFT Core Validator

Due to the pipe-lined finality, a proposal should not include IR change requests for slots with a valid IR change request in immediately preceding round.

Finally, the leader signs the proposal and broadcasts it to other BFT Core validators.

Proposal Validation On receiving a proposal, validator validates it. There are consensus-specific checks. Every IR change request is validated based on its kind; base rules are presented by Algorithm 9. Summary of the cases:

Quorum achieved The justification must prove the achievement of consensus by an aggregation shard. More than half of the shard nodes must have coherent votes.

Quorum not possible The justification proves that there are enough conflicting votes to render the consensus impossible.

t2 timeout The message states, that its timer have reached the timeout; all validators can confirm the timeout based on their own clocks, allowing a little drift.

After validating the proposal and checking the Voting Rule, the validator signs its vote data structure and sends it to the next leader in pipeline.

On encountering unexpected state hash the recovery process is initiated (see Section 5.3.6.2).

State Signing On assembling a Quorum Certificate (QC) with enough votes and verifying the Commit Rule the leader modifies state: for every newly certified IR element it updates its last UC array.

UC Generation If a leader updates the last UC array element then it returns CReS responses to all pending aggregation shard nodes.

QC is included to HotStuff message pipeline, so it is broadcast to other validators (together with the new proposal produced by this validator). On seeing new QC and knowing re-certified IRs, other validators update their last UC arrays (the state).

5.3.6.2 Proposal

Proposal is a signed set of IR change requests, supplemented with proofs—the necessary number of signed partition nodes' messages (*justification*). There are following options:

- Change requests with justifications.
- Repeat Certification Request where consensus is considered impossible.
- Repeat Certification Request on t_2 timeout of a shard.

State Synchronization A BFT Core validator must have up-to-date vector of Input Records of all shards. There is no persistent blockchain. Returned Unicity Certificates, certifying IRs, are possibly persisted by the underlying layers.

The state includes necessary meta-data like the round number. This is provided by including the Unicity Seal, which also authenticates the IR vector.

State may include atomic broadcast module specific data, e.g. uncommitted round information.

5.3.6.3 Atomic Broadcast Primitive

The Atomic Broadcast primitive is instantiated using an adaptation of HotStuff consensus protocol. The adaptation is optimized towards better latency on good conditions. Therefore, a “2-chain commit rule” is used. Changes and tweaks wrt. the original HotStuff paper are:

- “2-chain commit rule”
- Timeout Certificates for view change. This induces quadratic communication complexity on faulty leader, but enables the 2-chain rule instead of 3-chain.
- QC component votes go to the next leader directly and the next leader assembles QC.
- Use of aggregated signatures (instead of threshold signatures)

More formally, critical elements of the operation of a HotStuff-derived algorithm are specified by the following rules.

Let n denote round number, B – block of inputs, QC – Quorum Certificate, TC – Timeout Certificate.

Rule 1. Voting Rule

$B.n > \text{last vote round}$

$B.n = B.QC.n + 1 \vee (B.n = TC.n + 1 \wedge B.QC.n \geq \max(TC.tmo_high_qc_round))$

Rule 2. Timeout Rule

$n \geq \text{last vote round}$

$(n = QC.n + 1 \vee n = TC.n + 1) \wedge QC.n \geq \text{1-chain round}$

Rule 3. Commit Rule

It is safe to commit block B if there exist a sequential 2-chain $B \leftarrow QC \leftarrow B' \leftarrow QC$ such that $B'.n = B.n + 1$.

Please refer to the following papers for more details:

1. HotStuff: BFT Consensus in the Lens of Blockchain
2. DiemBFT v4: State Machine Replication in the Diem Blockchain

The concepts used in this specification map to the concepts used in the HotStuff and DiemBFT papers as follows:

State: Vector of Input Records

State Authenticator: State is identified by the root hash of Unicity Tree.

Block Proposal: List of changes to Input Records, with justifications; or a proposal to switch epochs.

Block: There are no (explicit) blocks. The set of Input Records can be seen as the cumulative state after applying all previous (virtual) blocks. Unicity Certificates are propagated downstream to shards where they could be saved as part of shard blocks. Validator implementation is encouraged to produce an audit log with all the block proposal payloads.

Blockchain: There is no such thing as the BFT Core Blockchain. However, Unicity Trust Base is somewhat blockchain-like: it gets a new entry added once per BFT Core epoch, and similarly to block headers, it can be interpreted as the Root of Trust.

Round Pipeline Committing the payload happens across many rounds due to the pipelined nature of HotStuff. In consecutive rounds, the flow is like this:

1. vector of IR change requests (payload of the proposal message)
2. updated IR-s (node block-tree) and Unicity Tree root (exec_state_id of a vote message)
3. committed Unicity Tree Root (commit_state_id of a vote message).

The output is commit_state_id from a Quorum Certificate which was formed by combining vote messages. QC is used to produce the Unicity Seal.

Pacemaker Pacemaker is a module responsible for advancing rounds, thereby providing liveness. Pacemaker sees votes from other validators and processes local time-out event.

Pacemaker either advances rounds on seeing a QC from the leader or on no progress, on seeing a TC. On local timeout or seeing $f + 1$ timeout messages a validator broadcasts signed TimeoutMsg message. TC is built from $2f + 1$ distinct TimeoutMsg messages. All messages must apply to currently known HighestQC.

The BFT Core should not tick faster than configured Target Round Rate. In order to throttle the speed, there is deterministic wait performed by leaders at every round.

The wait must be reasonably small to not trigger TimeoutMsg messages from other validators.

Leader Election Initially, a round-robin selection algorithm is used. In the roadmap there is stake weighted, unpredictable leader schedule.

Reputation is taken into account while producing per-epoch validator set assignments, for lowering the chance of inactive or unstable validator becoming a leader. One validator should not be the leader in two consecutive rounds.

Example: Take all validators. Remove one or more (fixed number) of the previous leaders. Remove all validators who did not participate in creation of the latest QC or TC. Pick one pseudo-randomly, and deterministically across all the validators, from the remainder.

5.4 Dynamic System

This section describes mechanisms making Unicity a dynamic system, by re-configuring partitions, shards and the BFT Core on the fly during execution. This introduces *epochs* – time periods where the configuration is stable. Configuration changes are executed at the switch of epoch. Partitions, shards and the BFT Core are not synchronized with each other, they have independent rounds and epochs.

Also, the changing nature of some global data structures is discussed below.

5.4.1 Configuration Changes

Table 1 provides a non-exhaustive list of configuration changes and the event synchronizing the change.

Table 1. Synchronizing configuration changes.

No	Change	RP epoch	Shard epoch
1.	Adding, removing aggregation layer nodes		✓
2.	Adding, removing BFT Core validators	✓	
3.	Communication address change of validators		
4.	Splitting (sharding) shards		✓
5.	Adding, removing partitions		✓
6.	RP's globally visible data structure changes (also increments versions of $\mathcal{T}$, C^x , see Sec 5.4.6.1)	✓	
7.	RP's operation rules	✓	
8.	Shard's data structure version changes resulting in changes in log content		✓
9.	Shard's data structure version changes resulting in changes in proofs		✓
10.	Changes in validation algorithms (ledger rules)		✓

Adding and removing of validators may incur changes in the respective quorum sizes. Communication layer changes are entirely handled by the communication layer. By “adding or removing a validator” we mean a change in actively participating validator identifier sets.

5.4.2 BFT Core Epoch Change

In order to facilitate a dynamic, responsive BFT Core, it is necessary to adjust its parameters on the fly. In particular, it is necessary to add new validators and retire some existing ones to maintain a healthy validator set, due to changing requirements and operating conditions.

The configuration can be changed once per epoch. The source information comes from an Governance Process whose output is change records called Validator Assignment Records (Table ??).

Any BFT Core protocol leader can initiate an epoch change, given it has received the change record. The procedure works as follows:

1. The leader produces a proposal where the usual payload is replaced by the Change Request justifying the increment of Epoch (Table 2);
2. Validators who approve the epoch change continue with execution flow and do not include usual payload until the epoch change proposal gets committed.
3. Next round after committing an epoch change is the first round of this epoch: Epoch in BlockData is incremented; and execution continues by the updated set of validators.

Fields 1, 2, 3 are copied from the Validator Assignment Records. The change record is a “justification”: it is used as helper data for validators, but not included into the finalized record and resulting entry in the Unicity Trust Base. An implementation may choose to rely on alternative channels for distributing change records.

BFT Core's epoch change updates the Unicity Trust Base. On record-oriented Unicity Trust Base, the new record becomes part of Root Validator's state (see section 5.4.6.6); and the

Table 2. BFT Core Epoch Change Request.

No	Field	Notation	Type
1.	Epoch number	e	$\mathbb{N}_{64}$
2.	Validator identifiers and stakes	$\{v, b_v\}_e$	$\{(\text{OCT}^*, \mathbb{N}_{64})\}$
3.	Quorum size (Voting power)	k_e	$\mathbb{N}_{64}$
4.	Hash of state summary	r	$\mathbb{H}$
5.	Hash of Change Record	h_{cr}	$\mathbb{H}$
6.	Hash of the previous record	h_{e-1}	$\mathbb{H}$
7.	(attached) Change Record		(Table ??)

new record gets propagated to Shard validators together with the next Unicity Certificate, as an extra field of the UCResp message.

Rule. BFT Core Epoch Change

For every proof, the Epoch Number in its Unicity Certificate's Unicity Seal must point to the entry in Unicity Trust Base which can be used for this proof's verification.

Validators must not execute invalid change records and approve proposals with invalid Change Requests. Instead, the latest valid change record must be used, whenever available; skipping over of some if necessary.

5.4.3 Shard Epoch Change

Shard Epoch Change is triggered by the BFT Core, by incrementing the Epoch number field in Technical Record, returned with a UC.

The next shard round after finalizing a round with UC with incremented Epoch value is processed according to the configuration of the next epoch. Leader and validators are selected according to next epoch configuration.

Rule. Shard Epoch Change

UC with incremented Epoch number in IR can be extended by a quorum of validators of the next epoch.

5.4.4 Controlling Shard Epochs

BFT Core triggers shard epoch changes. This happens in the following steps:

1. BFT Core validators obtain the Change Record for the next epoch of a shard.
2. If a BFT Core Leader includes an Input Record Change Request of a shard into a proposal and there is a pending epoch change of this shard, then the Change Record is included to the Request.
3. Presence of Change Record is the signal that Epoch should be incremented. Included Change Record decision is validated, and if valid then epoch number in Technical Record (TE) is incremented. All other IR, TE changes are validated and applied as well.
4. IR, TE-s get certified.

5. New CReS message is returned to the shard.
6. Shard's configuration is updated: quorum size (voting power needed for consensus), list of validators. This changes BFT Core's validation rules: the next Certification Request must be presented by a valid quorum of next epoch's aggregation shard nodes.

Rule. Shard Epoch Change Control

If a shard's state is certified by a UC with incremented Epoch number in certified TE, then the next shard round's requests get validated by the new epoch's configuration.

For the clarity of presentation, epoch handling is not present in the provided pseudocode. It supports the lower layer functionality of dynamic configuration changes.

5.4.5 Validator's life cycle

This section describes epoch changes from the viewpoint of an individual node, in particular, how a node joins and leaves shards.

Node accepts configuration updates through the Config API. The message is a set of change records, one per epoch. The records should cover the range from node's view of the current epoch to the epoch where node's status changes. The possible status changes (in respect to the current factual state of the node) are covered below.

It is assumed that the node have made itself already findable via the node discovery service.

5.4.5.1 Shard Validator, joining

Initially, a shard node is not an active validator. It receives a configuration message which indicates, that starting from epoch $e_{\beta,\sigma}$, this node is a validator in shard σ of partition β . Following procedure is executed:

1. Reset
 - Clean up state and storage, unless already synchronized to the required shard
2. Start accepting proposals
 - On incoming proposal message:
 - Extract the UC and update the last known UC if newer
3. Start accepting client requests
 - On incoming client transaction:
 - If it is possible to determine the leader then forward client requests to the leader.
 - Reject otherwise.
4. Obtain the latest UC for the shard (β, σ) from the BFT Core.
5. Launch recovery if behind
6. Subscribe to log feed of the shard (β, σ) .
 - On arrival of a new block of inputs:
 - Verify the inputs, apply the inputs to state and store in ledger
 - If block's UC indicates that the expected epoch arrives:
 - Starting from the next round, this node is full validator

- Execution and message handling continues as a full validator

If the input block log streaming protocol is not available the procedure is as follows:

1. Reset
Clean up state and storage, unless already synchronized to the required shard
2. Start accepting proposals
On incoming proposal message:
 - Extract the UC and update the last known UC if newer
3. Start accepting client requests
On incoming client transaction:
 - If it is possible to determine the leader then forward client requests to the leader.
 - Reject otherwise.
4. **loop:**
 - Obtain the latest UC from the BFT Core
 - Launch recovery if the node is behind
 - If the UC indicates that the epoch where node is a full validator have started:
 - this node is now a full validator,
 - exit loop.

The state graph is depicted at Figure 15.

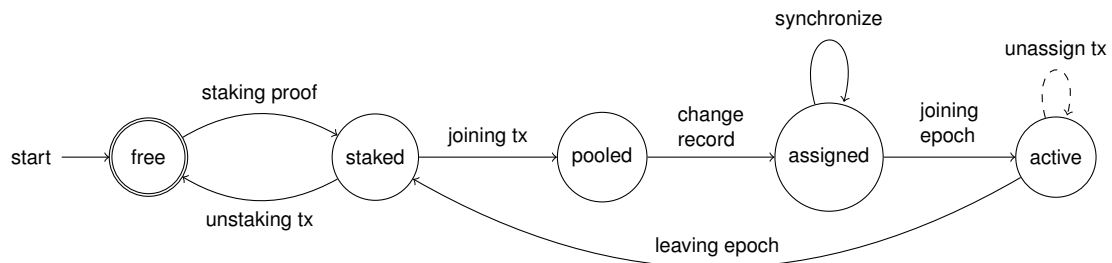


Figure 15. Node state graph (node's view)

5.4.5.2 Shard Node, leaving

If the node is an active validator and it receives indication that from an epoch e this node is not any more a member of shard (β, σ) , then the following procedure is executed:

On receiving a UC indicating the expected epoch update:

1. Starting from the next round, this node is not a validator.
2. Stop accepting proposals.
3. Stop accepting client request messages.
4. OK to reset the state.

5. Continue serving the log replication protocol; it is mandatory to retain the storage until the next quorum have committed at least one round.
6. OK to reset the log.

The node (more specifically the node operator) can initiate the leaving process by sending an “unassign transaction” to the Governance Body. Processing may take arbitrary time, and the unassignment may happen for other reasons as well. Validators are encouraged to not drop off without executing the unassignment process.

Now, the node may submit a request to the Governance Body to be assigned again as an active node, possibly at another shard. The node is encouraged to keep its identity.

Rule. Every Shard Node must ensure the availability of generated ledger until the next validator set takes over.

5.4.5.3 Shard Node, ambiguous records

If there is a double assignment before leaving then the validator must continue at the current shard. When eventually the validator gets dismissed, it must join according to the latest valid change record.

5.4.5.4 BFT Core Node, joining

On receiving a change record:

1. Obtain the current BFT Core configuration via the GetUnicityTrustBase RPC call
2. Validate the updated Unicity Trust Base
3. Make itself findable via the node discovery service
4. Start listening to BFT Core proposal messages.

On receiving a proposal where ProposalMsg.BlockData.Epoch is incremented and the epoch includes the node:

- Use the BFT Core recovery protocol (GetStateMsg / StateMsg) to synchronize the state,
- Process the proposal,
- Start processing shard Certification Requests,
- Continue as full BFT Core validator.

5.4.5.5 BFT Core Node, leaving

A BFT Core node may leave the active validator set after performing a successful epoch change, where this node is not a member of the new validator set anymore. The node is responsible for successful hand-over: by making its state available to the new joining nodes briefly after the beginning of the next epoch, and offering the Unicity Trust Base distribution service without limitations to all current and near-future validators.

A BFT Core node (more specifically, the operator of the node) can initiate the leaving process by sending a specific “unassign transaction” to the Governance Body. Processing

may take arbitrary time, and the unassignment may happen for other reasons as well. BFT Core validators are strongly encouraged to not drop off without executing the unassignment process.

Rule. BFT Core Node must keep running until the next epoch's Unicity Trust Base entry gets committed.

5.4.6 Dynamic Data Structures

5.4.6.1 Versioning

The content of Unicity Seal and Unicity Trust Base, globally used data structures across the Unicity Platform, depend on Distributed BFT Core implementation details. These data structures must be versioned to accommodate necessary iterative changes while the BFT Core implementation roadmap is being executed. That is, $\mathcal{T} = (v, \cdot)$ and $C^r = (v, \cdot)$, where v is the version number and the rest depends on the version. In the following sections, we assume that a section shares the same version number and it is omitted for brevity.

Accordingly, the function `VerifyUnicitySeal` must be able to verify a recent subset of Unicity Seal versions, based on an authentic copy of the up-to-date Unicity Trust Base. This can be imagined as a wrapper, where the version number of input chooses the right implementation.

5.4.6.2 Evolving

Unicity Trust Base itself evolves (e.g., new records are added, while format/version stays the same) when the BFT Core validator set changes. Every evolved copy of Unicity Trust Base is cryptographically verifiable based on an older authentic copy of the Unicity Trust Base.

We denote the initial, authentic⁵ Unicity Trust Base as $\mathcal{T}_{\text{base}}$ and updated Unicity Trust Base as $\mathcal{T}$, and for each version of data structures define the function

$$\text{VerifyUnicityTrustBase}_{\mathcal{T}_{\text{base}}}(\mathcal{T}).$$

For every supported version of proofs an implementation of `VerifyUnicitySeal` and the relevant Trust Base must be provided. Only the latest version of Unicity Trust Base can evolve.

At the launch, the system is bootstrapped to a genesis state where the content of Unicity Trust Base, together with Genesis Blocks, are created via some off-chain social consensus process. The relevant data structures are the same, while references to previous states and signatures created by previous states are hard-coded to zero values.

5.4.6.3 Monolithic, Static BFT Core

We start with one BFT Core validator, which does not change (and can not change its keys).

⁵Authenticity is guaranteed by e.g., off-band verification of the Genesis Block and embedding the newest possible Unicity Trust Base into the verifier code during release management process, analogously to *certificate pinning*.

- $\mathcal{T} = (\alpha, \text{pk})$,
where α is the system identifier and pk is the public key of the BFT Core.
- $C^r = (\alpha, n_r, t_r, r_-, r; s)$,
as defined in the Platform Specification; s is a digital signature created using BFT Core's secret key.

function VERIFYUNICITYSEAL($r, C^r, \mathcal{T}$)

return ($\mathcal{T}.\alpha = C^r.\alpha \wedge r = C^r.r \wedge \text{Ver}_{\mathcal{T}.\text{pk}}(C^r, C^r.s)$) $\triangleright$ Calculated over all fields except signature

end function

function VERIFYUNICITYTRUSTBASE($\mathcal{T}_{\text{base}}, \mathcal{T}$)

return ($\mathcal{T}_{\text{base}} = \mathcal{T}$)

$\triangleright$ No changes are allowed

end function

5.4.6.4 Monolithic, Dynamic BFT Core

In the case of dynamic BFT Core the validator(s) can change at epoch boundaries. The identifier (public key) of new validator is signed by the current one, and this signed record is appended to the Unicity Trust Base.

- $T_e = (\alpha, e, \text{pk}_e; s)$
is Unicity Trust Base Record, where $s = \text{Sig}_{\text{sk}_{e-1}}(T_e)$ is a cryptographic signature over verification record of epoch e (calculated over all fields except signature), signed by the previous epoch's secret key of the BFT Core.
- $\mathcal{T} = (T_j, T_{j+1}, \dots, T_k)$,
where j is the first epoch and k is the latest epoch covered. In the pseudocode below, we use notation $\mathcal{T}[j] := (T \in \mathcal{T} : T.e = j)$, that is, the element pointing to the epoch number j . System instance identifier of every epoch is invariant: $\forall i, j: T_i.\alpha = T_j.\alpha$.
- $C^r = (\alpha, n_r, t_r, r_-, r; (s, e))$
as defined in the Platform Specification; s is a cryptographic signature created using BFT Core's secret key of the epoch e .

function VERIFYUNICITYSEAL($r, C^r, \mathcal{T}$)

if ($\mathcal{T}.\alpha \neq C^r.\alpha \vee r \neq C^r.r$) **then**

return 0

end if

if $\mathcal{T}[C^r.e] = \perp$ **then**

return error

$\triangleright$ No record in trust base for the epoch

end if

return ($\text{Ver}_{\mathcal{T}.\text{pk}}(C^r, C^r.s) = 1$)

$\triangleright$ Calculated over all fields except signature

end function

Note that the caller must ensure that a relevant record is present in Unicity Trust Base, that is, $\mathcal{T}[C^r.e] \neq \perp$. Obtaining a fresh Unicity Trust Base is covered by the Chapter Unicity Anterior.

function VERIFYUNICITYTRUSTBASE($\mathcal{T}_{\text{base}}, \mathcal{T}$)

$bmax = \max_{T \in \mathcal{T}_{\text{base}}} T.e$

$\triangleright$ Last epoch in trust base

$tmin = \min_{T \in \mathcal{T}} T.e$

$\triangleright$ First epoch number to be verified

if $tmin > bmax + 1$ **then**

```

    return error
end if
for  $T \in \mathcal{T}$  do
    if  $\mathcal{T}_{\text{base}}[bmax].\alpha \neq T.\alpha$  then
        return error
    end if
end for
for  $j \in \{tmin \dots bmax + 1\}$  do
    if  $\text{Ver}_{\mathcal{T}_{\text{base}}[j-1], \text{pk}}(\mathcal{T}[j], \mathcal{T}[j].s) = 0$  then
        return 0
    end if
end for
for  $j \in \{tmin + 1 \dots \max_{T \in \mathcal{T}} T.e\}$  do
    if  $\text{Ver}_{\mathcal{T}[j-1], \text{pk}}(\mathcal{T}[j], \mathcal{T}[j].s) = 0$  then
        return 0
    end if
end for
return 1
end function

```

► Not a continuous chain

► Non-invariant system id

► Verify based on available trust base

► Verify consistency of new

For efficiency, the user must cache the verification results. For example: 1) at the startup, the bundled trust base is checked for consistency, and 2) each time an evolved trust base is encountered, a) it is checked for equivocation, b) if the trust base is newer than the base and verified using the function `VerifyTrustBase`, then the base trust base is updated with new records from the new trust base (or substituted with the latest version if the implementation is accumulator-like).

This version is illustrative and not for implementation.

5.4.6.5 Distributed, Static BFT Core

Unicity Trust Base is a map of BFT Core validator identifiers to validator public keys, and a number q which specifies the required quorum size, i.e.,

- $\mathcal{T} = (\alpha, \{(i, \text{pk}_i) \mid i \leftarrow 1 \dots v_r\}, q)$ where $q \geq v_r - f$,
- $C^r = (\alpha, n_r, t_r, r_-, r; s)$ where $s = \{(i, s_i) \mid i \in (1 \dots v_r)\}$ and $|s| \geq q$.

Unicity Trust Base does not change as the BFT Core configuration is static.

```

function VERIFYUNICITYSEAL( $r, C^r, \mathcal{T}$ )
    if  $\mathcal{T}.\alpha \neq C^r.\alpha \vee r \neq C^r.r$  then
        return 0
    end if
    if  $\neg(\forall m, n \in 1 \dots |C^r.s|: C^r.s[m].i \neq C^r.s[n].i)$  then
        return 0
    end if
    if  $|C^r.s| \leq \mathcal{T}.q$  then
        return 0
    end if

```

► Duplicate signers

► No quorum

```

for  $(i, s) \in C^r.s$  do
  if  $(\text{Ver}_{\mathcal{T}, \text{pk}_i}(C^r, s) = 0)$  then
    return 0
  end if
end for
return 1
end function

```

▶ Calculated over all fields except signature
 ▶ Invalid signature
 ▶ Success

5.4.6.6 Distributed, Dynamic BFT Core

The Unicity Trust Base Record is defined by Table 3.

Table 3. Unicity Trust Base Record of Dynamic Distributed BFT Core.

No	Field	Notation	Type
1.	Network instance identifier	α	$\mathbb{A}$ (invariant)
2.	Epoch number	e	$\mathbb{N}_{64}$
3.	Epoch starting round	n_e	$\mathbb{N}_{64}$
4.	Validator identifiers and stakes	$\{v, b_v\}_e$	$\{(\text{OCT}^*, \mathbb{N}_{64})\}$
5.	Quorum size (voting power)	k_e	$\mathbb{N}_{64}$
6.	Hash of state summary	r	$\mathbb{H}$
7.	Hash of related change record	h_{Cr}	$\mathbb{H}$
8.	Hash of previous record	h_{e-1}	$\mathbb{H}$
9.	Signature of previous epoch validators	s_{e-1}	<i>version-dependent</i>

Fields 1, 2, 4 and 5 are copied from the referenced Governance Record. Initially, the stakes are fixed to 1. When appropriate orchestrating processes implementing (delegated) Proof of Stake mechanisms are in place, the stakes reflect the epoch's locked stake amounts of each particular validator.

Unicity Trust Base is a chain of records defined by Table 3.

Unicity Seal is a record signed by the validator set of the respective epoch as defined in 5.4.6.4, with an additional requirement of using the required multi-party signature scheme.

The verification functions are as defined in 5.4.6.4, where the signatures are interpreted in broader sense as multi-party signatures, created by respective quorums of validators.

5.4.6.7 Signature Aggregation

This is an optimization of Distributed BFT Core data structures, reducing the sizes of produced proofs and the trust base. It is based on a cryptographic primitive implementing the “non-interactive, accountable subgroup multi-signature”, allowing identification of all parties whose (part-) signatures are aggregated into a final, aggregate signature. On the case of aggregatable signature schemes, the m-of-n aggregation of public keys is non-trivial though⁶, thus, it may be implemented further down the roadmap.

Unicity Trust Base is a tuple of aggregate public key and a numeric parameter (q) specifying the necessary quorum size. Signature on Unicity Seal is an aggregate signature, produced

⁶See e.g., <https://eprint.iacr.org/2018/483>

by combining at least q partial signatures, and a bit-field identifying the signers. Partial signatures are created by individual BFT Core validators using their private keys.

A standardization attempt of the closest appropriate signature scheme is available from IETF — <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bls-signature/>.

Specifically, threshold signature schemes are avoided because of 1) accountability requirement 2) complicated and security-critical key setup, and 3) missing support of non-equal voting powers.

6 PoW Chain

The Proof-of-Work (PoW) chain is the decentralization backbone of the Unicity Consensus Layer. Together with the BFT Core, it forms the two-faceted consensus mechanism: the PoW chain provides permissionless participation, robustness, and long-term security, while the BFT Core provides fast deterministic finality for the Aggregation Layer.

Two cross-certification processes link the PoW chain and the BFT Core:

Snapshot Publishing (Sec. 6.2): BFT Core epoch transitions produce new Unicity Trust Base entries, which PoW miners embed into PoW blocks. This anchors the BFT Core state into the PoW chain.

Finality Gadget (Sec. 6.3): BFT Core validators collectively certify PoW blocks, providing deterministic finality to the PoW chain and enabling reward token minting.

The BFT Core operates at approximately one round per second with finality in two rounds. The PoW chain targets a block rate of one block per 8640 seconds (≈ 2.4 hours). The two layers operate at vastly different timescales, with the PoW chain providing slow but robust anchoring and the BFT Core providing fast finality for practical transactions, with compact and stable root of trust.

6.1 PoW Blockchain

6.1.1 Motivation

Decentralization of the Consensus Layer requires a mechanism for permissionless participation. A PoW chain provides this: any party can mine blocks without prior authorization. PoW also provides robustness—if any number of miners leave or join the network, the chain continues to grow with the block rate eventually adjusting to the new total mining power. These properties are essential during the bootstrapping phase of a decentralized system, when the number of validators fluctuates, the financial value of tokens is low, and token distribution is relatively concentrated.

PoW chains do not provide fast deterministic finality: many blocks of confirmations are needed to achieve reasonable certainty. This is mitigated by the BFT Core finality gadget (Sec. 6.3). By combining PoW with BFT consensus, Unicity leverages the strengths of both: PoW provides permissionlessness, robustness, and high security for the base currency, while BFT provides fast, deterministic finality for the Aggregation Layer.

BFT Core validators are selected from a pool of recent, high-performing PoW miners based on a deterministic algorithm and PoW chain content; anyone can execute the algorithm to verify the selection. PoW validators may also delegate their BFT layer validation rights.

6.1.2 Longest Chain Rule

The PoW chain follows the *heaviest chain rule*: the valid chain with the greatest cumulative proof-of-work is considered the canonical chain. When a node observes two competing forks, it adopts the one with more accumulated work.

Chain reorganizations occur when an alternative fork accumulates more proof-of-work than the current active chain:

1. Find the common ancestor (fork point) of the current and candidate chains.
2. Disconnect blocks from the active chain back to the fork point.
3. Connect blocks from the candidate chain forward from the fork point.
4. Update the active chain tip.

The Finality Gadget (Sec. 6.3) limits the practical depth of reorganizations by providing BFT-certified checkpoints.

6.1.3 RandomX Proof-of-Work

The PoW chain uses RandomX⁷, a proof-of-work algorithm designed to be efficient on general-purpose CPUs while resisting acceleration by GPUs and ASICs. RandomX achieves this through:

- **Memory-hardness**: Requires ≈ 2 GB dataset per epoch, making hardware optimization difficult.
- **Random code execution**: Generates and executes random programs, exploiting the full complexity of a modern CPU instruction set.
- **Epoch-based dataset**: The dataset changes periodically, preventing pre-computation.

By selecting an ASIC-resistant mining algorithm, participation in mining is democratized: commodity hardware can compete effectively, lowering the barrier to entry.

Block validation uses a layered approach (Alg. 17):

1. **Commitment check** (~ 1 ms): The `hashRandomX` field is verified against the difficulty target `nBits`. This fast check rejects invalid blocks before invoking the expensive PoW computation, providing DoS protection.
2. **Full RandomX verification** (~ 50 ms): The block hash $h_B = H(\text{fields } 1\text{--}6)$ is recomputed from the header fields (excluding `hashRandomX`), and `RANDOMX( $h_B$ )` is verified to equal `hashRandomX`.

6.1.4 Block Structure

The PoW chain is a *permissionless blockchain*. The primary data unit is the block, which includes the consensus-critical mining fields and the block payload (currently reward claim and BFT checkpoint).

⁷<https://github.com/tevador/RandomX>

6.1.4.1 Block Header

Each block consists of a fixed-size 112-byte header containing the consensus-critical fields, followed by a variable-length payload `vPayload`. The header commits to the payload via `payloadRoot`; the RandomX PoW commitment is derived from the block hash (fields 1–6) and stored in `hashRandomX`.

Table 4. PoW Block Header (112 bytes, fixed size).

No	Field	Size	Type	Description
1.	<code>nVersion</code>	4 B	<code>int32</code>	Block version number
2.	<code>hashPrevBlock</code>	32 B	$\mathbb{H}$	Hash of the previous block (H of fields 1–6 of that block)
3.	<code>payloadRoot</code>	32 B	$\mathbb{H}$	Root of the payload commitment tree (Sec. 6.1.4.2)
4.	<code>nTime</code>	4 B	<code>uint32</code>	Block timestamp (Unix epoch, seconds)
5.	<code>nBits</code>	4 B	<code>uint32</code>	Difficulty target in compact format
6.	<code>nNonce</code>	4 B	<code>uint32</code>	Mining nonce
7.	<code>hashRandomX</code>	32 B	$\mathbb{H}$	RandomX PoW commitment; derived from the block hash, set last

All integer fields use little-endian byte order. Fields 1–7, serialized contiguously in the order above, form the fixed-size 112-byte block header. The variable-length payload `vPayload` is appended after the header; it is *not* part of the block hash computation (its integrity is guaranteed via `payloadRoot`).

Block hash. The *block hash* $H(B)$ is defined over the first six header fields only, *excluding* `hashRandomX`:

$$H(B) = H(\text{nVersion} \parallel \text{hashPrevBlock} \parallel \text{payloadRoot} \parallel \text{nTime} \parallel \text{nBits} \parallel \text{nNonce}).$$

This 80-byte prefix is also the input to the RandomX function. The `hashRandomX` field is set to $\text{RANDOMX}(H(B))$ and stored in the header after mining.

6.1.4.2 Block Payload

The `payloadRoot` field commits to two payload elements via a hash tree. Let `cbor(·)` denote CBOR encoding⁸ and let T_e be the Unicity Trust Base entry for the current epoch (or the CBOR nil value $\perp$ if no new entry is published in this block). The computation and validation rule is defined as:

$$\text{payloadRoot} = H(H(\text{tokenId}_{\text{reward}}) \parallel H(\text{cbor}(T_e))).$$

When no Trust Base entry is published, $\text{cbor}(T_e)$ is the CBOR encoding of the nil value.

If $\text{cbor}(T_e)$ is non-nil, the validator must additionally verify T_e against the locally maintained, authentic Unicity Trust Base (Sec. 6.2).

⁸Concise Binary Object Representation (CBOR), RFC 7049.

Table 5. PoW Block Payload (vPayload), serialized as a CBOR sequence.

No	Field	Size	Description
1.	$H(\text{tokenId}_{\text{reward}})$	32 B	Reward commitment leaf: hash of the reward token identifier generated by the miner. The salt from which the token identifier was generated acts as a commit-reveal preimage: knowledge of it is what entitles a party to mint the corresponding reward token, hence the salt MUST be unpredictable to all other parties until the miner reveals it at minting time as the salt field of the mint transaction (Sec. 2.1.3.1).
2.	$\text{cbor}(T_e)$	variable	CBOR encoding of the Unicity Trust Base entry T_e for the new epoch, or CBOR nil if the UTB epoch has not changed since the last published checkpoint.

6.1.5 Difficulty Adjustment

The PoW chain uses the ASERT (Absolutely Scheduled Exponentially Rising Targets) difficulty adjustment algorithm, providing per-block difficulty recalculation:

$$\text{target}_{\text{new}} = \text{target}_{\text{ref}} \cdot 2^{(\Delta t - \Delta t_{\text{ideal}}) / \tau_{1/2}}$$

where:

- $\text{target}_{\text{ref}}$ is the reference target from the anchor block,
- Δt is the elapsed time since the anchor block,
- Δt_{ideal} is the ideal elapsed time based on block count and target spacing,
- $\tau_{1/2}$ is the half-life parameter.

Table 6. ASERT Difficulty Adjustment Parameters.

Parameter	Value	Description
Target block spacing	8640 s (2.4 h)	Average time between blocks
Half-life ($\tau_{1/2}$)	172800 s (48 h)	Difficulty doubles/halves over this period

Timestamp rules. Block timestamps must be strictly increasing (each block's timestamp must exceed the previous block's timestamp). Timestamps must not exceed the current time plus 600 seconds (10 minutes).

6.1.6 Mining

6.1.6.1 Block Mining

Mining follows the standard PoW longest-chain paradigm. A miner:

1. Selects the current chain tip as the parent block.
2. Prepares the payload commitments (Sec. 6.1.4.2):
 - 2.1 Samples a 32-byte salt $\text{salt}_{\text{reward}} \xleftarrow{\$} \{0, 1\}^{256}$ (min-entropy ≥ 128 bits) and stores it as a secret. The salt is a commit-reveal preimage and MUST remain unpredictable to all other parties until revealed at minting time (Sec. 6.1.6.3); deterministic derivations from publicly observable block data are NOT acceptable. The derived reward token identifier is $\text{tokenId}_{\text{reward}} = H(\text{salt}_{\text{reward}}, \alpha)$, globally unique across networks under collision-resistance of H .
 - 2.2 Obtains the new Unicity Trust Base entry T_e if one is published, validates it against the authentic Unicity Trust Base (Sec. 6.2); otherwise uses the CBOR nil value.
 - 2.3 Computes $\text{payloadRoot} = H(H(\text{tokenId}_{\text{reward}}) \parallel H(\text{cbor}(T_e)))$.
3. Assembles the header with current timestamp, difficulty target from ASERT, payloadRoot , and $\text{hashPrevBlock} = H(\text{parent})$.
4. Iterates $n\text{Nonce}$ values. For each candidate, computes the block hash $h_B = H(n\text{Version} \parallel \text{hashPrevBlock} \parallel \text{payloadRoot} \parallel n\text{Time} \parallel n\text{Bits} \parallel n\text{Nonce})$ and the RandomX commitment $\text{hashRandomX} = \text{RANDOMX}(h_B)$, until hashRandomX meets the difficulty target $n\text{Bits}$.
5. Broadcasts the complete block (112-byte header with final hashRandomX , followed by $v\text{Payload}$) to the P2P network.

6.1.6.2 Block Validation

The steps to validate a block are documented by Algorithm 17.

6.1.6.3 Block Reward Token Minting

The miner who produces a winning block is entitled to a block reward in ALPHA currency. The reward value depends on the current *reward rate*, a protocol-defined schedule indexed by block height. The reward is *not immediately claimable*: the miner must wait for the Finality Gadget (Sec. 6.3) to certify the block containing the reward claim (or a descendant thereof).

The reward is a Unicity token minted through the standard minting process (Sec. 2.1.3.1), with the parameters specified in Table 7.

Verification of a reward token by a recipient requires checking the mint reason and auxiliary payload:

6.1.7 Network Parameters

Identifiers and hard limits are provided in Table 9.

6.2 Snapshot Publishing

Snapshot publishing is the process by which BFT Core epoch transitions are anchored into the PoW chain. This provides the PoW chain community with a verifiable link from the PoW chain to the BFT Core state.

Algorithm 17 PoW Block Validation

function VALIDATEPOWBLOCK($B, \mathcal{T}_{\text{local}}$)

```
ensure( $B.hashRandomX \leq TARGETFROMBITS(B.nBits)$ )
```

$$h_B \leftarrow H(B.nVersion \| B.hashPrevBlock \| B.payloadRoot \| B.nTime \| B.nBits \| B.nNonce)$$
$$\text{ensure}(\text{RANDOMX}(h_B) = B.\text{hashRandomX})$$
$$(\ell_0, \text{cbor_Te}) \leftarrow \text{CBORDECODE}(B.v\text{Payload})$$
$$\ell_1 \leftarrow H(\text{cbor_Te})$$

ensure($B.\text{payloadRoot} = H(\ell_0 || \ell_1)$) ▷ Payload integrity

- ▶ Correct difficulty

ensure($B.nTime > B.parent.nTime$) ▷ Strictly increasing

```
ensure(B.nTime < Now + 600)  ▷ Not too far in the future
```

```
ensure( $B.nVersion \geq 1$ )
```

if cbor Te $\neq$ CBORNIL then

$$T_e \leftarrow \text{CBORDECODE}(\text{cbor } T_e)$$
$$T_{e-1} \leftarrow \mathcal{T}_{\text{local}}[T_e.e - 1] \quad \triangleright \text{Last locally verified entry}$$

ensure($T_e.\alpha = T_{e-1}.\alpha = \mathcal{T}_{\text{local}}.\alpha$) ▶ Checkpoint is for the same network

ensure($T_e.e = T_{e-1}.e + 1$) ▷ Sequential epoch number

ensure($T_e.h_{e-1} = H(T_{e-1})$) ▷ Hash chain linkage

ensure(VERIFYSIG($T_{e.s_{e-1}}$, T_e , $T_{e-1}.\mathcal{V}$)) $\triangleright$ Signed by previous validators

d if

return

and function

One function

Table 7. Block Reward Token.

Field	Value
Network identifier (α)	The Unicity network instance in which the reward token can be minted
Owner predicate (ν')	Miner's chosen spending predicate
Token identifier salt (salt)	The 32-byte reward token salt $\text{salt}_{\text{reward}}$ that yields the reward token identifier $\text{tokenId}_{\text{reward}} = H(\text{salt}_{\text{reward}}, \alpha)$ whose hash $\ell_0 = H(\text{tokenId}_{\text{reward}})$ was committed via payloadRoot of the mined block.
Token type (ty)	ALPHA
Mint reason (reason)	Composite proof structure defined in Table 8
Auxiliary payload (aux')	Initial fungible payload containing value $\text{REWARDRATE}(h)$ for ALPHA at block height h

Table 8. Mint Reason for Block Reward Token.

No	Field	Type	Description
1.	Mined block	B_h	The PoW block at height h , proving the reward claim is committed in the block
2.	Continuation blocks	$\{B_i\}_{i=h+1}^{h'}$	Sequence of PoW blocks linking the mined block to the Finality Gadget certified block at height h'
3.	Finality certificate	cert_{FG}	Unicity Certificate from the Finality Gadget Partition certifying the PoW block at height h'

Algorithm 18 Block Reward Token Minting Process

function MINTREWARDTOKEN($\alpha, B_h, \text{sk}_{\text{miner}}, \mathcal{T}$)

▷ Precondition: B_h is a valid PoW block mined by this miner

▷ Step 1: Wait for Finality Gadget certification

$(h', \text{cert}_{\text{FG}}) \leftarrow \text{WAITFORFGCERT}(h)$ ▷ Block at $h' \geq h$ certified by FG

▷ Step 2: Collect continuation blocks

blocks $\leftarrow \{B_i\}_{i=h+1}^{h'}$

▷ Step 3: Construct mint reason and auxiliary payload

reason $\leftarrow (B_h, \text{blocks}, \text{cert}_{\text{FG}})$

aux' $\leftarrow \{(\text{ALPHA}, \text{REWARDRATE}(h))\}$

▷ Step 4: Construct mint transaction (Sec. 2.1.3.1)

salt $\leftarrow \text{salt}_{\text{reward}}$ ▷ such that $\ell_0 = H(H(\text{salt}, \alpha))$ was committed via $B_h.\text{payloadRoot}$

$v' \leftarrow \text{OWNERPREDICATE}(\text{sk}_{\text{miner}})$

$D_{\text{mint}} \leftarrow (\alpha, v', \text{salt}, \text{ALPHA}, \text{reason}, \text{aux}')$

▷ Step 5: Submit to Unicity Service

$h_{\text{st}} \leftarrow H(H(\text{salt}, \alpha), \text{MINT_SUFFIX})$ ▷ id = $H(\text{salt}, \alpha)$, then mint source state hash

$h_{\text{tx}} \leftarrow H(D_{\text{mint}})$

$u_0 \leftarrow \text{Sign}(\text{sk}_{\text{mint}}, H(h_{\text{st}}, h_{\text{tx}}))$ ▷ Known constants, network specific

$Q \leftarrow (\text{pk}_{\text{mint}}, h_{\text{st}}, h_{\text{tx}}, u_0)$

SUBMITTOUNICITYSERVICE(Q)

end function

Algorithm 19 Block Reward Mint Reason Verification

```
function VERIFYREWARDMINT( $D_{\text{mint}}, \mathcal{T}$ )
  ( $B_h$ , blocks, certFG)  $\leftarrow D_{\text{mint}}.\text{reason}$ 

   $\triangleright$  1. Verify the mined block is valid and contains the reward claim
  ensure(VALIDATEPoWBLOCK( $B_h, \mathcal{T}$ ))
  ( $\ell_0, \_$ )  $\leftarrow$  CBORDECODE( $B_h.v\text{Payload}$ )
  tokenIdreward  $\leftarrow H(D_{\text{mint}}.\text{salt}, D_{\text{mint}}.\alpha)$   $\triangleright$  Reward token identifier
  ensure( $\ell_0 = H(\text{tokenId}_{\text{reward}})$ )  $\triangleright$  Token identifier matches the reward commitment

   $\triangleright$  2. Verify chain of blocks from mined block to certified block
  prev  $\leftarrow H(B_h)$   $\triangleright H$  over fields 1–6 of  $B_h$ 
  for all  $B_i \in \text{blocks}$  do
    ensure( $B_i.\text{hashPrevBlock} = \text{prev}$ )
    ensure( $B_i.\text{hashRandomX} \leq \text{TARGETFROMBITS}(B_i.\text{nBits})$ )  $\triangleright$  Valid PoW
    prev  $\leftarrow H(B_i)$   $\triangleright H$  over fields 1–6 of  $B_i$ 
  end for

   $\triangleright$  3. Verify Finality Gadget certificate covers the chain tip
  ensure(VERIFYUNICITYCERT(certFG,  $\mathcal{T}$ ))
  ensure(certFG. $C^r.\alpha = \mathcal{T}.\alpha$ )
  ensure(certFG. $\beta = \beta_{\text{FP}}$ )  $\triangleright$  FG partition
  ensure(certFG certifies prev)  $\triangleright$  Certified block matches chain tip

   $\triangleright$  4. Verify reward value is correct for the block height
  ensure( $D_{\text{mint}}.\alpha = \mathcal{T}.\alpha$ )
  ensure( $D_{\text{mint}}.\text{ty} = \text{ALPHA}$ )
  ensure(VALUEOF( $D_{\text{mint}}.\text{aux}'$ , ALPHA) = REWARDRATE( $B_h.\text{height}$ ))
  return 1
end function
```

Table 9. PoW Chain Network Parameters.

Parameter	Value	Description
Protocol version	1	Wire protocol version
Mainnet magic	0x554E4943	Network identifier ("UNIC")
Mainnet port	9590	Default P2P port
Max message size	8 MB	Wire protocol limit
Max blocks per message	80000	Block sync batch size

6.2.1 Data Flow

1. The BFT Core completes an epoch change: the leader proposes a new epoch, validators approve the proposal and collectively sign a new Unicity Trust Base entry T_e defining the next set of BFT validators (Sec. 5.4.6.6, Table 3).
2. A PoW miner obtains the new entry T_e via the GetTrustBase protocol (Sec. 5.2.5.8), provided by BFT Core validators and aggregation layer nodes as a REST API.
3. The miner validates the new entry by verifying the Trust Base chain:
 - 3.1 Hash chain linkage: $T_e.h_{e-1} = H(T_{e-1})$.
 - 3.2 Epoch number progression: $T_e.e = T_{e-1}.e + 1$.
 - 3.3 Signature validity: $T_e.s_{e-1}$ is a valid signature from the epoch $e - 1$ validator set over T_e .
4. The miner CBOR-encodes T_e as $\text{cbor}(T_e)$ and includes it in vPayload alongside the reward commitment ℓ_0 . The commitment $\ell_1 = H(\text{cbor}(T_e))$ is folded into payloadRoot (Sec. 6.1.4.2).
5. If this block enters the longest chain, the checkpoint is published.
6. Other PoW nodes and interested verifiers replicate the block. Upon receiving a block, they CBOR-decode vPayload to extract $\text{cbor}(T_e)$, verify it against the locally maintained Trust Base chain (Layer 5 of Alg. 17), and if valid, extend their local Trust Base.

6.2.2 Snapshot Generation

Since BFT Core epochs change infrequently relative to the PoW block rate, most PoW blocks will carry the CBOR nil value in the checkpoint field of vPayload . When multiple unclaimed Trust Base entries exist, the miner publishes the latest one; the intermediate entries can be obtained and verified by any party through the GetTrustBase protocol.

6.3 Finality Gadget

The Finality Gadget is an independent process that provides deterministic finality to PoW blocks by leveraging the BFT Core's consensus. Its purposes are:

1. Providing faster deterministic finality to PoW blocks than the probabilistic finality of the longest chain rule.
2. Enabling reward token verification against the Unicity Trust Base as the root of trust, without requiring the verifier to maintain a full PoW chain.
3. Bounding the practical depth of chain reorganizations.

6.3.1 Finality Gadget Partition

The Finality Gadget operates as a dedicated partition under the BFT Core, the **Finality Gadget Partition** (FP), with partition identifier β_{FP} .

Assumptions:

- Every BFT Core validator (as an independent, self-governing, financially independent entity) also runs a node in the FP.

Algorithm 20 Snapshot Generation by a PoW Miner

State:

$\mathcal{T}_{\text{local}}$: locally verified Trust Base entries

e_{latest} : epoch number of the latest verified entry

function PREPARECHECKPOINT

▷ Query for new Trust Base entries
▷ Via REST API from BFT Core / aggregation nodes

$\mathcal{T}_{\text{new}} \leftarrow \text{GETTRUSTBASE}(\mathcal{T}_{\text{local}}.\alpha, e_{\text{latest}} + 1)$

if $\mathcal{T}_{\text{new}} = \text{NULL}$ **then**
 return CBORNIL ▷ No new checkpoint; payload leaf $\ell_1 = H(\text{cbor}(\perp))$
end if

▷ Validate each new entry against the local Trust Base

for all $T_e \in \mathcal{T}_{\text{new}}$ **do**
 ensure($T_e.\alpha = \mathcal{T}_{\text{local}}.\alpha$) ▷ Trust Base entry belongs to this network
 ensure($T_e.e = e_{\text{latest}} + 1$) ▷ Sequential epochs
 ensure($T_e.h_{e-1} = H(\mathcal{T}_{\text{local}}[e_{\text{latest}}])$) ▷ Hash chain
 ensure($\text{VERIFYSIG}(T_e.s_{e-1}, T_e, \mathcal{T}_{\text{local}}[e_{\text{latest}}].\mathcal{V})$) ▷ Signed by prev. validators
 $\mathcal{T}_{\text{local}}[T_e.e] \leftarrow T_e$
 $e_{\text{latest}} \leftarrow T_e.e$
end for

return CBOR_ENCODE($T_{e_{\text{latest}}}$) ▷ CBOR bytes for vPayload field 2
end function

- Each such entity also operates a PoW node (without mining) to replicate, validate, and maintain a locally verified copy of the PoW chain.
- The FP operates as a standard partition with a rotating partition leader, using the BFT Core consensus mechanisms described in Sec. 5.2.

6.3.2 Operation

The Finality Gadget partition operates in standard shard rounds:

1. The FP leader queries its local PoW node (via RPC) for the latest “final enough” PoW block. Since the leader operates its own PoW node, it can trust the local RPC interface.
2. A PoW block is considered *final enough* by the standard longest-chain criterion: it has sufficient cumulative work built on top and no visible competing forks. The required confirmation depth is a configuration parameter d_{FG} .
3. The leader includes the candidate PoW block in the FP block proposal and broadcasts it to FP follower nodes.
4. Each follower independently verifies the candidate against its own local PoW chain: the block must exist in their active chain with at least d_{FG} confirmations.
5. Followers vote for or against the proposal following the standard BFT partition protocol (Sec. 5.2.3.5).

6. The BFT Core validates the majority approval and returns a Unicity Certificate for the FP round.
7. The certified PoW block and its Unicity Certificate are stored as part of the FP partition blockchain.

Algorithm 21 Finality Gadget Round – Leader

State:

h_{FG} : height of the latest FG-certified PoW block

d_{FG} : required confirmation depth

function FGLEADERPROPOSE

$B_{tip} \leftarrow \text{GETPoWCHAINTip}$

▷ From local PoW node via RPC

$B_{cand} \leftarrow B_{tip}.\text{ancestor}(d_{FG})$

▷ Block at depth d_{FG} from tip

if $B_{cand}.\text{height} \leq h_{FG}$ **then**

return

▷ No new block to certify

end if

 ensure(no competing fork visible at $B_{cand}.\text{height}$)

▷ Propose via standard partition protocol

 SEND_BLOCKPROPOSALMSG($\alpha, \beta_{FP}, \sigma, \nu, luc, lte, \{B_{cand}\}$)

end function

Algorithm 22 Finality Gadget Round – Follower Verification

function FGFOLLOWERVERIFY(B_{cand})

▷ Look up the candidate block in local PoW chain

$B_{local} \leftarrow \text{GETBLOCKBYHASH}(H(B_{cand}))$

▷ From local PoW node

if $B_{local} = \text{NULL}$ **then**

return 0

▷ Block unknown, vote against

end if

 ensure(B_{local} is in the active chain)

▷ Not on a fork

$d \leftarrow \text{GETPoWCHAINTip}.\text{height} - B_{local}.\text{height}$

 ensure($d \geq d_{FG}$)

▷ Sufficient confirmations

return 1

▷ Vote for the proposal

end function

6.3.3 Obtaining Finalized PoW Blocks

The FP partition blockchain is the authoritative source of deterministically finalized PoW blocks. Parties requiring guaranteed PoW finality obtain certified blocks from this chain:

- **Miners** collecting Finality Gadget certificates for reward token minting (Sec. 6.1.6.3).
- **Maximalist verifiers** anchoring their trust in the PoW chain with BFT-backed finality.
- **Light clients** obtaining PoW chain checkpoints without running a full PoW node.

FP blocks are replicated using the standard ledger replication protocol (Sec. 5.2.3.6). Each FP block contains a certified PoW block; verification uses the standard VERIFYUNICITYCERT($\cdot, \mathcal{T}$) function.

6.4 Trust Model

The combination of PoW chain, BFT Core, Snapshot Publishing, and Finality Gadget enables two distinct modes of token verification, offering different trade-offs between trust assumptions and operational convenience.

6.4.1 Maximalist Token Verification

In the maximalist model, the root of trust is a *living copy* of the PoW chain. The user maintains a full PoW node, verifying the chain from genesis. This provides security guarantees analogous to those of Bitcoin: the verifier trusts nothing beyond the PoW chain and cryptographic proofs.

Upon receiving a token, the maximalist verifier checks:

1. **Token validity:** The complete token transaction history is valid (Sec. 2.1.3).
2. **Aggregation Layer integrity:** The Aggregation Layer has not forked—verified by checking that the Unicity Trust Base entries published as PoW chain checkpoints are consistent.
3. **No conflicting certifications:** The Aggregation Layer has not certified conflicting states for the same token—verified through non-deletion proofs, made efficient by recursive zero-knowledge proof compression.

Verification is not immediate. The verifier must wait until:

1. A BFT Core epoch covering the token's latest Unicity Certificate has completed, and its Trust Base entry has been published in a PoW block via Snapshot Publishing.
2. The PoW block containing the relevant checkpoint has accumulated sufficient confirmations (d_{maxi}) to be considered final under the longest chain rule.

The procedure is summarized in Algorithm 23. It is a wrapper around the standard token verification algorithm of Sec. 2.1.4: it derives the Unicity Trust Base $\mathcal{T}_{\text{PoW}}$ from the live PoW chain, then calls `VerifyToken`. The PoW validator may be operated by the verifier locally or hosted by a trusted entity; in either case it is accessed over the standard PoW node RPC interface.

6.4.2 Practical Token Verification

In the practical model, the root of trust is the verified Unicity Trust Base $\mathcal{T}$. The trust assumption is relaxed: a majority of BFT Core validators behave honestly and do not collude with the Aggregation Layer. Under this assumption, BFT layer forking or certifying conflicting states would produce strong cryptographic evidence (equivocation proofs, Sec. 5.2.3.4), which is detected and processed out of the critical path of serving users.

The Unicity Trust Base is compact: it grows by one entry per BFT Core epoch (a few aggregate signatures per week). A verifier initializes by obtaining and verifying $\mathcal{T}$; subsequent updates are incremental.

Token verification is fast: a transaction is finalized and an inclusion proof returned within a few seconds of submission to the Unicity Service. The recipient runs the standard `VerifyToken` algorithm of Sec. 2.1.4 directly against the locally maintained $\mathcal{T}$, without consulting external data beyond the previously obtained Trust Base.

Algorithm 23 Maximalist Token Verification

State:

$\mathcal{N}_{\text{PoW}}$: handle to a live, fully synced PoW validator

d_{maxi} : required confirmation depth

function VERIFYTOKENMAXI($\mathcal{L}$)

▷ Input: token $\mathcal{L} = (T_0^{\text{cert}}, T_1^{\text{cert}}, \dots, T_n^{\text{cert}})$

▷ Step 1: Identify the latest UC referenced by the token

$\pi_n \leftarrow$ inclusion proof component of T_n^{cert}

▷ Last transaction in the chain

$UC^* \leftarrow \pi_n.UC$

$\alpha \leftarrow UC^*.C^r.\alpha$

▷ Network identifier

$e^* \leftarrow UC^*.C^r.e_r$

▷ BFT Core epoch in which UC^* was issued

▷ Step 2: Wait for epoch e^* closure to be checkpointed and PoW-finalized

$h^* \leftarrow \perp$; $h_{\text{tip}} \leftarrow 0$

while $h^* = \perp$ **or** $h_{\text{tip}} - h^* < d_{\text{maxi}}$ **do**

 WAITFORNEWBLOCKS($\mathcal{N}_{\text{PoW}}$)

$h_{\text{tip}} \leftarrow \mathcal{N}_{\text{PoW}}.\text{GETCHAIN TIP}.\text{height}$

 ▷ Lowest PoW height carrying a T_e with $e \geq e^* + 1$, or $\perp$ if none:

$h^* \leftarrow \mathcal{N}_{\text{PoW}}.\text{HEIGHTOFFIRSTCHECKPOINT}(e \geq e^* + 1)$

end while

▷ Step 3: Pull the locally verified Trust Base from the PoW node

$\mathcal{T}_{\text{PoW}} \leftarrow \mathcal{N}_{\text{PoW}}.\text{GETVERIFIEDTRUSTBASE}(\alpha)$ ▷ Entries reconstructed from PoW snapshots (Sec. 6.2)

ensure($\mathcal{T}_{\text{PoW}}.\alpha = \alpha$)

ensure($e^* \leq \max\{e : T_e \in \mathcal{T}_{\text{PoW}}\} - 1$)

▷ Token's epoch is closed in $\mathcal{T}_{\text{PoW}}$

▷ Step 4: Standard token verification under the PoW-derived trust base

ensure(VERIFYTOKEN($\mathcal{L}, \mathcal{T}_{\text{PoW}}$))

▷ Algorithm of Sec. 2.1.4

▷ Step 5: Validate aggregator node's trustless operation based on consistency proofs

ensure(VERIFYCONSISTENCY($\mathcal{L}, \mathcal{N}_{\text{PoW}}, \mathcal{T}_{\text{PoW}}$))

return 1

end function

6.4.3 Comparison

Table 10. Trust Model Comparison.

	Maximalist	Practical
Root of trust	PoW chain	Unicity Trust Base $\mathcal{T}$
Trust assumption	Own PoW node validates everything	Honest BFT validator majority
Verification latency	Hours to days (checkpoint publishing and PoW confirmations)	Seconds (BFT finality in ≈ 2 rounds)
Bootstrapping cost	Full PoW chain sync from genesis	Obtain $\mathcal{T}$ (compact, grows slowly)
Offline verification	Yes, after blockchain sync	Yes, after obtaining $\mathcal{T}$
Security guarantee	Bitcoin-level PoW security	BFT consensus security (honest majority)

Both models verify the same token transaction history and Unicity Certificates. The difference lies in how the Unicity Trust Base is authenticated: the maximalist model derives it from the PoW chain (via Snapshot Publishing), while the practical model obtains it directly from the BFT Core. The Finality Gadget bridges the two by allowing PoW chain artifacts (block rewards) to be verified under the practical model.

A Bitstrings, Orderings, and Codes

A.1 Bitstrings and Orderings

By the *topological ordering* $<$ of $\{0, 1\}^*$ we mean the irreflexive total ordering defined as follows: $c||0||x < c < c||1||y$ for all c, x, y in $\{0, 1\}^*$.

For example, the subset $\{0, 1\}^{\leq 2}$ is ordered as follows: $\{00 < 0 < 01 < \sqcup < 10 < 1 < 11\}$.

A.2 Prefix-Free Codes

A *code* is a finite subset $\mathcal{C}$ of $\{0, 1\}^*$. A code $\mathcal{C}$ is *prefix-free*, if no codeword $c \in \mathcal{C}$ is an initial segment of another codeword $c' \in \mathcal{C}$, i.e. if $c \in \mathcal{C}$, then $c||c'' \notin \mathcal{C}$ for every bitstring $c'' \in \{0, 1\}^*$.

For every code $\mathcal{C}$, the *closure* of $\mathcal{C}$ is the code $\overline{\mathcal{C}} = \{c \in \{0, 1\}^* : \exists c'' : c||c'' \in \mathcal{C}\}$, i.e. $\overline{\mathcal{C}}$ consists of all possible prefixes (including $\sqcup$) of the codewords of $\mathcal{C}$.

A prefix-free code $\mathcal{C}$ is *irreducible*, if its closure $\overline{\mathcal{C}}$ satisfies the following property. For every $c \in \overline{\mathcal{C}}$, either $c \in \mathcal{C}$ or both $c||0, c||1 \in \overline{\mathcal{C}}$.

$$\begin{aligned}\mathcal{C} &= \{0, 10, 11\} \\ \overline{\mathcal{C}} &= \{\sqcup, 0, 1, 10, 11\}\end{aligned}$$

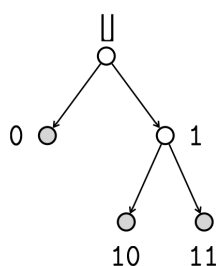


Figure 16. Irreducible prefix free code $\mathcal{C}$ and its closure $\overline{\mathcal{C}}$.

B Encodings

B.1 CBOR

Most data structures on the Unicity platform are serialized in the Concise Binary Object Representation (CBOR), as defined in the IETF RFC 8949⁹.

For data to be hashed or signed, the deterministic encoding rules (as defined in Sec. 4.2 of the RFC 8949) must be used.

B.2 Tuples

Tuples are encoded as CBOR arrays in the natural way: a tuple T consisting of n elements, denoted $(T_1, T_2, \dots, T_n)$ or $\langle T_1, T_2, \dots, T_n \rangle$ in the specification, is encoded as n -element CBOR array with T_1 the first element, T_2 the second element, and so on.

An optional element with no value, denoted $\perp$ in the specification, is not omitted from the array, but encoded as the CBOR simple value `null`.

B.3 Bit-strings

Bit-strings are not directly supported in CBOR. On the Unicity platform, a bit-string is encoded as follows:

1. First, one 1-bit followed by zero to seven 0-bits are appended to the original bit-string so that the total number of bits is a multiple of 8.
2. Then, the padded result is encoded as a CBOR byte-string in the left-to-right, highest-to-lowest order.

For example, the 12-bit string '010110101111' is first padded to the 16-bit string '010110101111000', which is then encoded as the 3-byte sequence 0x425af8:

0x42 byte-string, length 2
0x5a the bits '01011010'
0xf8 the bits '1111' and the padding '1000'

B.4 Hash Values

Hash values, although frequently defined as general bit-strings in cryptographic theory, in practice always have bit-lengths divisible by 8 and are encoded as simple CBOR byte-strings with no padding.

⁹<https://www.rfc-editor.org/rfc/rfc8949>

B.5 Unit Identifiers

Unit identifiers are encoded as CBOR byte-strings.

B.6 Time

Time is expressed as the number of milliseconds since 1970-01-01 00:00:00 UTC, encoded as an unsigned integer: the time value for 1970-01-01 00:00:00.000 UTC is 0, the value for 1970-01-01 00:00:01.000 UTC is 1000, etc. All days are considered to be exactly 86 400 seconds long, ignoring any leap seconds that have occurred in the past.

During a leap second, the part corresponding to full seconds (up from the fourth digit in decimal notation) is kept the same as during the previous second, but the part corresponding to fractions of a second (the three lowest digits in decimal notation) is reset to zero and counted up from there again. In other words, the value recorded for a time within a leap second is the same as the value recorded for the time exactly one second earlier.

B.7 Identifiers

Identifiers of Nodes (validators) are expressed as hashes of compressed ECDSA public keys; respective private key is controlled by the Node.

Identifiers of Transaction Systems are expressed as integers.

B.8 Cryptographic Algorithms

“ECDSA” denotes the Elliptic Curve Digital Signature Algorithm using secp256k1 curve

“SHA-256” denotes the SHA-2 hash algorithm with 256-bit output and “SHA-512” denotes the SHA-2 hash algorithm with 512-bit output; both are specified by NIST FIPS 180-4.

The list is non-exhaustive.

C Hash Trees

C.1 Plain Hash Trees

C.1.1 Function PLAIN_TREE_ROOT

Computes the root value of the plain hash tree with the given n values in its leaves.

Input: $L = \langle x_1, \dots, x_n \rangle \in \mathbb{H}^n$, the list of the values in the n leaves of the tree

Output: $r \in \mathbb{H} \cup \{\perp\}$, the value in the root of the tree

Computation:

```
function PLAIN_TREE_ROOT( $L$ )  
  if  $n = 0$  then                                      $\triangleright L = \langle \rangle$   
    return  $\perp$   
  else if  $n = 1$  then                                  $\triangleright L = \langle x_1 \rangle$   
    return  $x_1$   
  else  
     $m \leftarrow 2^{\lfloor \log_2(n-1) \rfloor}$                       $\triangleright$  Canonical tree  
     $L_{\text{left}} \leftarrow \langle x_1, \dots, x_m \rangle$   
     $L_{\text{right}} \leftarrow \langle x_{m+1}, \dots, x_n \rangle$   
    return  $H(\text{PLAIN\_TREE\_ROOT}(L_{\text{left}}), \text{PLAIN\_TREE\_ROOT}(L_{\text{right}}))$   
  end if  
end function
```

Note that $2^{\lfloor \log_2(n-1) \rfloor}$ is the value of the highest 1-bit in the binary representation of $n - 1$, which may be the preferred way to compute m in some environments. Splitting the leaves this way results in a structure that allows the root of the tree to be computed incrementally, without having all the leaves in memory at once.

C.1.2 Function PLAIN_TREE_CHAIN

Computes the hash chain from the i -th leaf to the root of the plain hash tree with the given n values in its leaves.

Input:

1. $L = \langle x_1, \dots, x_n \rangle \in \mathbb{H}^n$, the list of the values in the n leaves of the tree
2. $i \in \{1, \dots, n\}$, the index of the starting leaf of the chain

Output: $C = \langle (b_1, y_1), \dots, (b_\ell, y_\ell) \rangle \in (\mathbb{B} \times \mathbb{H})^\ell$, where y_j are the sibling hash values on the path from the i -th leaf to the root and b_j indicate whether the corresponding y_j is the right-

or left-hand sibling

Computation:

```

function PLAIN_TREE_CHAIN( $L; i$ )
  assert  $1 \leq i \leq n$ 
  if  $n = 1$  then                                      $\triangleright L = \langle x_1 \rangle$ 
    return  $\langle \rangle$ 
  else
     $m \leftarrow 2^{\lfloor \log_2(n-1) \rfloor}$                       $\triangleright$  Must match PLAIN_TREE_ROOT
     $L_{\text{left}} \leftarrow \langle x_1, \dots, x_m \rangle$ 
     $L_{\text{right}} \leftarrow \langle x_{m+1}, \dots, x_n \rangle$ 
    if  $i \leq m$  then
      return PLAIN_TREE_CHAIN( $L_{\text{left}}; i$ ) || (0, PLAIN_TREE_ROOT( $L_{\text{right}}$ ))
    else
      return PLAIN_TREE_CHAIN( $L_{\text{right}}; i - m$ ) || (1, PLAIN_TREE_ROOT( $L_{\text{left}}$ ))
    end if
  end if
end function

```

C.1.3 Function PLAIN_TREE_OUTPUT

Computes the output hash of the chain C on the input x .

Input:

1. $C = \langle (b_1, y_1), \dots, (b_\ell, y_\ell) \rangle \in (\mathbb{B} \times \mathbb{H})^\ell$, where y_j are the sibling hash values on the path from the i -th leaf to the root and b_j indicate whether the corresponding y_j is the right- or left-hand sibling
2. $x \in \mathbb{H}$, the input hash value

Output: $r \in \mathbb{H}$, output value of the hash chain

Computation:

```

function PLAIN_TREE_OUTPUT( $C; x$ )
  if  $\ell = 0$  then                                      $\triangleright C = \langle \rangle$ 
    return  $x$ 
  else
    assert  $b_\ell \in \mathbb{B}$ 
    if  $b_\ell = 0$  then
      return  $H(\text{PLAIN\_TREE\_OUTPUT}(\langle (b_1, y_1), \dots, (b_{\ell-1}, y_{\ell-1}) \rangle; x), y_\ell)$ 
    else
      return  $H(y_\ell, \text{PLAIN\_TREE\_OUTPUT}(\langle (b_1, y_1), \dots, (b_{\ell-1}, y_{\ell-1}) \rangle; x))$ 
    end if
  end if
end function

```

C.1.4 Inclusion Proofs

Plain hash trees can be used to provide and verify inclusion proofs. The process for this is as follows:

- To commit to the contents of a list $L = \langle x_1, \dots, x_n \rangle$:
 - Compute $r \leftarrow \text{PLAIN_TREE_ROOT}(L)$.
 - Authenticate r somehow (sign it, post it to an immutable ledger, etc).
- To generate inclusion proof for $x_i \in L$:
 - Compute $C \leftarrow \text{PLAIN_TREE_CHAIN}(L; i)$.
- To verify the inclusion proof $C = \langle (b_1, y_1), \dots, (b_\ell, y_\ell) \rangle$ for x :
 - Check that $\text{PLAIN_TREE_OUTPUT}(C; x) = r$, where r is the previously authenticated root hash value.

C.2 Indexed Hash Trees

C.2.1 Function INDEX_TREE_ROOT

Computes the root value of the indexed hash tree with the given n key-value pairs in its leaves.

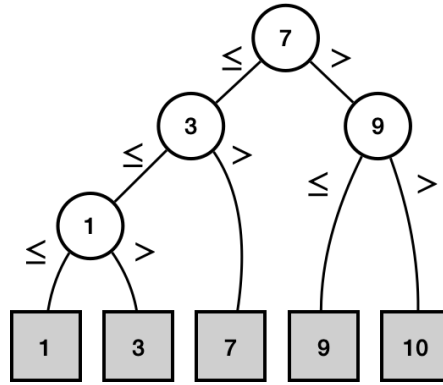


Figure 17. Keys of the nodes of an indexed hash tree.

Input: List $L = \langle (k_1, x_1), \dots, (k_n, x_n) \rangle \in (\mathbb{K} \times \mathbb{H})^n$, the list of the key-value pairs in the n leaves of the tree; $\mathbb{K}$ must be a linearly ordered type and the input pairs must be strictly sorted in this order, i.e. $k_1 < \dots < k_n$

Output: $r \in \mathbb{H} \cup \{\perp\}$, the value in the root of the tree

Computation:

```

function INDEX_TREE_ROOT( $L$ )
  assert  $k_1 < \dots < k_n$ 
  if  $n = 0$  then
    return  $\perp$ 
  else if  $n = 1$  then
    return  $H(1, k_1, x_1)$ 
  else
     $m \leftarrow \lceil n/2 \rceil$ 
     $L_{\text{left}} \leftarrow \langle (k_1, x_1), \dots, (k_m, x_m) \rangle$ 
     $L_{\text{right}} \leftarrow \langle (k_{m+1}, x_{m+1}), \dots, (k_n, x_n) \rangle$ 
    return  $H(0, k_m, \text{INDEX\_TREE\_ROOT}(L_{\text{left}}), \text{INDEX\_TREE\_ROOT}(L_{\text{right}}))$ 
  end if
end function

```

▷ $L = \langle \rangle$

▷ $L = \langle (k_1, x_1) \rangle$

▷ Most balanced tree

C.2.2 Function INDEX_TREE_CHAIN

Considers the indexed hash tree with the given n key-value pairs in its leaves. If there is a leaf containing the key k , computes the hash chain from that leaf to the root. If there is no such leaf, computes the hash chain from the leaf where k should be according to the ordering, which can be used as a proof of k 's absence.

Input:

1. $L = \langle (k_1, x_1), \dots, (k_n, x_n) \rangle \in (\mathbb{K} \times \mathbb{H})^n$, the list of the key-value pairs in the n leaves of the tree; $\mathbb{K}$ must be a linearly ordered type and the input pairs must be strictly sorted in this order, i.e. $k_1 < \dots < k_n$
2. $k \in \mathbb{K}$, the key to compute the path for

Output: $C = \langle (k_1, y_1), \dots, (k_\ell, y_\ell) \rangle \in (\mathbb{K} \times \mathbb{H})^\ell$, where k_j are the keys in the nodes on the path from the leaf to the root and y_j are the sibling hash values

Computation:

```

function INDEX_TREE_CHAIN( $L; k$ )
  assert  $k_1 < \dots < k_n$ 
  if  $n \in \{0, 1\}$  then                                      $\triangleright L = \langle \rangle$  or  $L = \langle (k_1, x_1) \rangle$ 
    return  $L$ 
  else
     $m \leftarrow \lceil n/2 \rceil$                                       $\triangleright$  Must match INDEX_TREE_ROOT
     $L_{\text{left}} \leftarrow \langle (k_1, x_1), \dots, (k_m, x_m) \rangle$ 
     $L_{\text{right}} \leftarrow \langle (k_{m+1}, x_{m+1}), \dots, (k_n, x_n) \rangle$ 
    if  $k \leq k_m$  then
      return INDEX_TREE_CHAIN( $L_{\text{left}}; k$ ) || ( $k_m$ , INDEX_TREE_ROOT( $L_{\text{right}}$ ))
    else
      return INDEX_TREE_CHAIN( $L_{\text{right}}; k$ ) || ( $k_m$ , INDEX_TREE_ROOT( $L_{\text{left}}$ ))
    end if
  end if
end function

```

C.2.3 Function INDEX_TREE_OUTPUT

Computes the output hash of the chain C on the input key k .

Input:

1. $C = \langle (k_1, y_1), \dots, (k_\ell, y_\ell) \rangle \in (\mathbb{K} \times \mathbb{H})^\ell$, where k_j are the keys in the nodes on the path from the leaf to the root and y_j are the sibling hash values
2. $k \in \mathbb{K}$, the input key

Output: $r \in \mathbb{H} \cup \{\perp\}$, the value in the root of the tree

Computation:

```

function INDEX_TREE_OUTPUT( $C; k$ )
  if  $\ell = 0$  then                                            $\triangleright C = \langle \rangle$ 
    return  $\perp$ 
  else if  $\ell = 1$  then                                        $\triangleright C = \langle (k_1, y_1) \rangle$ 

```

```

    return  $H(1, k_1, y_1)$ 
else
    if  $k \leq k_\ell$  then
        return  $H(0, k_\ell, \text{INDEX\_TREE\_OUTPUT}(\langle (k_1, y_1), \dots, (k_{\ell-1}, y_{\ell-1}) \rangle; k), y_\ell)$ 
    else
        return  $H(0, k_\ell, y_\ell, \text{INDEX\_TREE\_OUTPUT}(\langle (k_1, y_1), \dots, (k_{\ell-1}, y_{\ell-1}) \rangle; k))$ 
    end if
end if
end function

```

C.2.4 Inclusion and Exclusion Proofs

Indexed hash trees can be used to provide and verify both inclusion and exclusion proofs. The process for this is as follows:

- To commit to the contents of a list $L = \langle (k_1, x_1), \dots, (k_n, x_n) \rangle$ (where $k_1 < \dots < k_n$):
 - Compute $r \leftarrow \text{INDEX_TREE_ROOT}(L)$.
 - Authenticate r somehow (sign it, post it to an immutable ledger, etc).
- To generate inclusion proof for $(k_i, x_i) \in L$:
 - Compute $C \leftarrow \text{INDEX_TREE_CHAIN}(L; k_i)$.
- To verify the inclusion proof $C = \langle (k_1, y_1), \dots, (k_\ell, y_\ell) \rangle$ for (k, x) :
 - Check that $\text{INDEX_TREE_OUTPUT}(C; k) = r$, where r is the previously authenticated root hash value.
 - Check that $(k, x) = (k_1, y_1)$, where (k_1, y_1) is the first pair in the list C .
- To generate exclusion proof for $k \notin \{k_1, \dots, k_n\}$:
 - Compute $C \leftarrow \text{INDEX_TREE_CHAIN}(L; k)$.
- To verify the exclusion proof $C = \langle (k_1, y_1), \dots, (k_\ell, y_\ell) \rangle$ for k :
 - Check that $\text{INDEX_TREE_OUTPUT}(C; k) = r$, where r is the previously authenticated root hash value.
 - Check that $k \neq k_1$, where (k_1, y_1) is the first pair in the list C .

C.3 Sparse Merkle Sum Trees

A *Sparse Merkle Sum Tree* (SMST) extends Sparse Merkle Tree with value tracking. Each node carries both a hash and a non-negative integer value. Internal nodes sum their children's values, enabling cryptographic proofs that a leaf's value is part of a known total. This structure is used for value conservation proofs in token splitting (Sec. 2.5.3).

C.3.1 Node Structure

Each node in an SMST carries a pair (h, v) , where $h \in \mathbb{H}$ is the node hash and $v \in \mathbb{N}$ is the accumulated value.

- **Leaf node:** (h, v) where:
 - $p \in \{0, 1\}^*$ – path label from this leaf to its parent

- $d \in \text{OCT}^*$ – leaf data
- $v \in \mathbb{N}$ – leaf value (e.g. asset amount)
- $h = H(p, d, v)$
- **Branch node:** (h, v) where:
 - $p \in \{0, 1\}^*$ – path label from this node to its parent
 - (h_L, v_L) – left child hash and value
 - (h_R, v_R) – right child hash and value
 - $h = H(p, h_L, v_L, h_R, v_R)$
 - $v = v_L + v_R$
- **Root node:** Uses the fixed path label $p = 1$ in hash computation:
 - $h = H(1, h_L, v_L, h_R, v_R)$
 - $v = v_L + v_R$

An empty child is represented by $(h, v) = (\perp, 0)$.

C.3.2 Function SMST_LEAF_HASH

Computes the hash of an SMST leaf node.

Input:

1. $p \in \{0, 1\}^*$ – path segment from this leaf to its parent
2. $d \in \mathbb{H}$ – leaf data
3. $v \in \mathbb{N}$ – leaf value

Output: $h \in \mathbb{H}$ – hash of the leaf node

Computation:

```
function SMST_LEAF_HASH( $p, d, v$ )
  return  $H(p, d, v)$ 
end function
```

C.3.3 Function SMST_BRANCH_HASH

Computes the hash and accumulated value of an SMST branch node.

Input:

1. $p \in \{0, 1\}^*$ – path segment from this node to its parent
2. $(h_L, v_L) \in (\mathbb{H} \cup \{\perp\}) \times \mathbb{N}$ – left child
3. $(h_R, v_R) \in (\mathbb{H} \cup \{\perp\}) \times \mathbb{N}$ – right child

Output: $(h, v) \in \mathbb{H} \times \mathbb{N}$ – hash and value of the branch node

Computation:

```
function SMST_BRANCH_HASH( $p, (h_L, v_L), (h_R, v_R)$ )
   $h \leftarrow H(p, h_L, v_L, h_R, v_R)$ 
   $v \leftarrow v_L + v_R$ 
  return  $(h, v)$ 
end function
```

C.3.4 Inclusion Certificate

An *Inclusion Certificate* for an SMST leaf with key k is a sequence of triples:

$$C^{\text{inc}} = \langle (p_1, d_1, v_1), (p_2, d_2, v_2), \dots, (p_n, d_n, v_n) \rangle$$

where:

- (p_1, d_1, v_1) – the leaf's path segment, data, and value
- (p_i, d_i, v_i) for $i > 1$ – path segment p_i to parent, sibling hash d_i , and sibling value v_i

C.3.5 Function SMST_VERIFY_INCLUSION

Verifies an SMST inclusion certificate by reconstructing the root hash and total value.

Input:

1. $k \in \mathbb{H}$ – key
2. $v_{\text{leaf}} \in \mathbb{N}$ – expected leaf value
3. $C^{\text{inc}} = \langle (p_1, d_1, v_1), \dots, (p_n, d_n, v_n) \rangle$ – inclusion certificate
4. $h_{\text{root}} \in \mathbb{H}$ – expected root hash

Output: TRUE OR FALSE

Computation:

```

function SMST_VERIFY_INCLUSION( $k, v_{\text{leaf}}, C^{\text{inc}}, h_{\text{root}}$ )
  if  $v_1 \neq v_{\text{leaf}}$  then
    return FALSE                                ▶ Leaf value mismatch
  end if
   $h \leftarrow H(p_1, d_1, v_1)$                       ▶ Hash leaf node
   $v_{\text{sum}} \leftarrow v_1$ 
   $q \leftarrow p_1$                                 ▶ Reconstructed path
  for  $i \leftarrow 2$  to  $n$  do
     $b \leftarrow$  rightmost bit of  $p_{i-1}$ 
    if  $b = 0$  then
       $h \leftarrow H(p_i, h, v_{\text{sum}}, d_i, v_i)$       ▶ Current is left child
    else
       $h \leftarrow H(p_i, d_i, v_i, h, v_{\text{sum}})$       ▶ Current is right child
    end if
     $v_{\text{sum}} \leftarrow v_{\text{sum}} + v_i$ 
     $q \leftarrow q || p_i$                           ▶ Extend reconstructed path
  end for
  return  $(h = h_{\text{root}}) \wedge (q = k)$ 
end function

```

In addition to verifying the root hash, the accumulated sum v_{sum} at the root equals the total value in the tree, enabling value conservation checks: the verifier confirms both that the leaf value matches the claimed allocation and that the root sum matches the original token's total value for the given asset.

C.4 Radix Sparse Merkle Trees

A *Radix Sparse Merkle Tree* (RSMT) is a leaf-anchored, path-compressed binary Merkle tree over a 256-bit key space. It authenticates a finite map $k \mapsto v$, where every stored key appears as exactly one leaf and every internal node records the absolute bit position at which its descendant keys bifurcate. The implementation documented here uses LSB-first bit order.¹⁰ Each leaf hash commits to the full 256-bit key, and each internal node hash commits to the explicit bifurcation depth. Consequently, splitting an edge above an unchanged subtree does not alter that subtree's hash: batch insertion rehashes only the changed search frontier and the newly created nodes. The tree supports inclusion certificates, non-inclusion certificates and consistency proofs.

C.4.1 Tree Structure

Keys are elements of OCT^{32} (256 bits), addressed in LSB-first order¹¹. Bit position i means the i -th least significant bit.

C.4.1.1 Node Format

Let s denote the next key-bit position examined during a top-down traversal.

- **Empty tree:** The empty tree has root $\perp$.
- **Leaf node:** A leaf stores a pair (k, v) with $k \in \text{OCT}^{32}$ and $v \in \text{OCT}^*$. The full key k anchors the leaf position in the tree.
- **Internal node:** An internal node stores a tuple (q, d, c_0, c_1) , where $q \in \{0, 1\}^{d-s}$ is the maximal common path segment shared by all descendant keys from bit positions $s, \dots, d-1$, $d \in \{0, \dots, 255\}$ is the bifurcation depth, and c_0 and c_1 are the left and right child subtrees. A search key k may descend through the node if and only if its bits $k[s], \dots, k[d-1]$ equal q ; the next bit $k[d]$ then selects child c_0 for 0 and child c_1 for 1.

The path segment q is stored for routing but is not part of the Merkle hash. The commitment is determined entirely by the leaf contents, the child hashes and the absolute bifurcation depth.

C.4.2 Hash Computation

The hash function H is SHA-256. Domain separation is achieved via a one-byte prefix: $0x00$ for leaves and $0x01$ for internal nodes.

C.4.2.1 Function RSMT_LEAF_HASH

Computes the hash of an RSMT leaf node.

Input:

1. $k \in \text{OCT}^{32}$ – key (256-bit identifier)
2. $v \in \text{OCT}^*$ – value

¹⁰Reference Python implementation: `ndrsmt3o.py`.

¹¹See Appendix A for bit-string conventions.

Output: $h \in \mathbb{H}$

Computation:

```
function RSMT_LEAF_HASH( $k, v$ )  
  return  $H(0x00 \parallel k \parallel v)$   
end function
```

C.4.2.2 Function RSMT_NODE_HASH

Computes the hash of an RSMT internal node.

Input:

1. $h_L \in \mathbb{H}$ – left child hash
2. $h_R \in \mathbb{H}$ – right child hash
3. $d \in \mathbb{N}_8$ – bifurcation depth

Output: $h \in \mathbb{H}$

Computation:

```
function RSMT_NODE_HASH( $h_L, h_R, d$ )  
  return  $H(0x01 \parallel d \parallel h_L \parallel h_R)$   
end function
```

The internal-node hash does not include the compressed path segment q . Every leaf hash already commits to the full key, and every internal-node hash commits to the absolute branching depth d .

C.4.3 Batch Insertion

The batch insertion algorithm sorts the input batch once in the traversal order defined in Section C.4.7.1, removes repeated keys within the batch, and then merges the resulting sorted list with the affected frontier of the existing tree. Existing keys are dropped from the batch during this merge.

C.4.3.1 Function RSMT_BATCH_INSERT_NP

In the following, the helper procedure RSMT_MERGE_BATCH recursively merges a contiguous slice of the sorted batch into the current subtree root, and RSMT_BUILD_SUBTREE builds a new radix subtree from a non-empty sorted slice.

Input:

1. T – current RSMT
2. $B = \langle (k_1, v_1), \dots, (k_m, v_m) \rangle$ – unordered batch of candidate insertions

Output: T updated in place, with root replaced by the root of the post-state tree

Computation:

```
function RSMT_BATCH_INSERT_NP( $T, B$ )  
  sort  $B$  by RSMT_SORT_KEY and remove repeated keys to obtain  $B'$   
  if  $B' = \langle \rangle$  then
```

```

    return
end if
 $T.\text{root} \leftarrow \text{RSMT\_MERGE\_BATCH}(T.\text{root}, B'; 0)$ 
end function

```

The helper procedure $\text{RSMT_MERGE_BATCH}(r, S; s)$ receives a current subtree root r , a contiguous slice S of the traversal-sorted batch, and the next bit position s . The following pseudocode omits low-level bit manipulations.

```

function RSMT_MERGE_BATCH( $r, S; s$ )
  if  $S = \langle \rangle$  then
    return  $r$ 
  end if
  if  $r = \perp$  then
    return  $\text{RSMT\_BUILD\_SUBTREE}(S; s)$ 
  end if
  if  $r$  is a leaf ( $k_\ell, v_\ell$ ) then
    let  $U$  be the traversal-order merge of  $S$  with  $(k_\ell, v_\ell)$ , omitting any batch element
    whose key is  $k_\ell$ 
    if  $|U| = 1$  then
      return  $r$ 
    end if
    return  $\text{RSMT\_BUILD\_SUBTREE}(U; s)$ 
  end if
  parse  $r$  as internal node  $(q, d, r_0, r_1)$ 
  if the first or last key of  $S$  diverges from prefix  $q$  before depth  $d$  then
    let  $t < d$  be the first divergent depth
    partition  $S$  into  $S_0$  and  $S_1$  according to bit  $t$ 
    let  $b \in \{0, 1\}$  be the branch of the old subtree at depth  $t$ , as determined by  $q$ 
    if  $b = 0$  then
       $r'_0 \leftarrow \text{RSMT\_MERGE\_BATCH}(r, S_0; t)$ 
       $r'_1 \leftarrow \text{RSMT\_MERGE\_BATCH}(\perp, S_1; t)$ 
    else
       $r'_0 \leftarrow \text{RSMT\_MERGE\_BATCH}(\perp, S_0; t)$ 
       $r'_1 \leftarrow \text{RSMT\_MERGE\_BATCH}(r, S_1; t)$ 
    end if
    return a new parent node at depth  $t$  with children  $r'_0$  and  $r'_1$ 
  end if
  partition  $S$  into  $S_0$  and  $S_1$  according to bit  $d$ 
   $r'_0 \leftarrow \text{RSMT\_MERGE\_BATCH}(r_0, S_0; d)$ 
   $r'_1 \leftarrow \text{RSMT\_MERGE\_BATCH}(r_1, S_1; d)$ 
  return the node  $(q, d, r'_0, r'_1)$ 
end function

```

Because S is already sorted in traversal order, the implementation can locate the first relevant divergence by inspecting only the first and the last key of the slice. Thus each recursive call either descends into the already committed frontier or constructs a fresh subtree for a non-empty batch slice, without extra per-key membership lookups.

The helper procedure $\text{RSMT_BUILD_SUBTREE}(S; s)$ is applied only to a non-empty traversal-

sorted slice:

1. If $|S| = 1$, create a leaf for the single pair in S and return it.
2. Otherwise compute the first bit position $d \geq s$ at which the first and the last key of S differ. This determines the unique bifurcation depth of the subtree root.
3. Partition S into S_0 and S_1 according to bit d , recursively build the two child subtrees, and return the internal node whose compressed path is the common bit string from positions $s, \dots, d - 1$.

Unlike the consistency-proof-producing batch insertion path, this proof-free variant does not return the subset of leaves that were actually inserted. That auxiliary list is only needed for pairing with a consistency proof.

For an unordered batch of size m , the initial sort costs $O(m \log m)$. The recursive merge then touches only the affected search frontier of the existing tree and the newly created nodes, so the total running time remains within the same $O(m \log m)$ bound.

C.4.4 Shard Split

When a shard σ is split into two sub-shards $\sigma||0$ and $\sigma||1$ (Sec. 3.1), the RSMT maintained by the parent shard must be partitioned accordingly: each successor keeps only those leaves whose keys route to it under the sharding function f_{SH} . In the LSB-first convention of Section C.4.7.1, a key k belongs to $\sigma||b$ if bit $|\sigma|$ of k equals b .

C.4.4.1 Function RSMT_SHARD_SPLIT

Retains the sub-shard assigned by the bit b and discards the complement.

Input:

1. T – RSMT of the current shard σ (every leaf key agrees with σ on bits $0, \dots, |\sigma| - 1$)
2. $s \in \{0, \dots, 255\}$ – split depth, equal to $|\sigma|$
3. $b \in \{0, 1\}$ – selector bit of the sub-shard to retain

Output: T' – RSMT with exactly those leaves of T whose bit s equals b

Computation:

```

function RSMT_SHARD_SPLIT( $T, s, b$ )
  if  $T.\text{root} = \perp$  then
    return empty tree
  end if
   $T'.\text{root} \leftarrow \text{RSMT\_SPLIT\_NODE}(T.\text{root}, s, b)$ 
  return  $T'$ 
end function

```

The helper $\text{RSMT_SPLIT_NODE}(r, s, b)$ descends the subtree rooted at r to the first node whose bifurcation depth is at least s :

```

function RSMT_SPLIT_NODE( $r, s, b$ )
  if  $r$  is a leaf  $(k, v)$  then
    if bit  $s$  of  $k$  equals  $b$  then return  $r$ 

```

```

    else
        return  $\perp$ 
    end if
end if
parse  $r$  as internal node  $(q, d, c_0, c_1)$ 
if  $d = s$  then
    return  $c_b$ 
else
     $\triangleright d > s$ : all descendant keys agree on bit  $s$ 
    let  $b^* \in \{0, 1\}$  be bit  $s$  of any descendant key (equivalently, the bit of  $q$  at offset  $s$ )
    if  $b^* = b$  then
        return  $r$ 
    else
        return  $\perp$ 
    end if
end if
end function

```

Properties.

- *No rehashing*: Because internal-node hashes commit to the absolute bifurcation depth d (Sec. C.4.2.2) and leaf hashes commit to the full key (Sec. C.4.2.1), every retained node keeps its pre-split hash. The post-split root hash equals the hash of whichever existing node the procedure returns.
- *Inclusion Proof change*: An inclusion certificate (Sec. C.4.5) that was valid against the pre-split root is now different, the sibling entry at depth s , if any, from both the bitmap and the sibling list is dropped. This is compensated by the Unicity Certificates being larger, due to extra entry in Shard Tree.
- The procedure may be executed as many times as necessary to obtain necessary number of shards.

C.4.5 Inclusion Certificate

An *Inclusion Certificate* C^{inc} demonstrates that a specific (k, v) pair exists in a tree with root hash ρ .

C.4.5.1 Format

An inclusion certificate is a pair $C^{\text{inc}} = (\text{bitmap}, \text{siblings})$ where:

- $\text{bitmap} \in \text{OCT}^{32}$ – a 256-bit vector. Bit d is set if and only if a sibling hash is required at depth d on the path from the leaf to the root.
- $\text{siblings} = \langle s_1, \dots, s_n \rangle \in \mathbb{H}^n$ where $n = \text{bitcount}(\text{bitmap})$ – sibling hashes ordered leaf-to-root (ascending depth).

Wire format: $\text{bitmap}[32] \parallel s_1[32] \parallel \dots \parallel s_n[32]$, for a total of $32 + 32n$ bytes.

C.4.5.2 Generation

To generate an inclusion certificate for key k , walk from the root to the leaf matching k . At each internal node with bifurcation depth d on the path, set bit d in the bitmap and append

the sibling subtree's hash to the siblings list.

C.4.5.3 Function RSMT_VERIFY_INCLUSION

Verifies an inclusion certificate by reconstructing the root hash from the leaf upward.

Input:

1. $C^{\text{inc}} = (\text{bitmap}, \text{siblings})$ – inclusion certificate
2. $\rho \in \mathbb{H}$ – expected (authentic) root hash
3. $k \in \text{OCT}^{32}$ – key
4. $v \in \text{OCT}^*$ – value

Output: TRUE OR FALSE

Computation:

```

function RSMT_VERIFY_INCLUSION( $C^{\text{inc}}, \rho, k, v$ )
   $h \leftarrow \text{RSMT\_LEAF\_HASH}(k, v)$ 
   $j \leftarrow |\text{siblings}|$ 
  for  $d \leftarrow 255$  downto 0 do
    if bit  $d$  of  $\text{bitmap}$  is not set then
      continue
    end if
     $j \leftarrow j - 1$ 
    if  $j < 0$  then return FALSE
    end if
     $s \leftarrow \text{siblings}[j]$ 
    if bit  $d$  of  $k$  is 1 then                                      $\triangleright$  Leaf went right at depth  $d$ 
       $h \leftarrow \text{RSMT\_NODE\_HASH}(s, h, d)$ 
    else
       $h \leftarrow \text{RSMT\_NODE\_HASH}(h, s, d)$ 
    end if
  end for
  return  $(j = 0) \wedge (h = \rho)$ 
end function

```

C.4.6 Non-Inclusion Certificate

A *Non-Inclusion Certificate* C^{exc} demonstrates that a key k does *not* appear in a tree with root hash ρ . In a path-compressed RSMT, searching for a key that is absent terminates at a leaf whose key k_ℓ shares a common prefix with k up to some bifurcation depth but differs from k . The non-inclusion certificate carries the neighbouring leaf's key and hash (but not its value), together with the sibling hashes needed to reconstruct the root.

C.4.6.1 Format

A non-inclusion certificate is a tuple $C^{\text{exc}} = (k_\ell, h_\ell, \text{bitmap}, \text{siblings})$ where:

- $k_\ell \in \text{OCT}^{32}$ – key of the neighbouring leaf
- $h_\ell \in \mathbb{H}$ – hash of the neighbouring leaf ($= \text{RSMT_LEAF_HASH}(k_\ell, v_\ell)$)

- $bitmap \in \text{OCT}^{32}$ – branching-depth bitmap (same semantics as inclusion certificate, Section C.4.5)
- $siblings = \langle s_1, \dots, s_n \rangle \in \mathbb{H}^n$ where $n = \text{bitcount}(bitmap)$ – sibling hashes, leaf-to-root

Wire format: $k_\ell[32] \parallel h_\ell[32] \parallel bitmap[32] \parallel s_1[32] \parallel \dots \parallel s_n[32]$, for a total of $96 + 32n$ bytes.

For the special case of an empty tree ($\rho = \perp$), the non-inclusion certificate is empty—every key is trivially absent.

C.4.6.2 Generation

To generate a non-inclusion certificate for k , walk from the root toward k . The walk terminates at a leaf with key $k_\ell \neq k$ (since k is absent). Record k_ℓ and its leaf hash h_ℓ , then collect sibling hashes and the bitmap exactly as for an inclusion certificate (Section C.4.5).

C.4.6.3 Function RSMT_VERIFY_NON_INCLUSION

Verifies that key k is absent from a tree with root hash ρ .

Input:

1. $C^{\text{exc}} = (k_\ell, h_\ell, bitmap, siblings)$ – non-inclusion certificate
2. $\rho \in \mathbb{H} \cup \{\perp\}$ – root hash ($\perp$ if tree is empty)
3. $k \in \text{OCT}^{32}$ – queried key

Output: TRUE OR FALSE

Computation:

```

function RSMT_VERIFY_NON_INCLUSION( $C^{\text{exc}}, \rho, k$ )
  if  $\rho = \perp$  then
    return TRUE ▷ Empty tree contains no keys
  end if
  if  $k_\ell = k$  then
    return FALSE ▷ Neighbour key must differ from queried key
  end if
   $d^* \leftarrow$  lowest bit position where  $k$  and  $k_\ell$  differ
  if bit  $d^*$  of  $bitmap$  is set then
    return FALSE ▷ A branch at  $d^*$  means  $k$  could exist in the opposite subtree
  end if
   $h \leftarrow h_\ell$  ▷ Start from the neighbouring leaf hash
   $j \leftarrow |siblings|$ 
  for  $d \leftarrow 255$  downto 0 do
    if bit  $d$  of  $bitmap$  is not set then
      continue
    end if
     $j \leftarrow j - 1$ 
    if  $j < 0$  then return FALSE
    end if
     $s \leftarrow siblings[j]$ 
    if bit  $d$  of  $k_\ell$  is 1 then

```

```

         $h \leftarrow \text{RSMT\_NODE\_HASH}(s, h, d)$ 
    else
         $h \leftarrow \text{RSMT\_NODE\_HASH}(h, s, d)$ 
    end if
end for
return  $(j = 0) \wedge (h = \rho)$ 
end function

```

C.4.7 Consistency Proof

A *consistency proof* demonstrates that a batch B of new key-value pairs was correctly inserted into a tree with root ρ' , producing a new tree with root ρ , without modifying or deleting any pre-existing data.

The current implementation proves preservation of committed pre-state subtrees and incorporation of the declared batch into the post-state hash. It does *not* additionally prove that the resulting post-state is the unique canonical radix sparse Merkle tree induced by the final key set. This stronger structural property is omitted for efficiency reasons: enforcing it would require additional witness data, larger inclusion proofs and more expensive verification.

C.4.7.1 Sort Order

The batch and proof use *LSB-first traversal order*. The sort key of a 256-bit key k is obtained by reversing the byte order of k and then independently reversing the bits within each byte. This places the least significant bit of the key first in lexicographic order, matching the tree's branching order.

C.4.7.2 Proof Format

The proof π is a flat stream of opcodes emitted from a post-order, LSB-first traversal of the tree:

Opcode	Semantics	Wire encoding	Size
$S(h)$	Unchanged subtree with hash h	$0x00 \parallel h[32]$	33 bytes
L	New leaf (consumed from sorted batch)	$0x01$	1 byte
$N(d)$	Internal node at bifurcation depth d	$0x02 \parallel d[1]$	2 bytes

In the post-order encoding, the two children of every N node precede it in the stream (left child first, then right child), enabling efficient validation using a stack machine algorithm.

C.4.7.3 Generation

To generate the proof during batch insertion, perform a post-order traversal of the tree regions affected by the insertion. Emit $S(h)$ for every unchanged subtree (carrying its pre-existing hash), L for every newly inserted leaf, and $N(d)$ for every internal node at bifurcation depth d .

C.4.7.4 Function RSMT_VERIFY_CONSISTENCY

Verifies a consistency proof using a stack machine. Each opcode either pushes a pair of hashes (pre-state, post-state) onto the stack, or pops children and pushes a combined parent.

Input:

1. π – consistency proof (sequence of opcodes)
2. $\rho_0 \in \mathbb{H} \cup \{\perp\}$ – root hash before insertion ($\perp$ if tree was empty)
3. $\rho_1 \in \mathbb{H} \cup \{\perp\}$ – root hash after insertion
4. $B = \langle (k_1, v_1), \dots, (k_m, v_m) \rangle$ – batch of inserted key-value pairs

Output: TRUE or FALSE

Computation:

```

function RSMT_VERIFY_CONSISTENCY( $\pi, \rho_0, \rho_1, B$ )
  if  $B = \langle \rangle$  then
    return  $\rho_0 = \rho_1$ 
  end if
  Sort  $B$  by RSMT_SORT_KEY to obtain  $B' = \langle (k'_1, v'_1), \dots, (k'_m, v'_m) \rangle$ 
   $S \leftarrow \langle \rangle$  ▷ Stack of  $(h_{\text{pre}}, h_{\text{post}})$  pairs
   $b \leftarrow 1$  ▷ Batch index into  $B'$ 
  for each opcode  $op$  in  $\pi$  do
    if  $op = S(h)$  then
      push  $(h, h)$  onto  $S$ 
    else if  $op = L$  then
      if  $b > m$  then return FALSE
      end if
      push  $(\perp, \text{RSMT\_LEAF\_HASH}(k'_b, v'_b))$  onto  $S$ 
       $b \leftarrow b + 1$ 
    else if  $op = N(d)$  then
      if  $|S| < 2$  then return FALSE
      end if
      pop  $(r_0, r_1)$  from  $S$  ▷ Right child
      pop  $(\ell_0, \ell_1)$  from  $S$  ▷ Left child
       $h_0 \leftarrow \text{RSMT\_RESOLVE\_PRE\_STATE}(\ell_0, r_0, d)$ 
       $h_1 \leftarrow \text{RSMT\_NODE\_HASH}(\ell_1, r_1, d)$ 
      push  $(h_0, h_1)$  onto  $S$ 
    else
      return FALSE ▷ Unknown opcode
    end if
  end for
  if  $b \neq m + 1 \vee |S| \neq 1$  then
    return FALSE
  end if
  pop  $(\rho'_0, \rho'_1)$  from  $S$ 
  return  $(\rho'_0 = \rho_0) \wedge (\rho'_1 = \rho_1)$ 
end function

```

We define helper function `RSMT_RESOLVE_PRE_STATE` which resolves the pre-insertion hash of a node. When one child is new ($\perp$), the parent node did not exist in the pre-state; only the existing child's subtree occupied that position, so its hash passes through without hashing.

Input:

1. $h_L \in \mathbb{H} \cup \{\perp\}$ – left child pre-state hash ($\perp$ if new)
2. $h_R \in \mathbb{H} \cup \{\perp\}$ – right child pre-state hash ($\perp$ if new)
3. $d \in \mathbb{N}_8$ – bifurcation depth

Output: $h \in \mathbb{H} \cup \{\perp\}$

Computation:

```

function RSMT_RESOLVE_PRE_STATE( $h_L, h_R, d$ )
  if  $h_L = \perp \wedge h_R = \perp$  then
    return  $\perp$                                 ▶ Entire subtree is new
  else if  $h_L = \perp$  then
    return  $h_R$                                 ▶ Only right child existed
  else if  $h_R = \perp$  then
    return  $h_L$                                 ▶ Only left child existed
  else
    return RSMT_NODE_HASH( $h_L, h_R, d$ )        ▶ Both children existed
  end if
end function

```

C.4.8 ZK-Compressed Consistency Proofs

For high-throughput scenarios with large batches, the hash-based consistency proof size of $O(j \cdot d)$ (where j is the batch size and d is the tree depth) may become a bandwidth bottleneck. Cryptographic zero-knowledge proofs (ZK-SNARKs or ZK-STARKs) can compress the consistency proof to constant ($O(1)$) or logarithmic ($O(\log j)$) size while maintaining tight security guarantees.

C.4.8.1 ZK Proof Structure

A ZK-compressed consistency proof proves successful execution of the `rsmt_verify_consistency` algorithm (Appendix C.4.7.4) with the following parameters:

- **Public inputs (instance):** Values that must match the Input Record in the Certification Request:
 - h' – previous SMT root hash (matches $\mathcal{IR}.h'$)
 - h – current SMT root hash (matches $\mathcal{IR}.h$)
- **Private inputs (witness):** unique inserted batch and consistency proof (B, Π_{CP})

C.4.8.2 Verification

When receiving a ZK-compressed consistency proof, the verifier must:

1. Verify that the proof is proving the right statement (hash-based consistency proof verification algorithm)



2. Verify the ZK proof, obtaining authentic public inputs h' , h
3. Confirm these values match the claimed Input Record

D Unicity Token Encoding

This appendix specifies the concrete encodings for the Unicity Token defined in the Execution Layer chapter. All structures are serialized using CBOR (Appendix B) with deterministic encoding rules.

D.1 Constants

The Unicity Mainnet ($\alpha = 1$) MUST use following public constants. Other networks MAY use the same.

- MINT_SUFFIX = SHA-256("TOKENID") – a fixed 32-byte domain separator used to derive the source state hash of mint transactions.
- MINTER_SECRET =
0x495f414d5f554e4956455253414c5f4d494e5445525f464f525f
(ASCII: "I_AM_UNIVERSAL_MINTER_FOR")
– a fixed 26-byte constant used to derive deterministic minting key pairs.
- The minting key pair for a token with network identifier α and token identifier id is derived as:

$$sk_{\text{mint}} = \text{SHA-256}(\text{CBOR}(\text{MINTER_SECRET}, \text{id}))$$

The corresponding public key pk_{mint} is the secp256k1 public key for sk_{mint} . The private key is intentionally public; the key pair exists solely to provide a uniform request interface with the Unicity Service.

D.2 Predicate Encoding

A predicate is a value ν whose authority over a token state is defined by its canonical byte encoding

$$\text{EncPred}(\nu) \in \{0, 1\}^*.$$

Predicate equality means byte equality of canonical encodings:

$$\nu_1 = \nu_2 \iff \text{EncPred}(\nu_1) = \text{EncPred}(\nu_2).$$

The encoding EncPred is implemented as encoding to a CBOR array with three top-level elements:

Index	Type	Description
-------	------	-------------

0	unsigned integer	Predicate engine identifier. Value 1 denotes the built-in engine.
1	byte string	Predicate type code within the engine.
2	byte string or CBOR item	Serialized parameters (encoding depends on the predicate type; see below).

Throughout this section, pk denotes a 33-byte compressed secp256k1 public key (see Sec. D.5), and timestamps $\tau \in \mathbb{T}$ are encoded as CBOR unsigned integers (seconds; same convention as $C^r.t_r$). Hashes are 32 bytes.

D.2.1 Built-in Predicate Type Codes

Code	Predicate	Parameters encoding
0x01	v_{sig} (Signature)	Recipient public key pk .
0x02	v_{burn} (Burn)	reason hash h_{reason} .
0x03	v_{tlock} (Timelock)	CBOR array[2]: [pk (33-byte bstr), $\tau_{\min}$ (uint)].
0x04	v_{p2pkh} (Pay-to-Public-Key-Hash)	public key hash h_{pk} .
0x05	v_{p2sh} (Pay-to-Script-Hash)	predicate hash $h_v = H(\text{CBOR}(v'))$.
0x06	v_{msig} (Multi-signature)	CBOR array of n entries of pk_i .
0x07	v_{tsig} (Threshold-signature)	CBOR array[2]: [k (uint), [$pk_1, \dots, pk_n$]]; $1 \leq k \leq n$.
0x08	v_{htlc} (Hashed Timelock)	CBOR array[4]: [pk , pk' , y (32-byte bstr), $\tau_{\max}$ (uint)].

D.2.2 Unlocking Argument Encoding

The unlocking argument u accompanying a Unicity Service Request (Sec. 3.7) is encoded according to the current owner's predicate type. Signatures are 65 bytes in the $R||S||V$ format defined in Sec. D.5.

Pred.	Unlocking argument encoding
v_{sig}	signature s .
v_{burn}	None; the predicate never evaluates to 1 and no unlocking argument exists.
v_{tlock}	signature s .
v_{p2pkh}	CBOR array[2]: [pk , s].
v_{p2sh}	CBOR array[2]: [$\text{CBOR}(v')$ (bstr), u' (bstr)], where v' is the revealed predicate and u' is the unlocking argument encoded according to v' 's type.
v_{msig}	CBOR array of n signatures $[s_1, \dots, s_n]$, in the same order as the parameter public keys.
v_{tsig}	CBOR array of k entries, each a CBOR array[2]: [s_i , j_i (uint)], where $j_i \in \{1, \dots, n\}$ is the index of the parameter public key used to produce s_i . Indices must be pairwise distinct.
v_{htlc}	CBOR array[2]: [x (bstr), s (65-byte bstr)]. For the claim path x is the preimage with $H(x) = y$; for the refund path x may be empty (h').

D.2.3 Predicate Soundness

The following rules must apply to all predicates, including user-defined, custom programmable predicates.

1. **Canonical encoding.** EncPred MUST be deterministic, injective on valid predicates, and encoded using deterministic CBOR. A conforming decoder MUST reject non-canonical CBOR encodings, trailing bytes, duplicate map keys, invalid field counts, invalid field types, invalid public keys, invalid thresholds, and any type-specific malformed parameters.
2. **Round-trip stability.** For every accepted predicate byte string b :

$$\text{EncPred}(\text{DecodePred}(b)) = b.$$

For every valid predicate object v , repeated evaluations of $\text{EncPred}(v)$ MUST return identical bytes.

3. **Predicate immutability.** The value used in defined protocol steps is the canonical byte string $\text{EncPred}(v)$, not an in-memory object or function. Implementations MUST store and compare this byte string.
4. **State identifier binding.** The state identifier MUST be derived from exactly the same canonical predicate bytes that are authenticated by the unlocking argument:

$$\text{sid} = \text{SHA-256}(\text{CBOR}(\text{EncPred}(v), h_{\text{st}})).$$

Verification of an inclusion proof for a transaction spending state (v, h_{st}) MUST reject unless the predicate and source state hash contained in the certified request are byte-identical to the transaction's current-owner predicate and source state hash.

5. **Deterministic verification relation.** Each predicate type defines a deterministic verification relation

$$\text{VerifyPred}(v, u, h_{\text{st}}, h_{\text{tx}}, C) \in \{0, 1\},$$

where u is the unlocking argument and C is explicit public verification context, such as a certified timestamp if required. For fixed inputs, the result MUST be stable and MUST NOT depend on hidden local state, network calls, randomness, or implementation-specific object identity.

6. **Unlocking Argument canonicity.** Each unlocking argument u MUST have a deterministic CBOR encoding and a predicate-type-specific decoder. The decoder MUST reject malformed, non-canonical, ambiguous, duplicate, out-of-range, or trailing-byte encodings. Re-encoding an accepted unlocking argument MUST reproduce the accepted byte string.
7. **No side effects by unlock argument.** The unlocking argument MAY provide witnesses required by the predicate, such as signatures, public keys for P2PKH, scripts for P2SH, or hash preimages. It MUST NOT alter the current-owner predicate, source state hash, transaction hash, or state identifier. If u reveals a hidden predicate v' for P2SH, verification MUST first check:

$$\text{SHA-256}(\text{EncPred}(v')) = h_v,$$

before even trying to parse v' .

8. **Explicit public context.** Predicates that depend on time, block height, round number, or certificate metadata MUST define precisely which certified public field is used. Such predicates MUST NOT depend on any external data not committed by spending Unicity Certificate.
9. **User-defined predicate safety.** Extension engines MUST define:

(engine, code, params, unlock, VerifyPred)

with canonical encodings for both parameters and unlocking arguments. Extension predicates MUST satisfy the same determinism, immutability, and round-trip requirements as built-in predicates.

These requirements ensure that a relying party, wallet, SDK, and Unicity Service all derive the same state identifier and verify the same authorization statement.

D.2.4 Recipient Field

The *recipient* field in both mint transactions and transactions stores the next owner's predicate ν' encoded as in Sec. D.2.

By convention, a mint transaction uses $\nu' = \nu_{\text{sig}}(\text{pk}')$ (type code `0x01`) for the first owner.

If a sender wishes to commit only to a hash of a complex predicate (hiding its structure until spend time), they can do so explicitly by setting $\nu' = \nu_{\text{p2sh}}(H(\text{CBOR}(\nu'')))$ (Sec. 2.2.2.3); the P2SH predicate is itself encoded as a first-class predicate in the recipient field.

D.3 Transaction Encoding

D.3.1 Mint Transaction

A mint transaction is encoded as a CBOR array with six elements, ordered as $D_{\text{mint}} = (\alpha, \nu', \text{salt}, \text{ty}, \text{reason}, \text{aux}')$:

Index	Type	Description
0	unsigned integer	Network Identifier α (Sec. 3.2)
1	byte string	Recipient: CBOR-encoded first owner's predicate $\text{CBOR}(\nu')$ (Sec. D.2); by convention $\nu' = \nu_{\text{sig}}(\text{pk}')$
2	byte string	Token identifier salt salt (32 bytes).
3	byte string	Token Type ty (32 bytes)
4	byte string	Mint Reason reason (optional, arbitrary length)
5	byte string	Auxiliary Payload aux' (optional, arbitrary length)

The implicit token identifier is derived as $\text{id} = H(\text{salt}, \alpha)$ (Sec. D.4) from the unique salt chosen by the minter.

D.3.2 Transfer Transaction

A transfer transaction (spending a mint or prior transaction output) is encoded as a CBOR array with five elements:

Index	Type	Description
0	byte string	Source state hash h_{st} (32 bytes)
1	byte string	Unlocking Argument: CBOR-encoded $CBOR(u)$ (Sec. D.2); must unlock the recipient predicate of the preceding transaction
2	byte string	Recipient: CBOR-encoded next owner's predicate $CBOR(v')$ (Sec. D.2)
3	byte string	Random value x (32 bytes)
4	byte string	Auxiliary data aux'

D.3.3 Certified Transaction

A certified transaction (mint or transfer) is encoded as a CBOR array with two elements:

Index	Type	Description
0	CBOR array	The transaction
1	CBOR structure	Inclusion proof π

D.3.4 Token

A token is encoded as a CBOR array with two elements:

Index	Type	Description
0	CBOR array	Certified mint transaction
1	CBOR array	Array of certified transactions (may be empty)

D.4 Hash Derivations

All hash derivations use SHA-256 over CBOR-encoded inputs.

D.4.1 Token Identifier

The token identifier is derived from the mint-transaction salt and network identifier:

$$id = \text{SHA-256}(\text{CBOR}(\text{salt}, \alpha))$$

where salt is the 32-byte field at index 2 of D_{mint} and α is the network identifier at index 0.

D.4.2 State Identifier

The state identifier is derived from the predicate and state hash:

$$sid = \text{SHA-256}(\text{CBOR}(v, h_{st}))$$

D.4.3 State Hash

For mint transactions:

$$h_{st} = \text{SHA-256}(\text{CBOR}(id, \text{MINT_SUFFIX})), \quad \text{where } id = \text{SHA-256}(\text{CBOR}(\text{salt}, \alpha))$$

For transfer transactions, the next state hash:

$$h_{st}' = \text{SHA-256}(\text{CBOR}(h_{st}, x))$$

D.4.4 Transaction Hash

For mint transactions:

$$h_{tx} = \text{SHA-256}(\text{CBOR}(\alpha, v', \text{salt}, \text{ty}, \text{reason}, \text{aux}'))$$

For transactions:

$$h_{tx} = \text{SHA-256}(\text{CBOR}(v', x, \text{aux}'))$$

where $\text{CBOR}(\dots)$ denotes the CBOR array encoding of the arguments as specified in Sec. D.3.

D.5 Signature Format

All standard predicates that verify digital signatures (v_{sig} , v_{p2pkh} , v_{tlock} , v_{msig} , v_{tsig} , v_{htlc}) use ECDSA over the secp256k1 curve with the following formats.

- **Public keys:** compressed format, 33 bytes (1-byte prefix $0x02$ or $0x03$, followed by 32-byte x -coordinate).
- **Signatures:** 65 bytes ($R \parallel S \parallel V$), where R and S are 32-byte integers and V is a 1-byte recovery identifier.
- **Signed message:** the predicate message hash $m = \text{SHA-256}(\text{CBOR}(h_{\text{st}}, h_{\text{tx}}))$, as defined in Sec. 2.2.1.

D.6 Split Proof Encoding

D.6.1 Split Reason

A split reason is encoded as a CBOR array with two elements:

Index	Type	Description
0	CBOR structure	The burned source token (complete token encoding)
1	CBOR array	Array of split reason proofs (one per asset type)

D.6.2 Split Reason Proof

A split reason proof is encoded as a CBOR array with three elements:

Index	Type	Description
0	byte string	Asset identifier aid
1	CBOR structure	Aggregation tree path (Sparse Merkle Tree path to the aggregation root)
2	CBOR structure	Asset sum tree path (Sparse Merkle Sum Tree path proving value allocation)

E Data Structures (illustrative)

Below is an informal illustration on how Unicity data structures integrate the service layers and to HotStuff consensus primitive; in ABNF format¹². (Distributed, dynamic BFT Core.)

```
; Unicity Protocol Data Structures (ABNF)

; =====
; 1. BFT CORE (shared data structures)

UC = IR TechRecordHash ShardTreeCertificate UnicityTreeCertificate UnicitySeal ; UC = (IR, ht, Cshard, Cuni, Cr)
IR = RoundNumber Epoch PreviousHash Hash Time ; IR = (n, e, h', h, t)
ShardTreeCertificate = ShardIdentifier *SiblingHash ; Cshard = (σ; h1s, ..., h|σ|s)
UnicityTreeCertificate = PartitionIdentifier PartitionDescriptionHash *HashStep
HashStep = PartitionIdentifier SiblingHash ; Cuni = (β, dhash; (β2, h2), ..., (βℓ, hℓ)))

UnicitySeal = Version NetworkIdentifier RootPartitionRoundNumber Epoch Timestamp PreviousHash Hash
              *Signatures ; Cr = (α, nr, er, tr, r-, r'; s)
              ; where |Signatures| = quorumThreshold > 2f

; - Statistical and Technical Records -
StatisticalRecord = NumRounds SumRoundSize SumStateSize MaxRoundSize MaxStateSize ; SR = (ne, ℓB, ℓS, ℓB, ℓS)
TechRecord = Round Epoch LeaderID SRHash ; TE = (nr, er, vℓ, hsr)

ShardInfo = Round PreviousHash PrevStatisticalRecord StatisticalRecord *ValidatorIdentifier
            LeaderIdentifier PrevUC ; (n, h-, SR-, SR, V, vℓ, UC-)

; Evolving trust base:
UnicityTrustBase = Version *UnicityTrustBaseEntry
UnicityTrustBaseEntry = NetworkIdentifier Epoch *(NodeID Pubkey Stake) QuorumThreshold StateHash
                      ChangeRecordHash PreviousEntryHash *Signatures

; =====
; BFT CORE (internal data structures)

; - Internal Consensus Structures -
LedgerCommitInfo = UnicitySeal | UnicityTrustBaseEntry ; seal without signatures
QC = VoteInfo LedgerCommitInfo *Signatures

VoteInfo = RoundInfo
RoundInfo = RoundNumber Epoch Timestamp ParentRoundNumber CurrentRootHash | UnicityTrustBaseEntryHash

; - BFT Core Messages -
VoteMsg = VoteInfo LedgerCommitInfo HighQC Author Signature

ProposalMsg = BlockData [LastRoundTc] Signature
BlockData = Author Round Epoch Timestamp Payload AncestorQC
Payload = *IRChangeReq | RPEpochChangeReq
IRChangeReq = PartitionIdentifier CertReason *CR [EpochChangeJustification] SenderSignature
              ; presence of justification ==> epoch++
RPEpochChangeReq = Epoch *(NodeID Pubkey Stake) QuorumThreshold StateHash ChangeRecordHash
                  PreviousEntryHash SenderSignature [ChangeRecord]

TimeoutMsg = Timeout Author Signature [LastTC]
              ; If HighQC is not from prev. round then there must be TC of prev,
              ; justifying the incremented round number
```

¹²<https://tools.ietf.org/html/rfc5234>

```

Timeout = Epoch Round HighQC ; HighQC - highest known Quorum Certificate to the validator
TC = Timeout *Signatures ; 2f+1 Signatures ; TC - Timeout Certificate

CertReason = 'quorum' | 'quorum-not-possible' | 't2-timeout' ; flags in appropriate encoding

; - Recovery Messages -
GetStateMsg = NodeId ; id of the validator requesting the state
StateMsg = *UC CommittedHead BlockNode UnicityTrustBaseEntry
CommittedHead = BlockNode = RecoveryBlock
RecoveryBlock = BlockData *InputData QC CommitQC
InputData = PartitionIdentifier Shard IR Sdrh

; =====
; 3. BFT CORE (protocols)

CR = NetworkIdentifier PartitionIdentifier Shard NodeIdentifier IR BatchSize StateSize Signature
[ConsistencyProof]
; Certification Request  $CR = \langle \alpha, \beta, \sigma, v; IR, \ell_B, \ell_S; s; [\pi_{CP}] \rangle$ 
; Returned UC can be repeated cert for prevStateTreeHash which triggers next attempt using different
; leader
; a validator can have multiple pending requests extending the same hash; latest one is identified using
; IR.n

CReS = NetworkIdentifier PartitionIdentifier ShardIdentifier UC TechRecord [UnicityTrustBaseEntry]
; Certification Response

; Subscription - Subscribe to CReS message feed in order to obtain the latest UC for synchronization
SubscriptionMsg = NetworkIdentifier PartitionIdentifier Shard NodeIdentifier signature
; this request provides or updates validator connection parameters at
; transport layer so that Root can return CReS messages

; Trust Base Distribution protocol
GetTrustBaseMsg = NetworkIdentifier EpochStart [EpochEnd]
TrustBaseResponse = *UnicityTrustBaseEntry

; =====
; 4. AGGREGATION LAYER

; The aggregation layer uses a Radix Sparse Merkle Tree (RSMT, Appendix C.4).
; Leaf-anchored: hashes commit to full 256-bit key and bifurcation depth.
; LSB-first bit addressing.

StateIdentifier = Bytes32 ; sid =  $H(\text{CBOR}(v, h_{st}))$  - SMT leaf key
LeafData = TransactionHash ;  $h_{tx}$  - SMT leaf value
SMTLeaf = StateIdentifier LeafData ;  $L = (\text{sid}, h_{tx})$ 

InclusionCertificate = Bitmap *SiblingHash ;  $C^{inc} = (\text{bitmap}, \langle s_1, \dots, s_n \rangle)$ 
; bitmap: 32 bytes (256 bits); bit  $d$  set iff sibling needed at depth  $d$ 
; siblings:  $n = \text{bitcount}(\text{bitmap})$  hashes, leaf-to-root order
; Wire:  $\text{bitmap}[32] || s_1[32] || \dots || s_n[32]$ 

NonInclusionCertificate = Key Hash Bitmap *SiblingHash ;  $C^{exc} = (k_\ell, h_\ell, \text{bitmap}, \langle s_1, \dots, s_n \rangle)$ 
;  $k_\ell$ : key of neighbouring leaf
;  $h_\ell$ : hash of neighbouring leaf (=  $\text{RSMT\_LEAF\_HASH}(k_\ell, v_\ell)$ )
; bitmap, siblings: same semantics as InclusionCertificate
; Wire:  $k_\ell[32] || h_\ell[32] || \text{bitmap}[32] || s_1[32] || \dots || s_n[32]$ 

InclusionProof = StateIdentifier LeafData InclusionCertificate UC ;  $\pi^{inc} = (\text{sid}, h_{tx}, C^{inc}, UC)$ 
; Proves that sid EXISTS in the SMT with value  $h_{tx}$ 

NonInclusionProof = NonInclusionCertificate UC ;  $\pi^{exc} = (C^{exc}, UC)$ 
; Proves that a queried sid does NOT exist in the SMT

ConsistencyProof = *ConsistencyProofOp | ZKProof
;  $\pi_{CP}$  - proves correct append-only SMT manipulation
ConsistencyProofOp = OpS | OpL | OpN ; post-order flat opcode stream
OpS = %x00 Hash ;  $S(h)$ : unchanged subtree, 33 bytes
OpL = %x01 ; L: new leaf (consumed from sorted batch), 1 byte
OpN = %x02 Depth ;  $N(d)$ : internal node at bifurcation depth  $d$ , 2 bytes
Depth = OCTET ;  $d \in \{0, \dots, 255\}$ 
Bitmap = Bytes32 ; 256-bit branching-depth vector
ZKProof = Bytes ; Opaque ZK proof (SNARK/STARK)

```

```

; =====
; 5. EXECUTION LAYER

; - Predicates -
Predicate = Engine Code Params ;  $v = (\text{engine}, \text{code}, \text{params})$  - CBOR array[3]
; Engine: unsigned integer (1 = BUILT_IN)
; Code: byte string, built-in type code (see below)
; Params: byte string or CBOR item (encoding depends on predicate type)

; Built-in predicate type codes and parameter encodings:
; 0x01 sig - Signature: pk (33-byte bstr)
; 0x02 burn - Burn: reason hash (32-byte bstr)
; 0x03 tlock - Timelock: CBOR array[2]: [pk,  $\tau_{\min}$  (uint)]
; 0x04 p2pkh - Pay-to-Public-Key-Hash:  $h_{pk}$  (32-byte bstr)
; 0x05 p2sh - Pay-to-Script-Hash:  $h_{pred} = H(\text{CBOR}(\text{pred}'))$  (32-byte bstr)
; 0x06 msig - Multi-signature: CBOR array of  $pk_i$ 
; 0x07 tsig - Threshold-signature: CBOR array[2]: [k (uint), [ $pk_1, \dots, pk_n$ ]]
; 0x08 htlc - Hashed Timelock: CBOR array[4]: [pk, pk', y (32-byte bstr),  $\tau_{\max}$  (uint)]

; - Unlocking Arguments -
; Encoding of u depends on current owner's predicate type:
; sig / tlock : signature s (65 bytes, R||S||V)
; burn : none (predicate never evaluates to 1)
; p2pkh : CBOR array[2]: [pk, s]
; p2sh : CBOR array[2]: [CBOR(pred') (bstr), u' (bstr)]
; msig : CBOR array of n signatures [ $s_1, \dots, s_n$ ]
; tsig : CBOR array of k entries, each CBOR array[2]: [ $s_i$ ,  $j_i$  (uint)]
; htlc : CBOR array[2]: [x (bstr), s (65-byte bstr)]

; - Transactions -
MintTransaction = NetworkIdentifier Recipient Salt TokenType MintReason Data
;  $T_0 = (\alpha, v', \text{salt}, \text{ty}, \text{reason}, \text{aux}')$  - CBOR array[6]
; Token identifier is derived:  $\text{id} = H(\text{salt}, \alpha)$  (see Hash Derivations)

TransferTransaction = SourceStateHash UnlockingArgument Recipient StateMask Data
;  $T = (h_{st}, u, v', x, \text{aux}')$  - CBOR array[5]

CertifiedMintTransaction = MintTransaction InclusionProof ;  $(T_0, \pi)$  - CBOR array[2]
CertifiedTransferTransaction = TransferTransaction InclusionProof ;  $(T, \pi)$  - CBOR array[2]

Token = CertifiedMintTransaction * CertifiedTransferTransaction
;  $\mathcal{L} = (T_0^{\text{cert}}, T_1^{\text{cert}}, \dots, T_n^{\text{cert}})$  - CBOR array[2]: [genesis, [transfers...]]

; - Fields -
Salt = Bytes32 ; salt - 32-byte minter-chosen salt, required to be unique among the minter's choices
; used to derive token identifier  $\text{id} = H(\text{salt}, \alpha)$ 
TokenType = Bytes32 ; ty - token type classifier (32 bytes)
MintReason = Bytes ; reason - mint reason (justification)
SourceStateHash = Hash ;  $h_{st}$  - current state hash (32 bytes)
UnlockingArgument = Bytes ; CBOR(u) - CBOR-encoded unlocking argument (see above)
Recipient = Bytes ; CBOR( $v'$ ) - CBOR-encoded next owner predicate
; (use  $v_{p2sh}$  to commit only to a hash)

StateMask = Bytes32 ;  $x \xleftarrow{\$} \{0,1\}^{256}$  - random value for state derivation
Data = Bytes ; aux' - auxiliary payload (optional, arbitrary length)
TransactionHash = Hash ;  $h_{tx} = H(D)$  - hash of transaction data
UnlockScript = Bytes ; u - unlocking argument (e.g. signature)

; - State -
StateId = Hash ;  $\text{sid} = H(\text{CBOR}(v, h_{st}))$  - key in Unicity Service registry

; - Certification -
CertificationData = Recipient SourceStateHash TransactionHash UnlockingArgument
;  $Q = (v, h_{st}, h_{tx}, u)$  - CBOR array[4], submitted to Unicity Service

; - Hash Derivations -
;  $\text{id} = H(\text{salt}, \alpha)$  - token identifier (derived; globally unique across networks)
;  $h_{st} = H(\text{id}, \text{MINT\_SUFFIX})$  - mint source state hash
;  $h'_{st} = H(h_{st}, x)$  - next state hash
;  $h_{tx} = H(\alpha, \text{recipient}, \text{salt}, \text{ty}, \text{reason}, \text{aux}')$  - mint transaction hash (the value in Unicity Service if seen as key-value store)
;  $h_{tx} = H(\text{recipient}, x, \text{aux}')$  - transfer transaction hash (value in Unicity Service)

```



```
; - Fungible Token Structures -  
Asset = AssetId Value ; (aid,v) - CBOR array[2]  
AssetId = Bytes ; asset type identifier  
Value = Bytes ; non-negative integer (bigint-encoded)  
  
SplitReason = Token *SplitReasonProof ; CBOR array[2]: [burned source token, [proofs...]]  
SplitReasonProof = AssetId AggregationPath AssetTreePath ; (aid, $\pi_{\text{agg}}$ , $\pi_{\text{sum}}$ ) - CBOR array[3]  
AggregationPath = SparseMerkleTreePath ; SMT path to aggregation root  $h_{\text{agg}}$   
AssetTreePath = SparseMerkleSumTreePath ; SMST path proving value allocation
```


Index

C^{exc} (Non-Inclusion Certificate), 29
 C^{inc} (Inclusion Certificate), 29
 C^r (unicity seal), 31
 C^{shard} (shard tree certificate), 29
 C^{uni} (unicity tree certificate), 30
 IR (input record), 37
 Q (unicity service request), 27
 UC (unicity certificate), 32
 α (network identifier), 24
 β (partition identifier), 24
 $\mathbb{C}$ (partition identifier type), 24
 CD (partition description), 25
 $\mathcal{T}$ (unicity trust base), 78
 sid (state identifier), 11
 π^{exc} (non-inclusion proof), 33
 π^{inc} (inclusion proof), 32
 π_{CP} (consistency proof), 34
 τ , 14
 e (epoch number), 39

aggregation layer node, 24
ASERT, 86
asset, 19

BFT Core
 BFT Core state, 39
 distributed, 65
 functional description, 40
 monolithic, 64
block reward, 87
burn transaction, 10, 20

certification request (CR), 39
certification response (CReS), 40
chain reorganization, 84
consistency proof, 25, 34

epoch, 72
exclusion proof, 33

Finality Gadget, 91
Finality Gadget Partition, 91
functions
 CompShardTreeCert, 30
 CreateShardTree, 38
 CreateShardTreeCert, 30
 CreateUnicityTree, 39
 CreateUnicityTreeCert, 31
 VerifyConsistencyProof, 35
 VerifyInclusionProof, 33
 VerifyNonInclusionProof, 33
 VerifyToken, 13
 VerifyUnicityCert, 32

Inclusion Certificate, 29
 RSMT, 111
inclusion proof, 32

longest chain rule, 84

maximalist verification, 94
mint justification, 19
mint reason, 19
mint transaction, 10
 structure, 11
MINT_SUFFIX, 12

Non-Inclusion Certificate, 29
 RSMT, 112
non-inclusion proof, 33

partition, 24
partition description, 25
PoW blockchain, 84
PoW chain, 83
practical verification, 94
predicate, 14
 burn (v_{burn}), 16
 framework, 14
 hashed timelock (v_{htlc}), 17

- multi-signature (v_{msig}), 15
- P2PKH (v_{p2pkh}), 15
- P2SH (v_{p2sh}), 15
- signature (v_{sig}), 15
- threshold-signature (v_{tsig}), 16
- timelock (v_{tlock}), 16
- Radix Sparse Merkle Tree, 107
- RandomX, 84
- repeat UC, 43
- RSMT, 29
- shard, 24
 - $\mathcal{SH}$ (sharding scheme), 24
 - shard info, 38
- shard tree, 38
- shard tree certificate, 29
- SMT, 29
- snapshot publishing, 87
- Sparse Merkle Sum Tree, 19, 104
- sparse merkle tree, 29
- split transaction, 10
- state identifier, 11
- state mask, 11
- statistical record (SR), 37
- technical record (TE), 37
- time extraction, 14
- token, 10
 - identifier, 12
 - splitting, 19
 - state, 11
 - structure, 11
 - verification, 13
- token compression, 10
- transfer transaction, 10
 - structure, 12
- unicity seal, 31
- Unicity Service
 - execution layer interaction, 17
- Unicity Tree, 39
- unicity tree certificate, 30
 - computation, 31